From Ratings to Recommendations

Understanding the Algorithms Behind Personalized Recommendations

---

Dr. Sowmini Devi Veeramachaneni
Assistant Professor, Mahindra University

Workshop on Recommender Systems

GITAM · 12 October 2026

Outline

1. From user behaviour to recommendations
2. Collaborative filtering: similarity, neighbourhoods, prediction
3. Evaluating a CF recommender
4. Matrix factorization: latent factor models
5. Beyond the basics: real systems and cross-domain recommendation

1. From User Behaviour to Recommendations

Have you ever wondered...

- How does **YouTube** predict which video you will watch next?

Have you ever wondered...

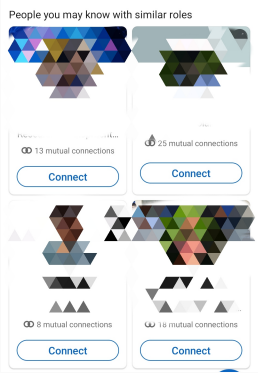
- How does **YouTube** predict which video you will watch next?
- How does **Amazon** know which products you may like?

Have you ever wondered...

- How does **YouTube** predict which video you will watch next?
- How does **Amazon** know which products you may like?
- How does **Facebook** suggest friends?

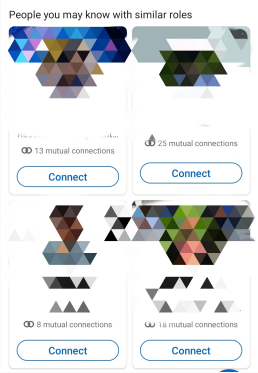
Have you ever wondered...

- How does **YouTube** predict which video you will watch next?
- How does **Amazon** know which products you may like?
- How does **Facebook** suggest friends?
- How does **LinkedIn** know the people you might know?



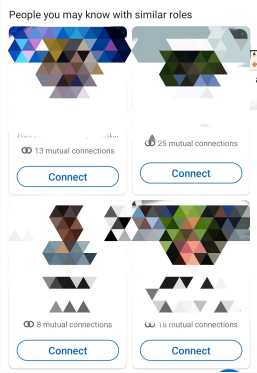
Have you ever wondered...

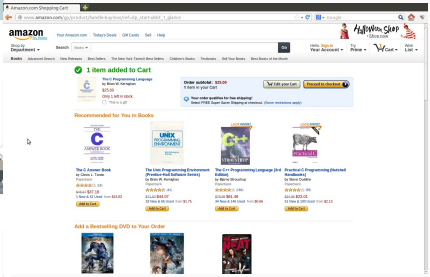
- How does **YouTube** predict which video you will watch next?
- How does **Amazon** know which products you may like?
- How does **Facebook** suggest friends?
- How does **LinkedIn** know the people you might know?
- How does **Swiggy/Zomato** decide which restaurants to show first?



Have you ever wondered. . .

- How does **YouTube** predict which video you will watch next?
- How does **Amazon** know which products you may like?
- How does **Facebook** suggest friends?
- How does **LinkedIn** know the people you might know?
- How does **Swiggy/Zomato** decide which restaurants to show first?





Recommendations are everywhere



Platform	What is recommended	Where you see it	Main signal
Netflix	Films and series	“Because you watched . . . ” rows	Viewing history

Recommendations are everywhere



Platform	What is recommended	Where you see it	Main signal
Netflix	Films and series	"Because you watched ..." rows	Viewing history
Amazon	Products	"Customers who bought this ..."	Purchases, views

Recommendations are everywhere



Platform	What is recommended	Where you see it	Main signal
Netflix	Films and series	"Because you watched ..." rows	Viewing history
Amazon	Products	"Customers who bought this ..."	Purchases, views
YouTube	Next video	Home feed, up-next panel	Watch time, clicks

Recommendations are everywhere



Platform	What is recommended	Where you see it	Main signal
Netflix	Films and series	"Because you watched . . . " rows	Viewing history
Amazon	Products	"Customers who bought this . . . "	Purchases, views
YouTube	Next video	Home feed, up-next panel	Watch time, clicks
Spotify	Songs, playlists	"Discover Weekly"	Listening, skips

Recommendations are everywhere



Platform	What is recommended	Where you see it	Main signal
Netflix	Films and series	"Because you watched . . . " rows	Viewing history
Amazon	Products	"Customers who bought this . . . "	Purchases, views
YouTube	Next video	Home feed, up-next panel	Watch time, clicks
Spotify	Songs, playlists	"Discover Weekly"	Listening, skips
Swiggy / Zomato	Restaurants, dishes	Ordering of the home screen	Orders, location, time

Recommendations are everywhere



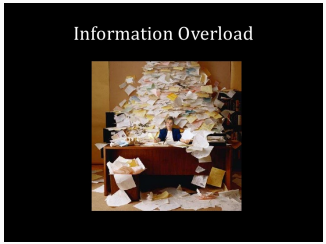
Platform	What is recommended	Where you see it	Main signal
Netflix	Films and series	"Because you watched . . . " rows	Viewing history
Amazon	Products	"Customers who bought this . . . "	Purchases, views
YouTube	Next video	Home feed, up-next panel	Watch time, clicks
Spotify	Songs, playlists	"Discover Weekly"	Listening, skips
Swiggy / Zomato	Restaurants, dishes	Ordering of the home screen	Orders, location, time

Different products, **the same problem**: from past behaviour, rank the items each user has not seen yet.

We are leaving the Information Age...

"We are leaving the Information Age and entering the Recommendation Age."

— Chris Anderson, *The Long Tail*

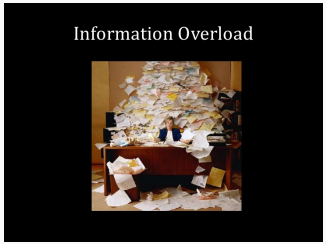


Information overload

We are leaving the Information Age...

"We are leaving the Information Age and entering the Recommendation Age."

— Chris Anderson, *The Long Tail*



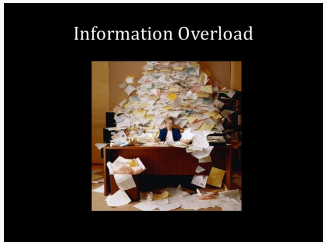
Information overload

- 500+ hours of video uploaded to YouTube every minute

We are leaving the Information Age...

"We are leaving the Information Age and entering the Recommendation Age."

— Chris Anderson, *The Long Tail*



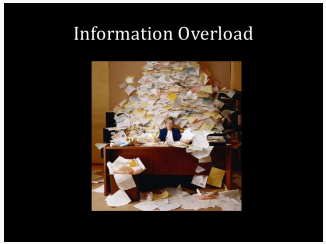
Information overload

- 500+ hours of video uploaded to YouTube every minute
- Millions of products, songs, articles

We are leaving the Information Age...

"We are leaving the Information Age and entering the Recommendation Age."

— Chris Anderson, *The Long Tail*



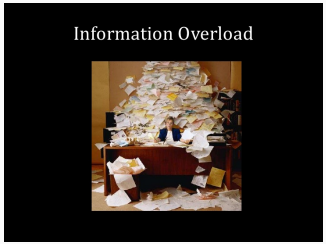
Information overload

- 500+ hours of video uploaded to YouTube every minute
- Millions of products, songs, articles
- Users cannot search for what they do not know exists

We are leaving the Information Age...

"We are leaving the Information Age and entering the Recommendation Age."

— Chris Anderson, *The Long Tail*



Information overload

- 500+ hours of video uploaded to YouTube every minute
- Millions of products, songs, articles
- Users cannot search for what they do not know exists

Business value

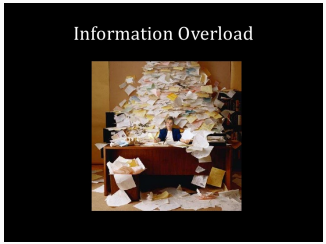
- Netflix: about **2/3** of hours streamed come from recommendations

Gomez-Uribe & Hunt, *The Netflix Recommender System: Algorithms, Business Value, and Innovation*, ACM TMIS 2016.

We are leaving the Information Age...

"We are leaving the Information Age and entering the Recommendation Age."

— Chris Anderson, *The Long Tail*



Information overload

- 500+ hours of video uploaded to YouTube every minute
- Millions of products, songs, articles
- Users cannot search for what they do not know exists

Business value

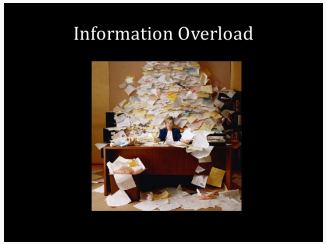
- Netflix: about **2/3** of hours streamed come from recommendations
- Google News: **38%** more click-throughs

Gomez-Uribe & Hunt, *The Netflix Recommender System: Algorithms, Business Value, and Innovation*, ACM TMIS 2016.

We are leaving the Information Age...

"We are leaving the Information Age and entering the Recommendation Age."

— Chris Anderson, *The Long Tail*



Information overload

- 500+ hours of video uploaded to YouTube every minute
- Millions of products, songs, articles
- Users cannot search for what they do not know exists

Business value

- Netflix: about **2/3** of hours streamed come from recommendations
- Google News: **38%** more click-throughs
- Amazon: around **35%** of sales

Gomez-Uribe & Hunt, *The Netflix Recommender System: Algorithms, Business Value, and Innovation*, ACM TMIS 2016.

What a recommender actually does

Recommendation = predicting preference
under incomplete information

What a recommender actually does

Recommendation = predicting preference
under incomplete information

- Users interact with only a **tiny fraction** of items

What a recommender actually does

Recommendation = predicting preference
under incomplete information

- Users interact with only a **tiny fraction** of items
- We observe *some* preferences and must infer the rest

What a recommender actually does

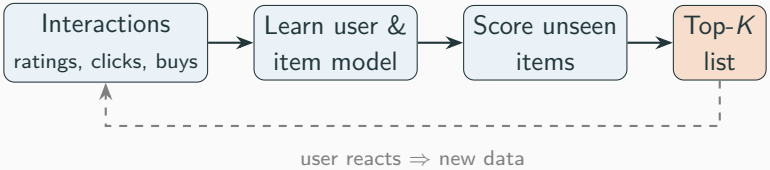
Recommendation = predicting preference under incomplete information

- Users interact with only a **tiny fraction** of items
- We observe *some* preferences and must infer the rest
- Then **rank**: show the items with the highest predicted preference

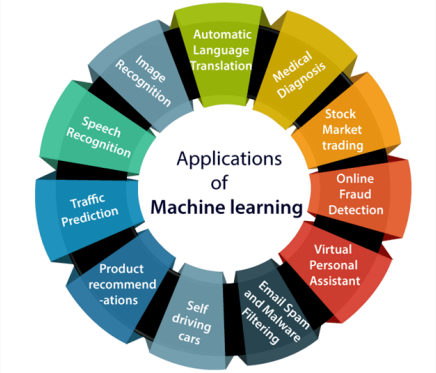
What a recommender actually does

Recommendation = predicting preference under incomplete information

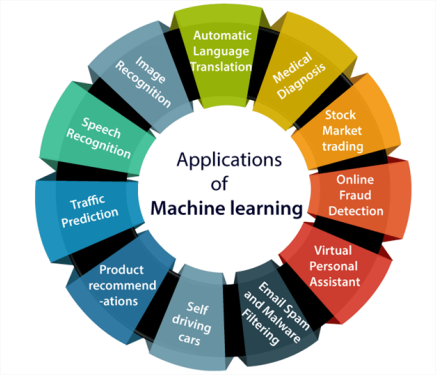
- Users interact with only a **tiny fraction** of items
- We observe *some* preferences and must infer the rest
- Then **rank**: show the items with the highest predicted preference

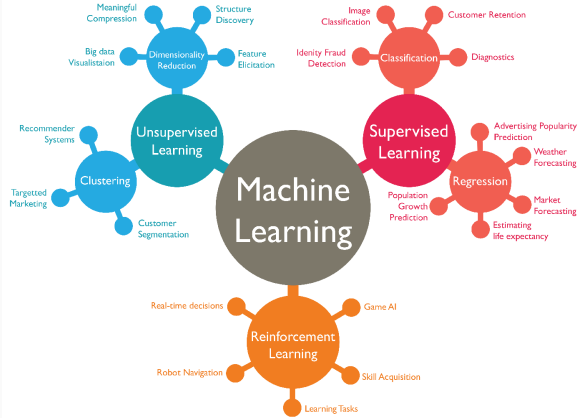


Recommendation is one of the biggest applications of machine learning



Recommendation is one of the biggest applications of machine learning





https://wordstream-files-prod.s3.amazonaws.com/s3fs-public/machine-learning.png

Tom Mitchell's definition

A program learns from experience E w.r.t. task T and performance P if its performance at T, measured by P, improves with E.

Tom Mitchell's definition

A program learns from experience E w.r.t. task T and performance P if its performance at T, measured by P, improves with E.

- T: predict ratings / rank items for a user

Where recommender systems sit in machine learning

Tom Mitchell's definition

A program learns from experience E w.r.t. task T and performance P if its performance at T, measured by P, improves with E.

- T: predict ratings / rank items for a user
- P: RMSE, MAE, Precision@K, clicks in an A/B test

Where recommender systems sit in machine learning

Tom Mitchell's definition

A program learns from experience E w.r.t. task T and performance P if its performance at T, measured by P, improves with E.

- T: predict ratings / rank items for a user
- P: RMSE, MAE, Precision@K, clicks in an A/B test
- E: the history of user–item interactions

Where recommender systems sit in machine learning

Tom Mitchell's definition

A program learns from experience E w.r.t. task T and performance P if its performance at T, measured by P, improves with E.

- T: predict ratings / rank items for a user
- P: RMSE, MAE, Precision@K, clicks in an A/B test
- E: the history of user–item interactions

Is it supervised or unsupervised?

Where recommender systems sit in machine learning

Tom Mitchell's definition

A program learns from experience E w.r.t. task T and performance P if its performance at T, measured by P, improves with E.

- T: predict ratings / rank items for a user
- P: RMSE, MAE, Precision@K, clicks in an A/B test
- E: the history of user–item interactions

Is it supervised or unsupervised?

- Like **supervised**: historical user interactions (clicks, views, ratings) act as explicit or implicit ground-truth labels for prediction

Where recommender systems sit in machine learning

Tom Mitchell's definition

A program learns from experience E w.r.t. task T and performance P if its performance at T, measured by P, improves with E.

- T: predict ratings / rank items for a user
- P: RMSE, MAE, Precision@K, clicks in an A/B test
- E: the history of user–item interactions

Is it supervised or unsupervised?

- Like **supervised**: historical user interactions (clicks, views, ratings) act as explicit or implicit ground-truth labels for prediction
- Like **unsupervised**: we discover hidden preferences and latent item characteristics (e.g., matrix factorization factors) without manual tagging

Where recommender systems sit in machine learning

Tom Mitchell's definition

A program learns from experience E w.r.t. task T and performance P if its performance at T, measured by P, improves with E.

- T: predict ratings / rank items for a user
- P: RMSE, MAE, Precision@K, clicks in an A/B test
- E: the history of user–item interactions

Is it supervised or unsupervised?

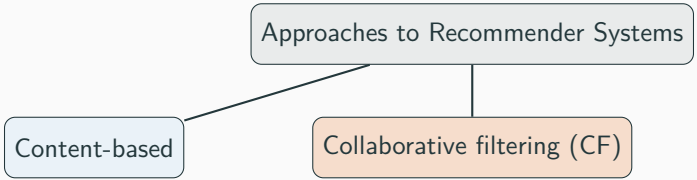
- Like **supervised**: historical user interactions (clicks, views, ratings) act as explicit or implicit ground-truth labels for prediction
- Like **unsupervised**: we discover hidden preferences and latent item characteristics (e.g., matrix factorization factors) without manual tagging
- Dynamic feedback loop: modern systems use **reinforcement learning** to optimize sequential decisions and long-term session rewards rather than single predictions

Approaches to Recommender Systems

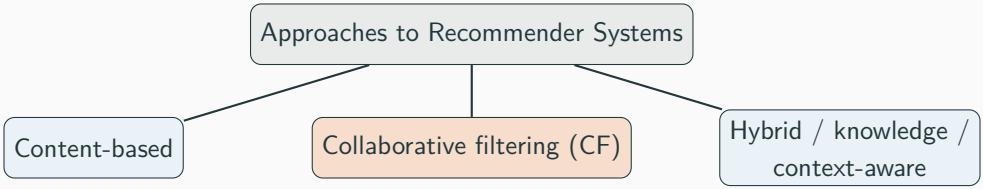
Approaches to recommender systems



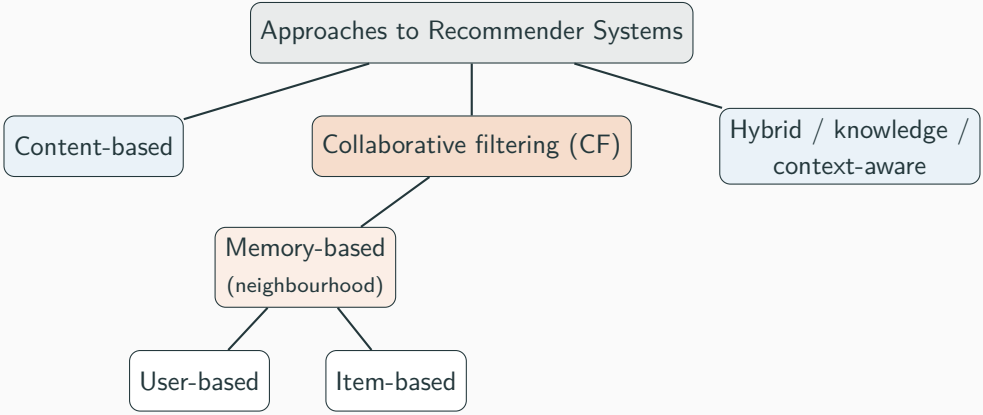
Approaches to recommender systems



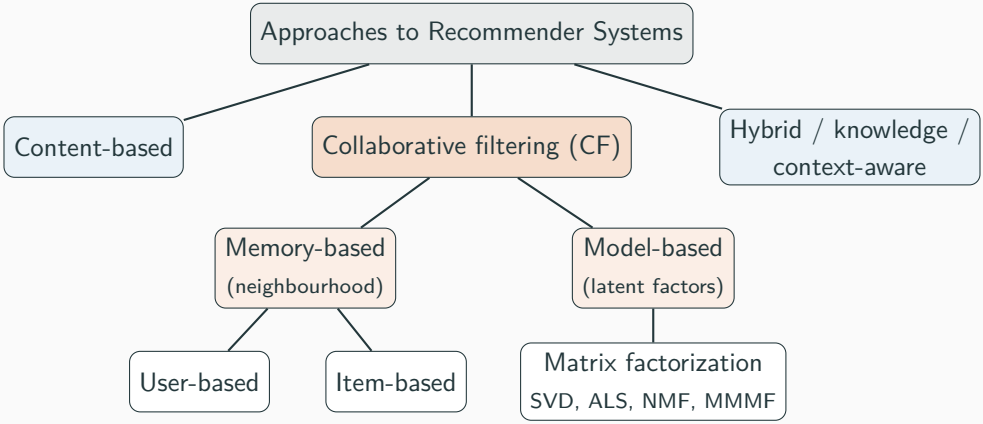
Approaches to recommender systems



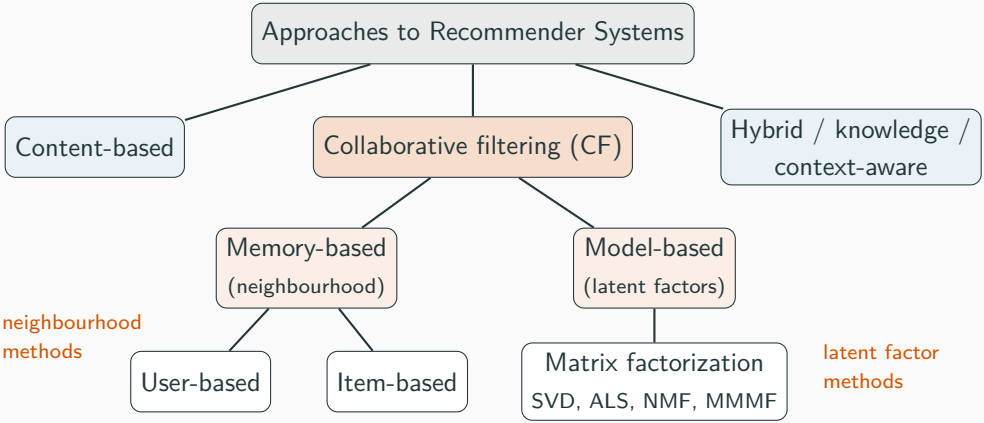
Approaches to recommender systems



Approaches to recommender systems



Approaches to recommender systems



Types of recommender systems

Type	Core idea	Example
Popularity-based	Recommend what most people like	Trending, Top-10 charts

Types of recommender systems

Type	Core idea	Example
Popularity-based	Recommend what most people like	Trending, Top-10 charts
Content-based	Items similar in *features* to what you liked	Same genre / director

Types of recommender systems

Type	Core idea	Example
Popularity-based	Recommend what most people like	Trending, Top-10 charts
Content-based	Items similar in *features* to what you liked	Same genre / director
Collaborative filtering	Learn from *behaviour* of similar users	"People like you watched..."

Types of recommender systems

Type	Core idea	Example
Popularity-based	Recommend what most people like	Trending, Top-10 charts
Content-based	Items similar in *features* to what you liked	Same genre / director
Collaborative filtering	Learn from *behaviour* of similar users	"People like you watched..."
Knowledge-based	Explicit constraints & rules	Cars, houses, loans

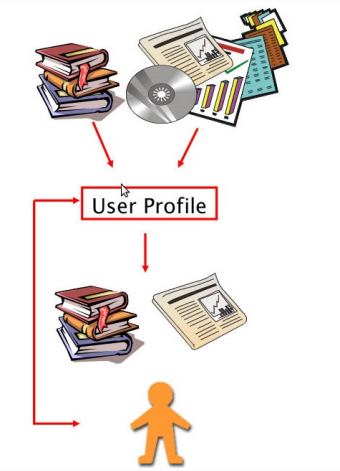
Types of recommender systems

Type	Core idea	Example
Popularity-based	Recommend what most people like	Trending, Top-10 charts
Content-based	Items similar in *features* to what you liked	Same genre / director
Collaborative filtering	Learn from *behaviour* of similar users	"People like you watched..."
Knowledge-based	Explicit constraints & rules	Cars, houses, loans
Context-aware	Add time, location, device	Lunch spots near you at 1 pm

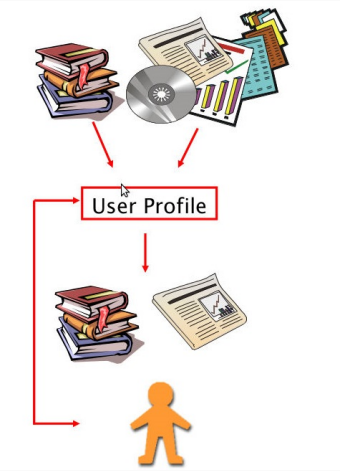
Types of recommender systems

Type	Core idea	Example
Popularity-based	Recommend what most people like	Trending, Top-10 charts
Content-based	Items similar in *features* to what you liked	Same genre / director
Collaborative filtering	Learn from *behaviour* of similar users	"People like you watched..."
Knowledge-based	Explicit constraints & rules	Cars, houses, loans
Context-aware	Add time, location, device	Lunch spots near you at 1 pm
Hybrid	Combine several of the above	Netflix, YouTube

Content-based filtering: building a user profile

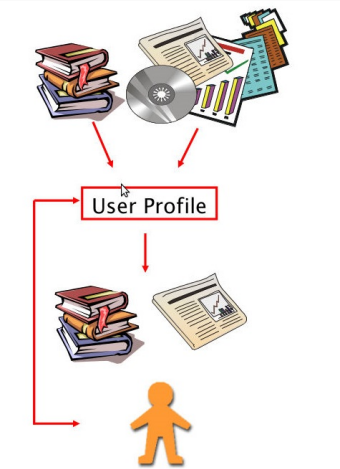


Content-based filtering: building a user profile



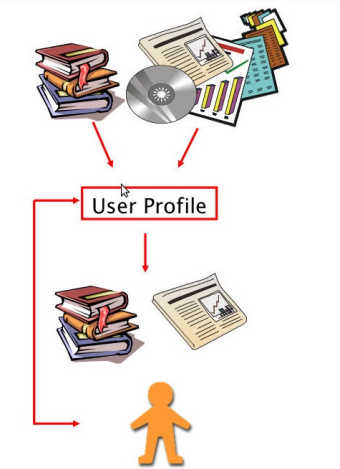
1. **Describe every item by its features**: genre, author, keywords, director

Content-based filtering: building a user profile



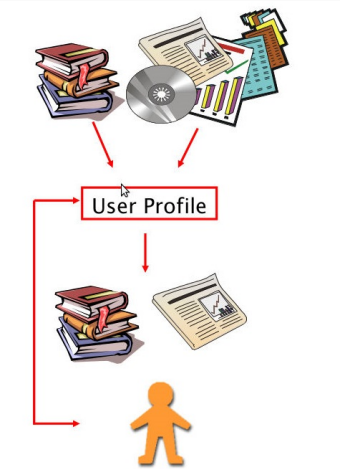
1. **Describe every item by its features**: genre, author, keywords, director
2. **Build the user profile** from the items the user liked
   e.g. profile = (sci-fi 0.8, drama 0.3, comedy 0.1)

Content-based filtering: building a user profile



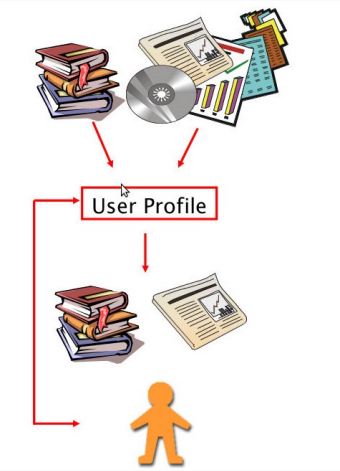
1. **Describe every item by its features**: genre, author, keywords, director
2. **Build the user profile** from the items the user liked
   e.g. profile = (sci-fi 0.8, drama 0.3, comedy 0.1)
3. **Score each new item** by how well its features match the profile

Content-based filtering: building a user profile



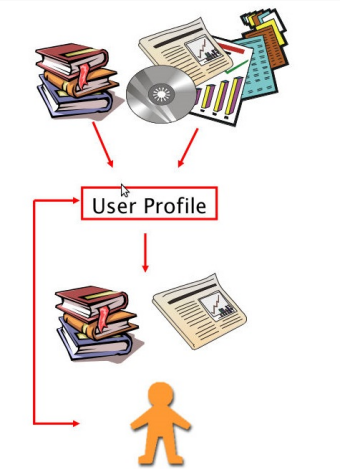
1. **Describe every item by its features**: genre, author, keywords, director
2. **Build the user profile** from the items the user liked
   e.g. profile = (sci-fi 0.8, drama 0.3, comedy 0.1)
3. **Score each new item** by how well its features match the profile
4. **Recommend** the best-matching items

Content-based filtering: building a user profile



1. **Describe every item by its features**: genre, author, keywords, director
2. **Build the user profile** from the items the user liked
   e.g. profile = (sci-fi 0.8, drama 0.3, comedy 0.1)
3. **Score each new item** by how well its features match the profile
4. **Recommend** the best-matching items
5. **Update the profile** with the user's reaction (the loop on the left)

Content-based filtering: building a user profile



1. **Describe every item by its features**: genre, author, keywords, director
2. **Build the user profile** from the items the user liked
   e.g. profile = (sci-fi 0.8, drama 0.3, comedy 0.1)
3. **Score each new item** by how well its features match the profile
4. **Recommend** the best-matching items
5. **Update the profile** with the user's reaction (the loop on the left)

Uses only *this* user's history and the item descriptions — no other users are needed.

Collaborative filtering: from word-of-mouth to algorithms

- Humans have always asked friends with *similar taste*

Collaborative filtering: from word-of-mouth to algorithms

- Humans have always asked friends with *similar taste*
- CF automates this “digital word-of-mouth” at the scale of millions

Collaborative filtering: from word-of-mouth to algorithms

- Humans have always asked friends with *similar taste*
- CF automates this "digital word-of-mouth" at the scale of millions
- **Collective wisdom**: the crowd's behaviour reveals what goes together

Collaborative filtering: from word-of-mouth to algorithms

- Humans have always asked friends with *similar taste*
- CF automates this "digital word-of-mouth" at the scale of millions
- **Collective wisdom**: the crowd's behaviour reveals what goes together
- **Personalisation**: every user gets a different list

Collaborative filtering: from word-of-mouth to algorithms

- Humans have always asked friends with *similar taste*
- CF automates this "digital word-of-mouth" at the scale of millions
- **Collective wisdom**: the crowd's behaviour reveals what goes together
- **Personalisation**: every user gets a different list
- **Adaptability**: updates as new interactions arrive

Collaborative filtering: from word-of-mouth to algorithms

- Humans have always asked friends with *similar taste*
- CF automates this "digital word-of-mouth" at the scale of millions
- **Collective wisdom**: the crowd's behaviour reveals what goes together
- **Personalisation**: every user gets a different list
- **Adaptability**: updates as new interactions arrive

1992 Tapestry (Xerox PARC): email filtering

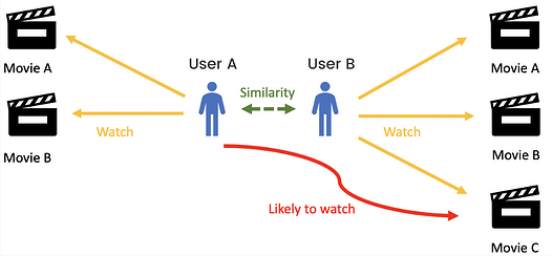
1994 GroupLens: Usenet news ratings

2003 Amazon: item-to-item CF

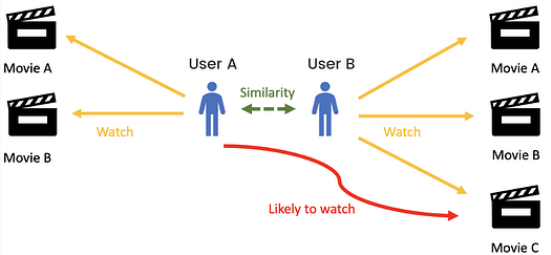
2006 Netflix Prize: $1M challenge

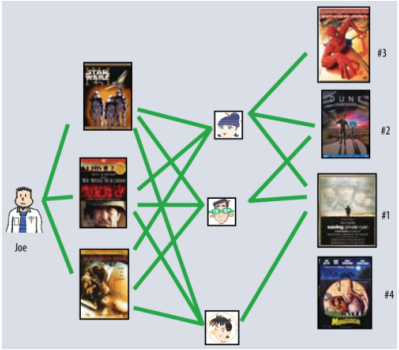
2010s Deep & sequential recommenders

Collaborative filtering in one picture



Collaborative filtering in one picture





Koren, Bell & Volinsky, IEEE Computer 2009

Content-based vs. collaborative: the key contrast

Content-based

- Needs item features (genre, cast, text)
- "You liked a sci-fi film $\rightarrow$ more sci-fi"

$+$ Works for new items

$-$ Over-specialises; no serendipity

Content-based vs. collaborative: the key contrast

Content-based

- Needs item features (genre, cast, text)
- "You liked a sci-fi film $\rightarrow$ more sci-fi"

\+ Works for new items

– Over-specialises; no serendipity

Collaborative filtering

- Needs only who-interacted-with-what
- "Users like you also liked..."

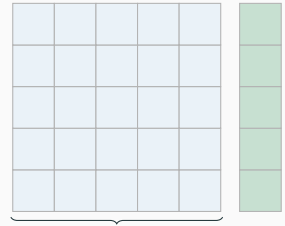
\+ Discovers non-obvious tastes

– Struggles with new users/items

Classification vs. collaborative filtering

Classification vs. collaborative filtering

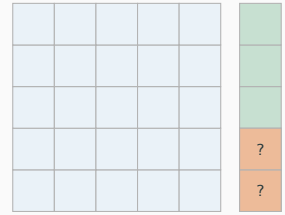
Classification



Adapted from Aggarwal, *Recommender Systems: The Textbook*, 2016, Ch. 1.

Classification vs. collaborative filtering

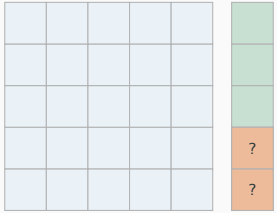
Classification



Adapted from Aggarwal, *Recommender Systems: The Textbook*, 2016, Ch. 1.

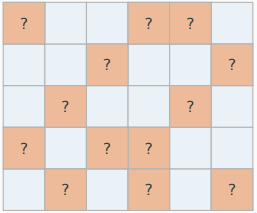
Classification vs. collaborative filtering

Classification



features (always known) label

Collaborative filtering

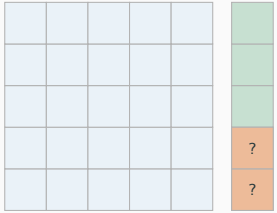


no fixed feature / label columns

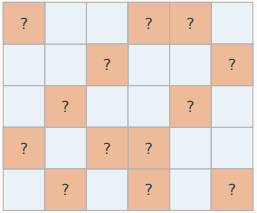
Adapted from Aggarwal, *Recommender Systems: The Textbook*, 2016, Ch. 1.

Classification vs. collaborative filtering

Classification



Collaborative filtering



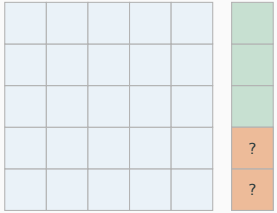
no fixed feature / label columns

- Classification: missing values only in the **label column**; train/test split by *rows*

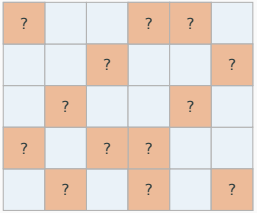
Adapted from Aggarwal, *Recommender Systems: The Textbook*, 2016, Ch. 1.

Classification vs. collaborative filtering

Classification



Collaborative filtering

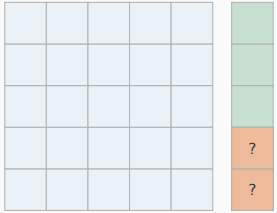


- Classification: missing values only in the **label column**; train/test split by *rows*
- CF: **any entry** may be missing; train/test split by *entries*

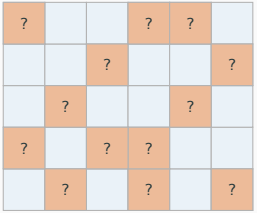
Adapted from Aggarwal, *Recommender Systems: The Textbook*, 2016, Ch. 1.

Classification vs. collaborative filtering

Classification



Collaborative filtering



- Classification: missing values only in the **label column**; train/test split by *rows*
- CF: **any entry** may be missing; train/test split by *entries*
- No clean distinction between features and labels ⇒ a different family of methods

Adapted from Aggarwal, *Recommender Systems: The Textbook*, 2016, Ch. 1.

The problem, formally

$$R = \begin{array}{c} \\ u_1 \\ u_2 \\ u_3 \end{array} \begin{array}{c} \begin{array}{cccc} i_1 & i_2 & i_3 & i_4 \end{array} \\ \begin{bmatrix} 5 & 4 & & 1 \\ 4 & & 5 & \\ & 5 & 4 & 3 \end{bmatrix} \end{array}$$

- Rows = users U, columns = items I
- r_{ui} = rating of user u for item i

The problem, formally

$$R = \begin{array}{c} \\ u_1 \\ u_2 \\ u_3 \end{array} \begin{array}{c} \begin{array}{cccc} i_1 & i_2 & i_3 & i_4 \end{array} \\ \begin{bmatrix} 5 & 4 & ? & 1 \\ 4 & ? & 5 & ? \\ ? & 5 & 4 & 3 \end{bmatrix} \end{array}$$

- Rows = users U, columns = items I
- r_{ui} = rating of user u for item i
- Most entries are unknown

The problem, formally

$$R = \begin{array}{c} \\ u_1 \\ u_2 \\ u_3 \end{array} \begin{array}{c} \begin{array}{cccc} i_1 & i_2 & i_3 & i_4 \end{array} \\ \begin{bmatrix} 5 & 4 & ? & 1 \\ 4 & ? & 5 & ? \\ ? & 5 & 4 & 3 \end{bmatrix} \end{array}$$

- Rows = users U, columns = items I
- r_{ui} = rating of user u for item i
- Most entries are unknown
- Ω = set of observed (u, i) pairs

The problem, formally

$$R = \begin{array}{c} \\ u_1 \\ u_2 \\ u_3 \end{array} \begin{array}{c} \begin{array}{cccc} i_1 & i_2 & i_3 & i_4 \end{array} \\ \begin{bmatrix} 5 & 4 & ? & 1 \\ 4 & ? & 5 & ? \\ ? & 5 & 4 & 3 \end{bmatrix} \end{array}$$

- Rows = users U, columns = items I
- r_{ui} = rating of user u for item i
- Most entries are unknown
- Ω = set of observed (u, i) pairs

Goal

Learn a function $\hat{r}_{ui} = f(u, i)$ that fills in the ?s, then recommend the top-K items with the highest $\hat{r}_{ui}$.

The problem, formally

$$R = \begin{array}{c} \\ u_1 \\ u_2 \\ u_3 \end{array} \begin{array}{c} \begin{array}{cccc} i_1 & i_2 & i_3 & i_4 \end{array} \\ \begin{bmatrix} 5 & 4 & ? & 1 \\ 4 & ? & 5 & ? \\ ? & 5 & 4 & 3 \end{bmatrix} \end{array}$$

- Rows = users U, columns = items I
- r_{ui} = rating of user u for item i
- Most entries are unknown
- Ω = set of observed (u, i) pairs

Goal
Learn a function $\hat{r}_{ui} = f(u, i)$ that fills in the ?s, then recommend the top-K items with the highest $\hat{r}_{ui}$.

In MovieLens, about **98%** of this matrix is empty. We will compute this ourselves.

Two formulations of the task

Rating prediction

- Learn $f : U \times I \to \mathbb{R}$
- Minimise error on held-out $(u, i) \in \Omega_{\text{test}}$

$$\text{RMSE} = \sqrt{\frac{1}{|\Omega_{\text{test}}|} \sum (r_{ui} - \hat{r}_{ui})^2}$$

- Every test cell counts equally
- Natural for explicit ratings (MovieLens, Netflix Prize)

Two formulations of the task

Rating prediction

- Learn $f : U \times I \to \mathbb{R}$
- Minimise error on held-out $(u, i) \in \Omega_{\text{test}}$

$$\text{RMSE} = \sqrt{\frac{1}{|\Omega_{\text{test}}|} \sum (r_{ui} - \hat{r}_{ui})^2}$$

- Every test cell counts equally
- Natural for explicit ratings (MovieLens, Netflix Prize)

Top-N recommendation

- For each u, output a ranked list $L_u \subset I \setminus I_u$, $|L_u| = N$
- Maximise relevant items near the top

$$\text{Precision@}N = \frac{|L_u \cap \text{Rel}_u|}{N}$$

- Only the head of the list matters
- Natural for implicit data (clicks, purchases)

Notation we will use all day

Symbol	Meaning
U, I	set of users, set of items
r_{ui}	rating of user u for item i ($\hat{r}_{ui}$: prediction)
I_u	items rated by user u
U_i	users who rated item i
$I_{uv} = I_u \cap I_v$	items rated by *both* u and v
$U_{ij} = U_i \cap U_j$	users who rated *both* i and j
μ_u (or $\bar{r}_u$)	mean rating of user u
Ω	set of observed (user, item) pairs

Explicit vs. implicit feedback

Explicit

- Star ratings, thumbs up/down, reviews
- Clear signal of like *and* dislike
- most users never rate
- *MovieLens is explicit*

Explicit vs. implicit feedback

Explicit

- Star ratings, thumbs up/down, reviews
- Clear signal of like *and* dislike
- most users never rate
- *MovieLens is explicit*

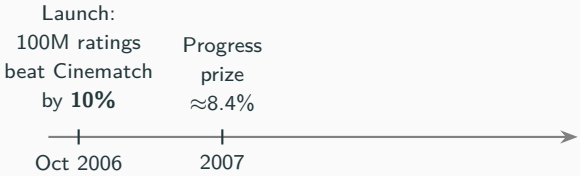
Implicit

- Clicks, watch time, purchases, skips
- Abundant, collected automatically
- Noisy; no direct negative signal
- *Most of industry is implicit*

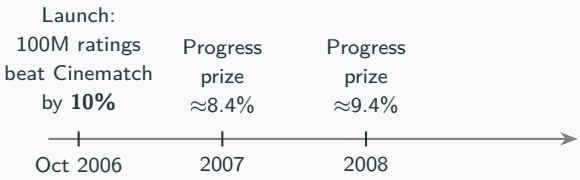
The Netflix Prize (2006–2009)



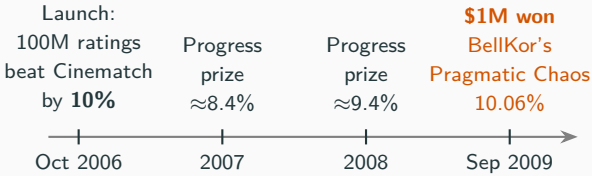
The Netflix Prize (2006–2009)



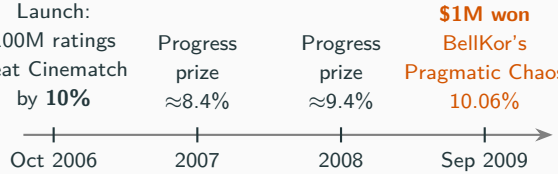
The Netflix Prize (2006–2009)



The Netflix Prize (2006–2009)

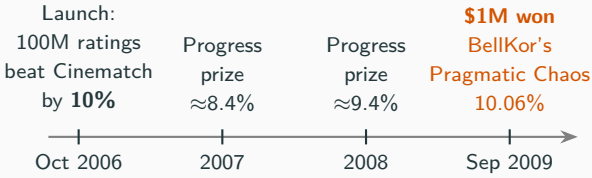


The Netflix Prize (2006–2009)



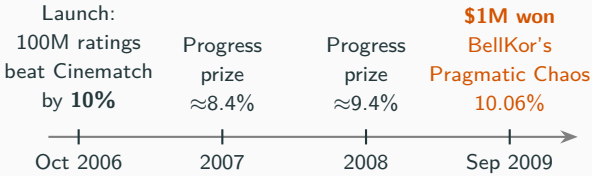
- Popularised **matrix factorization** (Funk SVD, SVD++)

The Netflix Prize (2006–2009)



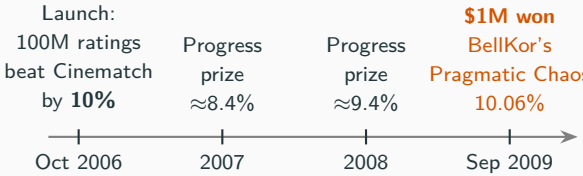
- Popularised **matrix factorization** (Funk SVD, SVD++)
- Showed the power of **biases** and **ensembles**

The Netflix Prize (2006–2009)



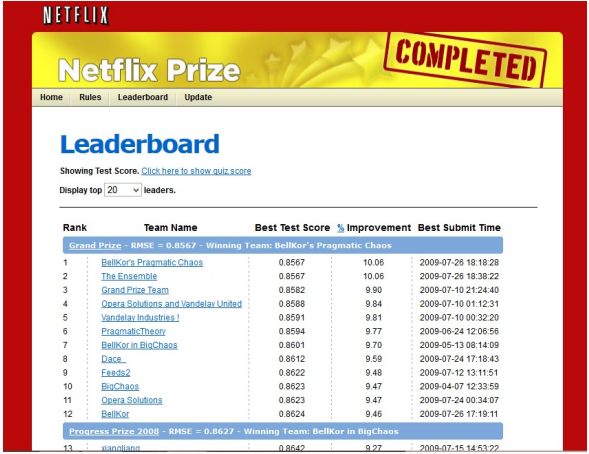
- Popularised **matrix factorization** (Funk SVD, SVD++)
- Showed the power of **biases** and **ensembles**
- Evaluated with **RMSE** — exactly what we use today

The Netflix Prize (2006–2009)



- Popularised **matrix factorization** (Funk SVD, SVD++)
- Showed the power of **biases** and **ensembles**
- Evaluated with **RMSE** — exactly what we use today
- Lesson: Netflix never deployed the full winning ensemble — engineering cost mattered more

The final Netflix Prize leaderboard



NETFLIX

Netflix Prize

COMPLETED

Home | Rules | Leaderboard | Update

Leaderboard

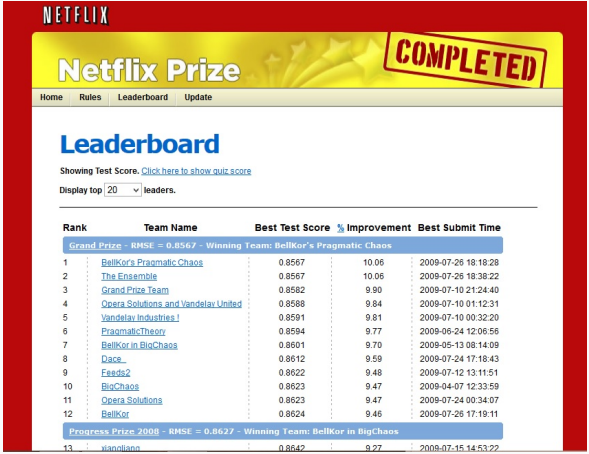
Showing Test Score. Click here to show quiz score

Display top 20 leaders.

Rank	Team Name	Best Test Score	% Improvement	Best Submit Time
Grand Prize - RMSE = 0.8567 - Winning Team: BellKor's Pragmatic Chaos				
1	BellKor's Pragmatic Chaos	0.8567	10.06	2009-07-26 18:18:28
2	The Ensemble	0.8567	10.06	2009-07-26 18:38:22
3	Grand Prize Team	0.8582	9.90	2009-07-10 21:24:40
4	Opera Solutions and Vandelay United	0.8588	9.84	2009-07-10 01:12:31
5	Vandelay Industries !	0.8591	9.81	2009-07-10 00:32:20
6	PragmaticTheory	0.8594	9.77	2009-06-24 12:06:56
7	BellKor in BigChaos	0.8601	9.70	2009-05-13 08:14:09
8	Dace	0.8612	9.59	2009-07-24 17:18:43
9	Feeds2	0.8622	9.48	2009-07-12 13:11:51
10	BigChaos	0.8623	9.47	2009-04-07 12:33:59
11	Opera Solutions	0.8623	9.47	2009-07-24 00:34:07
12	BellKor	0.8624	9.46	2009-07-26 17:19:11
Progress Prize 2008 - RMSE = 0.8627 - Winning Team: BellKor in BigChaos				
13	xiangliang	0.8642	9.27	2009-07-15 14:53:22

- Top two teams: test RMSE **0.8567**, a **10.06%** improvement over Netflix's own system

The final Netflix Prize leaderboard



NETFLIX

Netflix Prize

COMPLETED

Home | Rules | Leaderboard | Update

Leaderboard

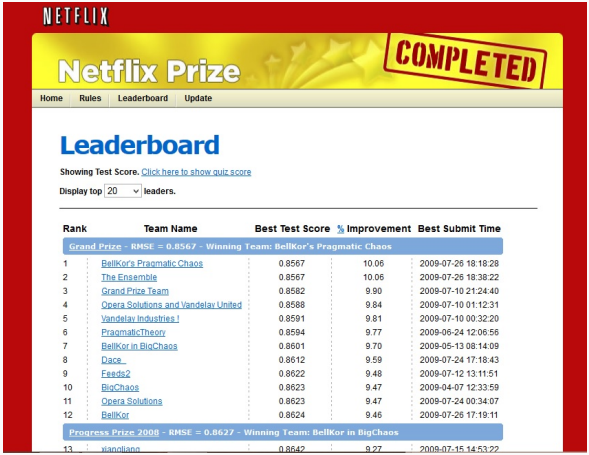
Showing Test Score. Click here to show quiz score

Display top 20 leaders.

Rank	Team Name	Best Test Score	% Improvement	Best Submit Time
Grand Prize - RMSE = 0.8567 - Winning Team: BellKor's Pragmatic Chaos				
1	BellKor's Pragmatic Chaos	0.8567	10.06	2009-07-26 18:18:28
2	The Ensemble	0.8567	10.06	2009-07-26 18:38:22
3	Grand Prize Team	0.8582	9.90	2009-07-10 21:24:40
4	Opera Solutions and Vandelay United	0.8588	9.84	2009-07-10 01:12:31
5	Vandelay Industries !	0.8591	9.81	2009-07-10 00:32:20
6	PragmaticTheory	0.8594	9.77	2009-06-24 12:06:56
7	BellKor in BigChaos	0.8601	9.70	2009-05-13 08:14:09
8	Dace	0.8612	9.59	2009-07-24 17:18:43
9	Feeds2	0.8622	9.48	2009-07-12 13:11:51
10	BigChaos	0.8623	9.47	2009-04-07 12:33:59
11	Opera Solutions	0.8623	9.47	2009-07-24 00:34:07
12	BellKor	0.8624	9.46	2009-07-26 17:19:11
Progress Prize 2008 - RMSE = 0.8627 - Winning Team: BellKor in BigChaos				
13	xiangliang	0.8642	9.27	2009-07-15 14:53:22

- Top two teams: test RMSE **0.8567**, a **10.06%** improvement over Netflix's own system
- BellKor's Pragmatic Chaos and The Ensemble tied — BellKor won by submitting about **20 minutes earlier**

The final Netflix Prize leaderboard


NETFLIX

Netflix Prize

COMPLETED

Home | Rules | Leaderboard | Update

Leaderboard

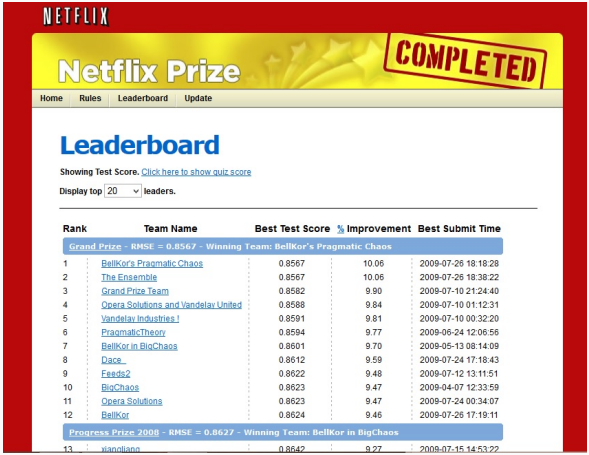
Showing Test Score. Click here to show quiz score

Display top 20 leaders.

Rank	Team Name	Best Test Score	% Improvement	Best Submit Time
Grand Prize - RMSE = 0.8567 - Winning Team: BellKor's Pragmatic Chaos				
1	BellKor's Pragmatic Chaos	0.8567	10.06	2009-07-26 18:18:28
2	The Ensemble	0.8567	10.06	2009-07-26 18:38:22
3	Grand Prize Team	0.8582	9.90	2009-07-10 21:24:40
4	Opera Solutions and Vandelay United	0.8588	9.84	2009-07-10 01:12:31
5	Vandelay Industries !	0.8591	9.81	2009-07-10 00:32:20
6	PragmaticTheory	0.8594	9.77	2009-06-24 12:06:56
7	BellKor in BigChaos	0.8601	9.70	2009-05-13 08:14:09
8	Dace	0.8612	9.59	2009-07-24 17:18:43
9	Feeds2	0.8622	9.48	2009-07-12 13:11:51
10	BigChaos	0.8623	9.47	2009-04-07 12:33:59
11	Opera Solutions	0.8623	9.47	2009-07-24 00:34:07
12	BellKor	0.8624	9.46	2009-07-26 17:19:11
Progress Prize 2008 - RMSE = 0.8627 - Winning Team: BellKor in BigChaos				
13	xiangliang	0.8642	9.27	2009-07-15 14:53:22

- Top two teams: test RMSE **0.8567**, a **10.06%** improvement over Netflix's own system
- BellKor's Pragmatic Chaos and The Ensemble tied — BellKor won by submitting about **20 minutes earlier**
- Both were large ensembles built around **matrix factorization**

The final Netflix Prize leaderboard



NETFLIX

Netflix Prize

COMPLETED

Home | Rules | Leaderboard | Update

Leaderboard

Showing Test Score. Click here to show quiz score

Display top 20 leaders.

Rank	Team Name	Best Test Score	% Improvement	Best Submit Time
Grand Prize - RMSE = 0.8567 - Winning Team: BellKor's Pragmatic Chaos				
1	BellKor's Pragmatic Chaos	0.8567	10.06	2009-07-26 18:18:28
2	The Ensemble	0.8567	10.06	2009-07-26 18:38:22
3	Grand Prize Team	0.8582	9.90	2009-07-10 21:24:40
4	Opera Solutions and Vandelay United	0.8588	9.84	2009-07-10 01:12:31
5	Vandelay Industries !	0.8591	9.81	2009-07-10 00:32:20
6	PragmaticTheory	0.8594	9.77	2009-06-24 12:06:56
7	BellKor in BigChaos	0.8601	9.70	2009-05-13 08:14:09
8	Dace	0.8612	9.59	2009-07-24 17:18:43
9	Feeds2	0.8622	9.48	2009-07-12 13:11:51
10	BigChaos	0.8623	9.47	2009-04-07 12:33:59
11	Opera Solutions	0.8623	9.47	2009-07-24 00:34:07
12	BellKor	0.8624	9.46	2009-07-26 17:19:11
Progress Prize 2008 - RMSE = 0.8627 - Winning Team: BellKor in BigChaos				
13	xiangliang	0.8642	9.27	2009-07-15 14:53:22

- Top two teams: test RMSE **0.8567**, a **10.06%** improvement over Netflix's own system
- BellKor's Pragmatic Chaos and The Ensemble tied — BellKor won by submitting about **20 minutes earlier**
- Both were large ensembles built around **matrix factorization**
- RMSE — the same measure we use today

The one question we answer all day

We have 100,000 ratings.

The one question we answer all day

We have 100,000 ratings.

Can we predict what a user will like next?

The one question we answer all day

We have 100,000 ratings.

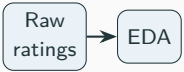
Can we predict what a user will like next?



The one question we answer all day

We have 100,000 ratings.

Can we predict what a user will like next?



The one question we answer all day

We have 100,000 ratings.

Can we predict what a user will like next?



The one question we answer all day

We have 100,000 ratings.

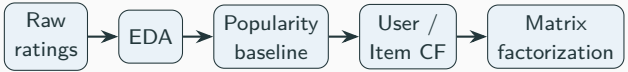
Can we predict what a user will like next?



The one question we answer all day

We have 100,000 ratings.

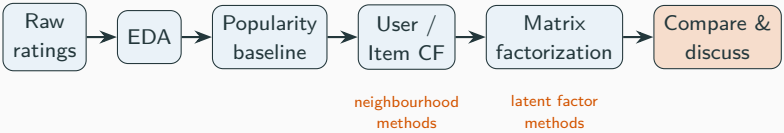
Can we predict what a user will like next?



The one question we answer all day

We have 100,000 ratings.

Can we predict what a user will like next?



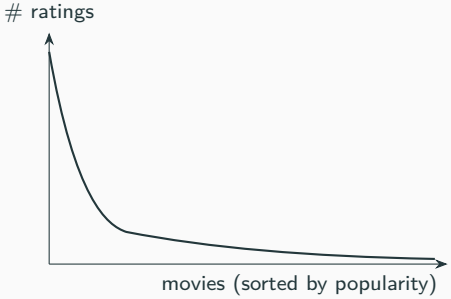
Each method fixes a weakness of the previous one.

Tools and data

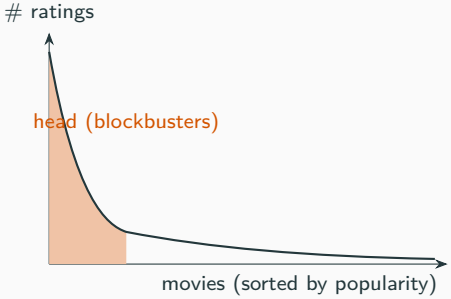
```
pip install pandas numpy matplotlib scikit-learn
```

- Dataset: **MovieLens** `ml-latest-small`: 610 users, 9.7K movies, ≈100K ratings
- Download: `https://grouplens.org/datasets/movielens/`
- Files we use: `ratings.csv`, `movies.csv`

The long tail

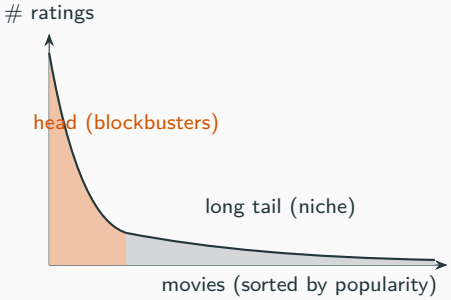


The long tail



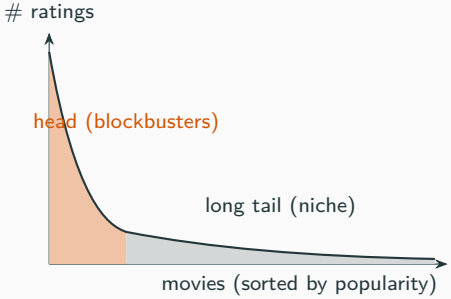
- A few blockbusters get most ratings

The long tail



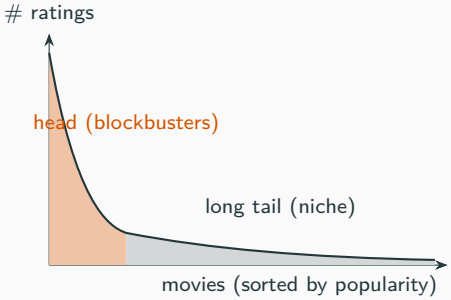
- A few blockbusters get most ratings
- Thousands of movies have < 5 ratings

The long tail



- A few blockbusters get most ratings
- Thousands of movies have < 5 ratings
- Online stores have "unlimited shelf space" $\Rightarrow$ niche items can still make money

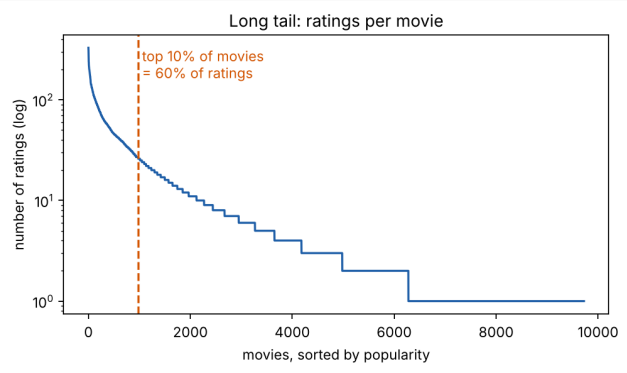
The long tail



- A few blockbusters get most ratings
- Thousands of movies have < 5 ratings
- Online stores have "unlimited shelf space" $\Rightarrow$ niche items can still make money

Consequence
Naive models recommend only the head.
The value of personalisation is in the tail.

Long tail in our data



The top 10% of movies receive **60%** of all ratings; 3,446 movies (35%) have exactly **one** rating.

Bias in recommender data

Bias = a systematic skew in what the data (or the algorithm) favours.

Bias in recommender data

Bias = a systematic skew in what the data (or the algorithm) favours.

- **Popularity bias** — popular items get recommended more ⇒ become more popular

Bias in recommender data

Bias = a systematic skew in what the data (or the algorithm) favours.

- **Popularity bias** — popular items get recommended more ⇒ become more popular
- **Selection bias** — users rate what they chose; active users dominate the data

Bias in recommender data

Bias = a systematic skew in what the data (or the algorithm) favours.

- **Popularity bias** — popular items get recommended more ⇒ become more popular
- **Selection bias** — users rate what they chose; active users dominate the data
- **Exposure bias** — some items are never shown, so never get a chance to be rated

Bias in recommender data

Bias = a systematic skew in what the data (or the algorithm) favours.

- **Popularity bias** — popular items get recommended more ⇒ become more popular
- **Selection bias** — users rate what they chose; active users dominate the data
- **Exposure bias** — some items are never shown, so never get a chance to be rated
- **Rating-scale bias** — generous vs. harsh raters (we fix this with normalisation)

The key problem

We know only $\sim 2\%$ of the ratings.

The key problem

We know only $\sim 2\%$ of the ratings.

The popularity list is the **same for everyone**.

The key problem

We know only $\sim 2\%$ of the ratings.

The popularity list is the **same for everyone**.

How do we estimate the missing 98% *per user*?

The key problem

We know only $\sim 2\%$ of the ratings.

The popularity list is the **same for everyone**.

How do we estimate the missing 98% *per user*?

$\rightarrow$ **Collaborative filtering**

2. Collaborative Filtering: How Similarity Works

Start with a puzzle

	Movie A	Movie B	Movie C	Movie D
User 1	5	4	1	?
User 2	5	4	2	2
User 3	1	2	5	5

Start with a puzzle

	Movie A	Movie B	Movie C	Movie D
User 1	5	4	1	?
User 2	5	4	2	2
User 3	1	2	5	5

Start with a puzzle

	Movie A	Movie B	Movie C	Movie D
User 1	5	4	1	?
User 2	5	4	2	2
User 3	1	2	5	5

- User 1 behaves like **User 2**, opposite to User 3

Start with a puzzle

	Movie A	Movie B	Movie C	Movie D
User 1	5	4	1	?
User 2	5	4	2	2
User 3	1	2	5	5

- User 1 behaves like **User 2**, opposite to User 3
- User 2 gave D a 2 $\Rightarrow$ predict low

Start with a puzzle

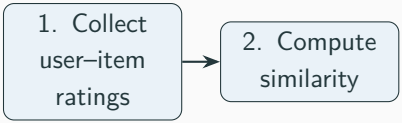
	Movie A	Movie B	Movie C	Movie D
User 1	5	4	1	?
User 2	5	4	2	2
User 3	1	2	5	5

- User 1 behaves like **User 2**, opposite to User 3
- User 2 gave D a 2 $\Rightarrow$ predict low
- That is the whole idea of **user-based CF**: *people who agreed in the past will agree in the future*

The CF workflow

1. Collect user–item ratings

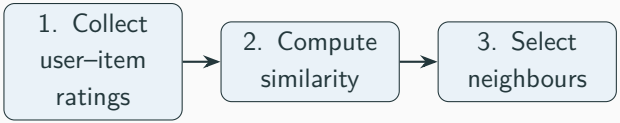
The CF workflow



Similarity choices

- Cosine
- Adjusted cosine
- Pearson (PCC)
- Jaccard

The CF workflow



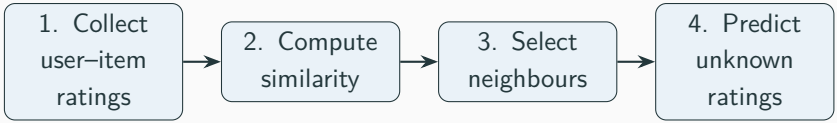
Similarity choices

- Cosine • Pearson (PCC)
- Adjusted cosine • Jaccard

Neighbour choices

- Top-k by similarity
- Similarity above a threshold
- Same cluster • random

The CF workflow



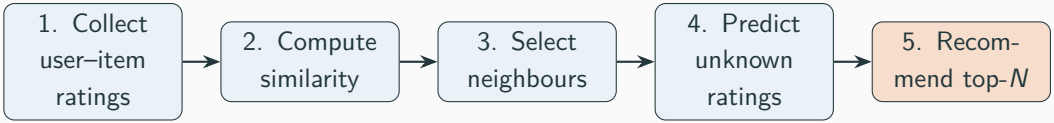
Similarity choices

- Cosine • Pearson (PCC)
- Adjusted cosine • Jaccard

Neighbour choices

- Top-k by similarity
- Similarity above a threshold
- Same cluster • random

The CF workflow



Similarity choices

- Cosine • Pearson (PCC)
- Adjusted cosine • Jaccard

Neighbour choices

- Top-k by similarity
- Similarity above a threshold
- Same cluster • random

Measuring similarity

Cosine similarity — angle between rating vectors

$$\text{sim}(u, v) = \frac{\sum_{i \in I_{uv}} r_{ui}\, r_{vi}}{\sqrt{\sum_{i \in I_{uv}} r_{ui}^2} \sqrt{\sum_{i \in I_{uv}} r_{vi}^2}}$$

Measuring similarity

Cosine similarity — angle between rating vectors

$$\text{sim}(u, v) = \frac{\sum_{i \in I_{uv}} r_{ui}\, r_{vi}}{\sqrt{\sum_{i \in I_{uv}} r_{ui}^2}\, \sqrt{\sum_{i \in I_{uv}} r_{vi}^2}}$$

Pearson correlation (PCC) — cosine after removing each user's mean

$$\text{sim}(u, v) = \frac{\sum_{i \in I_{uv}} (r_{ui} - \mu_u)(r_{vi} - \mu_v)}{\sqrt{\sum_{i \in I_{uv}} (r_{ui} - \mu_u)^2}\, \sqrt{\sum_{i \in I_{uv}} (r_{vi} - \mu_v)^2}}$$

Properties of similarity measures

Measure	Range	Shift-invariant	Scale-invariant	Uses values
Cosine (raw)	$[0, 1]$ for ratings > 0	no	yes	yes
Pearson (PCC)	$[-1, 1]$	yes	yes	yes
Adjusted cosine (items)	$[-1, 1]$	per user	yes	yes
Jaccard	$[0, 1]$	—	—	no

Properties of similarity measures

Measure	Range	Shift-invariant	Scale-invariant	Uses values
Cosine (raw)	$[0, 1]$ for ratings > 0	no	yes	yes
Pearson (PCC)	$[-1, 1]$	yes	yes	yes
Adjusted cosine (items)	$[-1, 1]$	per user	yes	yes
Jaccard	$[0, 1]$	—	—	no

- **Shift-invariant**: $r_{vi} \mapsto r_{vi} + c$ leaves PCC unchanged, since $r_{vi} - \mu_v$ is unchanged

Properties of similarity measures

Measure	Range	Shift-invariant	Scale-invariant	Uses values
Cosine (raw)	$[0, 1]$ for ratings > 0	no	yes	yes
Pearson (PCC)	$[-1, 1]$	yes	yes	yes
Adjusted cosine (items)	$[-1, 1]$	per user	yes	yes
Jaccard	$[0, 1]$	—	—	no

- **Shift-invariant**: $r_{vi} \mapsto r_{vi} + c$ leaves PCC unchanged, since $r_{vi} - \mu_v$ is unchanged
- **Scale-invariant**: $r_{vi} \mapsto a\, r_{vi}$ $(a > 0)$ cancels between numerator and denominator

Properties of similarity measures

Measure	Range	Shift-invariant	Scale-invariant	Uses values
Cosine (raw)	$[0, 1]$ for ratings > 0	no	yes	yes
Pearson (PCC)	$[-1, 1]$	yes	yes	yes
Adjusted cosine (items)	$[-1, 1]$	per user	yes	yes
Jaccard	$[0, 1]$	—	—	no

- **Shift-invariant**: $r_{vi} \mapsto r_{vi} + c$ leaves PCC unchanged, since $r_{vi} - \mu_v$ is unchanged
- **Scale-invariant**: $r_{vi} \mapsto a\, r_{vi}$ $(a > 0)$ cancels between numerator and denominator
- Example: $u = (5, 5, 4, 5)$, $v = (2, 2, 1, 2)$: PCC $= 1.0$ — same taste, different generosity

Properties of similarity measures

Measure	Range	Shift-invariant	Scale-invariant	Uses values
Cosine (raw)	$[0, 1]$ for ratings > 0	no	yes	yes
Pearson (PCC)	$[-1, 1]$	yes	yes	yes
Adjusted cosine (items)	$[-1, 1]$	per user	yes	yes
Jaccard	$[0, 1]$	—	—	no

- **Shift-invariant**: $r_{vi} \mapsto r_{vi} + c$ leaves PCC unchanged, since $r_{vi} - \mu_v$ is unchanged
- **Scale-invariant**: $r_{vi} \mapsto a\, r_{vi}$ $(a > 0)$ cancels between numerator and denominator
- Example: $u = (5, 5, 4, 5)$, $v = (2, 2, 1, 2)$: PCC $= 1.0$ — same taste, different generosity
- Raw cosine is not shift-invariant: adding a constant to all ratings pushes every pair towards 1

Worked example (1): cosine for two users

User	Item 1	Item 2	Item 3	Item 4	Item 5	Item 6
U1	5	3	4	4	–	1
U2	3	1	2	3	3	2

Worked example (1): cosine for two users

User	Item 1	Item 2	Item 3	Item 4	Item 5	Item 6
U1	5	3	4	4	–	1
U2	3	1	2	3	3	2

- Co-rated items $I_{12} = \{1, 2, 3, 4, 6\}$: $u_1 = (5, 3, 4, 4, 1)$, $u_2 = (3, 1, 2, 3, 2)$

Worked example (1): cosine for two users

User	Item 1	Item 2	Item 3	Item 4	Item 5	Item 6
U1	5	3	4	4	–	1
U2	3	1	2	3	3	2

- Co-rated items $I_{12} = \{1, 2, 3, 4, 6\}$: $u_1 = (5, 3, 4, 4, 1)$, $u_2 = (3, 1, 2, 3, 2)$

$$\cos(u_1, u_2) = \frac{15 + 3 + 8 + 12 + 2}{\sqrt{67}\,\sqrt{27}} = \frac{40}{42.53} \approx 0.94$$

Worked example (1): cosine for two users

User	Item 1	Item 2	Item 3	Item 4	Item 5	Item 6
U1	5	3	4	4	–	1
U2	3	1	2	3	3	2

- Co-rated items $I_{12} = \{1, 2, 3, 4, 6\}$: $u_1 = (5, 3, 4, 4, 1)$, $u_2 = (3, 1, 2, 3, 2)$

$$\cos(u_1, u_2) = \frac{15 + 3 + 8 + 12 + 2}{\sqrt{67}\sqrt{27}} = \frac{40}{42.53} \approx 0.94$$

$$\text{PCC}(u_1, u_2): \quad \mu_1 = 3.4,\ \mu_2 = 2.2 \ \Rightarrow\ \frac{2.6}{\sqrt{9.2}\sqrt{2.8}} \approx 0.51$$

Worked example (2): finding neighbours of User 3

User	1	2	3	4	5	6			
1	7	6	7	4	5	4			
2	6	7	?	4	3	4			
3	?	3	3	1	1	?			
4	1	2	2	3	3	4			
5	1	?	1	2	3	3			

Example from Aggarwal, *Recommender Systems: The Textbook*, 2016, Table 2.1.

Worked example (2): finding neighbours of User 3

User	1	2	3	4	5	6			
1	7	6	7	4	5	4			
2	6	7	?	4	3	4			
3	?	3	3	1	1	?			
4	1	2	2	3	3	4			
5	1	?	1	2	3	3			

- Target: **User 3**; we want $\hat{r}_{31}$ and $\hat{r}_{36}$

Example from Aggarwal, *Recommender Systems: The Textbook*, 2016, Table 2.1.

Worked example (2): finding neighbours of User 3

User	1	2	3	4	5	6	Mean μ_u		
1	7	6	7	4	5	4	5.5		
2	6	7	?	4	3	4	4.8		
3	?	3	3	1	1	?	2.0		
4	1	2	2	3	3	4	2.5		
5	1	?	1	2	3	3	2.0		

- Target: **User 3**; we want $\hat{r}_{31}$ and $\hat{r}_{36}$

Example from Aggarwal, *Recommender Systems: The Textbook*, 2016, Table 2.1.

Worked example (2): finding neighbours of User 3

User	1	2	3	4	5	6	Mean μ_u	Cosine$(u, 3)$
1	7	6	7	4	5	4	5.5	0.956
2	6	7	?	4	3	4	4.8	0.981
3	?	3	3	1	1	?	2.0	1.0
4	1	2	2	3	3	4	2.5	0.789
5	1	?	1	2	3	3	2.0	0.645

- Target: **User 3**; we want $\hat{r}_{31}$ and $\hat{r}_{36}$

Example from Aggarwal, *Recommender Systems: The Textbook*, 2016, Table 2.1.

Worked example (2): finding neighbours of User 3

User	1	2	3	4	5	6	Mean μ_u	Cosine$(u, 3)$	Pearson$(u, 3)$
1	7	6	7	4	5	4	5.5	0.956	0.894
2	6	7	?	4	3	4	4.8	0.981	0.939
3	?	3	3	1	1	?	2.0	1.0	1.0
4	1	2	2	3	3	4	2.5	0.789	-1.0
5	1	?	1	2	3	3	2.0	0.645	-0.817

- Target: **User 3**; we want $\hat{r}_{31}$ and $\hat{r}_{36}$

Example from Aggarwal, *Recommender Systems: The Textbook*, 2016, Table 2.1.

Worked example (2): finding neighbours of User 3

User	1	2	3	4	5	6	Mean μ_u	Cosine$(u, 3)$	Pearson$(u, 3)$
1	7	6	7	4	5	4	5.5	0.956	**0.894**
2	6	7	?	4	3	4	4.8	0.981	**0.939**
3	?	3	3	1	1	?	2.0	1.0	1.0
4	1	2	2	3	3	4	2.5	0.789	-1.0
5	1	?	1	2	3	3	2.0	0.645	-0.817

- Target: **User 3**; we want $\hat{r}_{31}$ and $\hat{r}_{36}$
- Top-2 neighbours by Pearson: **Users 1 and 2**

Example from Aggarwal, *Recommender Systems: The Textbook*, 2016, Table 2.1.

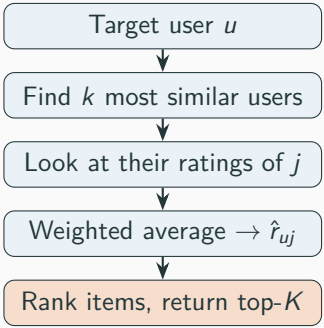
Worked example (2): finding neighbours of User 3

User	1	2	3	4	5	6	Mean μ_u	Cosine$(u, 3)$	Pearson$(u, 3)$
1	7	6	7	4	5	4	5.5	0.956	**0.894**
2	6	7	?	4	3	4	4.8	0.981	**0.939**
3	?	3	3	1	1	?	2.0	1.0	1.0
4	1	2	2	3	3	4	2.5	0.789	-1.0
5	1	?	1	2	3	3	2.0	0.645	-0.817

- Target: **User 3**; we want $\hat{r}_{31}$ and $\hat{r}_{36}$
- Top-2 neighbours by Pearson: **Users 1 and 2**
- User 4: Cosine compares raw ratings, while Pearson compares deviations from users' averages, revealing perfectly opposite rating patterns.

Example from Aggarwal, *Recommender Systems: The Textbook*, 2016, Table 2.1.

User-based CF: the prediction step

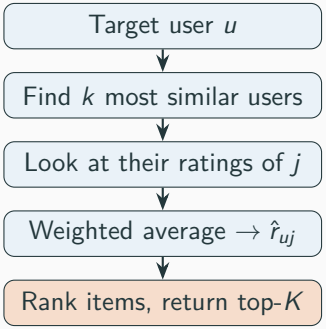


Standard weighted average

$$\hat{r}_{uj} = \frac{\sum_{v \in P_u(j)} \text{sim}(u, v)\ r_{vj}}{\sum_{v \in P_u(j)} |\text{sim}(u, v)|}$$

$P_u(j)$: the k closest users to u who rated item j ("peers").

User-based CF: the prediction step



Standard weighted average

$$\hat{r}_{uj} = \frac{\sum_{v \in P_u(j)} \text{sim}(u, v)\ r_{vj}}{\sum_{v \in P_u(j)} |\text{sim}(u, v)|}$$

Mean-centred prediction

$$\hat{r}_{uj} = \mu_u + \frac{\sum_{v \in P_u(j)} \text{sim}(u, v)\,(r_{vj} - \mu_v)}{\sum_{v \in P_u(j)} |\text{sim}(u, v)|}$$

$P_u(j)$: the k closest users to u who rated item j ("peers").

Worked example (3): predicting for User 3

Neighbours: $\text{sim}(1,3) = 0.894$, $\text{sim}(2,3) = 0.939$; $\mu_1 = 5.5,\ \mu_2 = 4.8,\ \mu_3 = 2.0$

Raw weighted average

Worked example (3): predicting for User 3

Neighbours: $\text{sim}(1,3) = 0.894$, $\text{sim}(2,3) = 0.939$; $\mu_1 = 5.5,\ \mu_2 = 4.8,\ \mu_3 = 2.0$

Raw weighted average

$$\hat{r}_{31} = \frac{7(0.894) + 6(0.939)}{0.894 + 0.939} \approx 6.49$$

Worked example (3): predicting for User 3

Neighbours: $\text{sim}(1,3) = 0.894$, $\text{sim}(2,3) = 0.939$; $\mu_1 = 5.5,\ \mu_2 = 4.8,\ \mu_3 = 2.0$

Raw weighted average

$$\hat{r}_{31} = \frac{7(0.894) + 6(0.939)}{0.894 + 0.939} \approx 6.49$$

$$\hat{r}_{36} = \frac{4(0.894) + 4(0.939)}{0.894 + 0.939} = 4.0$$

Worked example (3): predicting for User 3

Neighbours: $\text{sim}(1,3) = 0.894$, $\text{sim}(2,3) = 0.939$; $\mu_1 = 5.5,\ \mu_2 = 4.8,\ \mu_3 = 2.0$

Raw weighted average

$$\hat{r}_{31} = \frac{7(0.894) + 6(0.939)}{0.894 + 0.939} \approx 6.49$$

$$\hat{r}_{36} = \frac{4(0.894) + 4(0.939)}{0.894 + 0.939} = 4.0$$

Problem

User 3 never rates above 3! Neighbours are *generous* raters.

Worked example (3): predicting for User 3

Neighbours: $\text{sim}(1,3) = 0.894$, $\text{sim}(2,3) = 0.939$; $\mu_1 = 5.5,\ \mu_2 = 4.8,\ \mu_3 = 2.0$

Raw weighted average

$$\hat{r}_{31} = \frac{7(0.894) + 6(0.939)}{0.894 + 0.939} \approx 6.49$$

$$\hat{r}_{36} = \frac{4(0.894) + 4(0.939)}{0.894 + 0.939} = 4.0$$

Problem
User 3 never rates above 3! Neighbours are *generous* raters.

Mean-centred

$$\hat{r}_{31} = 2 + \frac{1.5(0.894) + 1.2(0.939)}{1.833} \approx \mathbf{3.35}$$

Worked example (3): predicting for User 3

Neighbours: $\text{sim}(1,3) = 0.894$, $\text{sim}(2,3) = 0.939$; $\mu_1 = 5.5,\ \mu_2 = 4.8,\ \mu_3 = 2.0$

Raw weighted average

$$\hat{r}_{31} = \frac{7(0.894) + 6(0.939)}{0.894 + 0.939} \approx 6.49$$

$$\hat{r}_{36} = \frac{4(0.894) + 4(0.939)}{0.894 + 0.939} = 4.0$$

Mean-centred

$$\hat{r}_{31} = 2 + \frac{1.5(0.894) + 1.2(0.939)}{1.833} \approx \mathbf{3.35}$$

$$\hat{r}_{36} = 2 + \frac{-1.5(0.894) - 0.8(0.939)}{1.833} \approx \mathbf{0.86}$$

Problem
User 3 never rates above 3! Neighbours are *generous* raters.

Worked example (3): predicting for User 3

Neighbours: $\text{sim}(1,3) = 0.894$, $\text{sim}(2,3) = 0.939$; $\mu_1 = 5.5,\ \mu_2 = 4.8,\ \mu_3 = 2.0$

Raw weighted average

$$\hat{r}_{31} = \frac{7(0.894) + 6(0.939)}{0.894 + 0.939} \approx 6.49$$

$$\hat{r}_{36} = \frac{4(0.894) + 4(0.939)}{0.894 + 0.939} = 4.0$$

Problem
User 3 never rates above 3! Neighbours are *generous* raters.

Mean-centred

$$\hat{r}_{31} = 2 + \frac{1.5(0.894) + 1.2(0.939)}{1.833} \approx \mathbf{3.35}$$

$$\hat{r}_{36} = 2 + \frac{-1.5(0.894) - 0.8(0.939)}{1.833} \approx \mathbf{0.86}$$

Interpretation
Item 1 is User 3's *favourite* among the unseen; item 6 is disliked. Recommend item 1.

Z-score normalisation

When users differ in *spread* as well as *mean*:

$$z_{uj} = \frac{r_{uj} - \mu_u}{\sigma_u}, \qquad \sigma_u = \sqrt{\frac{\sum_{j \in I_u} (r_{uj} - \mu_u)^2}{|I_u| - 1}}$$

Z-score normalisation

When users differ in *spread* as well as *mean*:

$$z_{uj} = \frac{r_{uj} - \mu_u}{\sigma_u}, \qquad \sigma_u = \sqrt{\frac{\sum_{j \in I_u} (r_{uj} - \mu_u)^2}{|I_u| - 1}}$$

Prediction: normalise, average, then **undo** the normalisation

$$\hat{r}_{uj} = \mu_u + \sigma_u \cdot \frac{\sum_{v \in P_u(j)} \mathrm{sim}(u, v)\ z_{vj}}{\sum_{v \in P_u(j)} |\mathrm{sim}(u, v)|}$$

Z-score normalisation

When users differ in *spread* as well as *mean*:

$$z_{uj} = \frac{r_{uj} - \mu_u}{\sigma_u}, \qquad \sigma_u = \sqrt{\frac{\sum_{j \in I_u} (r_{uj} - \mu_u)^2}{|I_u| - 1}}$$

Prediction: normalise, average, then **undo** the normalisation

$$\hat{r}_{uj} = \mu_u + \sigma_u \cdot \frac{\sum_{v \in P_u(j)} \mathrm{sim}(u, v)\ z_{vj}}{\sum_{v \in P_u(j)} |\mathrm{sim}(u, v)|}$$

Example: $\sigma_1 = 1.38,\ \sigma_2 = 1.64,\ \sigma_3 = 1.15; \quad z_{11} = 1.09,\ z_{21} = 0.73$

$\hat{r}_{31} = 2 + 1.15 \cdot \frac{0.894(1.09) + 0.939(0.73)}{1.833} \approx \mathbf{3.04}, \quad \hat{r}_{36} \approx \mathbf{1.10}$

Z-score normalisation

When users differ in *spread* as well as *mean*:

$$z_{uj} = \frac{r_{uj} - \mu_u}{\sigma_u}, \qquad \sigma_u = \sqrt{\frac{\sum_{j \in I_u} (r_{uj} - \mu_u)^2}{|I_u| - 1}}$$

Prediction: normalise, average, then **undo** the normalisation

$$\hat{r}_{uj} = \mu_u + \sigma_u \cdot \frac{\sum_{v \in P_u(j)} \mathrm{sim}(u, v)\; z_{vj}}{\sum_{v \in P_u(j)} |\mathrm{sim}(u, v)|}$$

Example: $\sigma_1 = 1.38,\ \sigma_2 = 1.64,\ \sigma_3 = 1.15;\quad z_{11} = 1.09,\ z_{21} = 0.73$

$\hat{r}_{31} = 2 + 1.15 \cdot \frac{0.894(1.09) + 0.939(0.73)}{1.833} \approx \mathbf{3.04}, \quad \hat{r}_{36} \approx \mathbf{1.10}$

Rule

If you apply $g(\cdot)$ to normalise ratings, apply $g^{-1}(\cdot)$ to the prediction. Z-scores can push predictions *outside* the rating scale — clip them.

Making similarity more reliable

1. **Significance weighting** — distrust similarities based on few co-rated items

$$\mathrm{sim}'(u, v) = \mathrm{sim}(u, v) \cdot \frac{\min\{|I_u \cap I_v|,\ \beta\}}{\beta}$$

Making similarity more reliable

1. **Significance weighting** — distrust similarities based on few co-rated items

$$\mathrm{sim}'(u, v) = \mathrm{sim}(u, v) \cdot \frac{\min\{|I_u \cap I_v|,\ \beta\}}{\beta}$$

$\beta = 5$, only 3 common ratings, raw sim $0.8 \Rightarrow 0.8 \times 3/5 = \mathbf{0.48}$

Making similarity more reliable

1. **Significance weighting** — distrust similarities based on few co-rated items

$$\mathrm{sim}'(u, v) = \mathrm{sim}(u, v) \cdot \frac{\min\{|I_u \cap I_v|,\ \beta\}}{\beta}$$

$\beta = 5$, only 3 common ratings, raw sim $0.8 \Rightarrow 0.8 \times 3/5 = \mathbf{0.48}$

2. **Inverse user frequency (IUF)** — popular items say little about taste

$$w_j = \log \frac{m}{m_j} \qquad (m \text{ users in total, } m_j \text{ rated item } j)$$

Making similarity more reliable

1. **Significance weighting** — distrust similarities based on few co-rated items

$$\mathrm{sim}'(u,v) = \mathrm{sim}(u,v) \cdot \frac{\min\{|I_u \cap I_v|,\ \beta\}}{\beta}$$

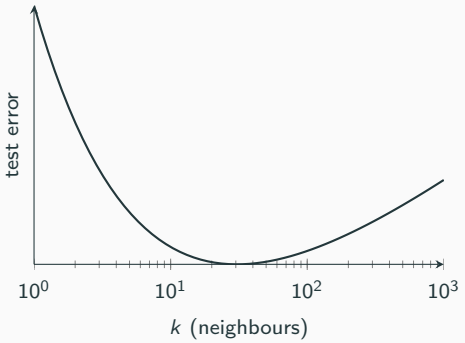
$\beta = 5$, only 3 common ratings, raw sim $0.8 \Rightarrow 0.8 \times 3/5 = \mathbf{0.48}$

2. **Inverse user frequency (IUF)** — popular items say little about taste

$$w_j = \log \frac{m}{m_j} \qquad (m \text{ users in total, } m_j \text{ rated item } j)$$

Weighted Pearson: put w_k inside every sum of the Pearson formula.

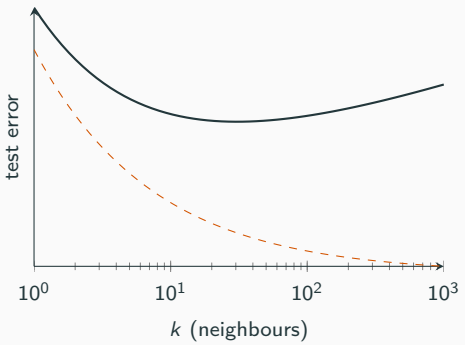
Choosing the neighbourhood size k



Schematic curves, not measured data.

- Total error = variance + bias^2 + noise

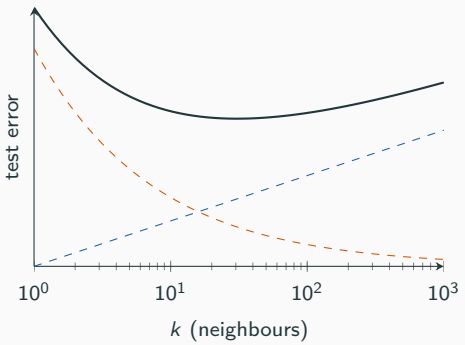
Choosing the neighbourhood size k



Schematic curves, not measured data.

- Total error = variance + bias^2 + noise
- Small k: prediction rests on a few users
  $\Rightarrow$ high **variance**

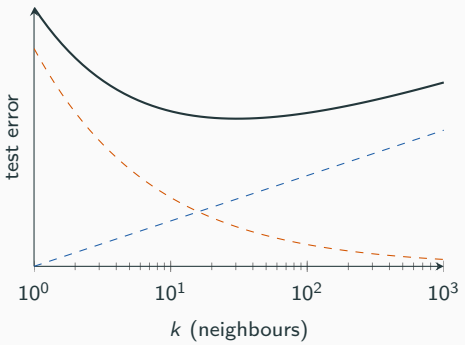
Choosing the neighbourhood size k



Schematic curves, not measured data.

- Total error = variance + bias^2 + noise
- Small k: prediction rests on a few users $\Rightarrow$ high **variance**
- Large k: dissimilar users dilute the signal, prediction drifts to $\mu_u \Rightarrow$ high **bias**

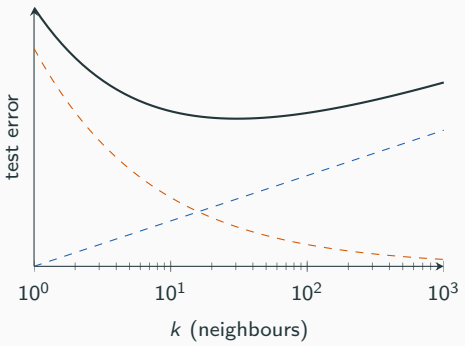
Choosing the neighbourhood size k



Schematic curves, not measured data.

- Total error = variance + bias^2 + noise
- Small k: prediction rests on a few users $\Rightarrow$ high **variance**
- Large k: dissimilar users dilute the signal, prediction drifts to $\mu_u \Rightarrow$ high **bias**
- Choose k by validation; on MovieLens typically 20–50

Choosing the neighbourhood size k



Schematic curves, not measured data.

- Total error = variance + bias^2 + noise
- Small k: prediction rests on a few users $\Rightarrow$ high **variance**
- Large k: dissimilar users dilute the signal, prediction drifts to $\mu_u \Rightarrow$ high **bias**
- Choose k by validation; on MovieLens typically 20–50
- Same trade-off as k in k-NN classification

Jaccard similarity for implicit data

When we only know *whether* a user interacted (click, purchase, view):

$$\text{Jacc}(u, v) = \frac{|I_u \cap I_v|}{|I_u \cup I_v|}$$

Jaccard similarity for implicit data

When we only know *whether* a user interacted (click, purchase, view):

$$\text{Jacc}(u, v) = \frac{|I_u \cap I_v|}{|I_u \cup I_v|}$$

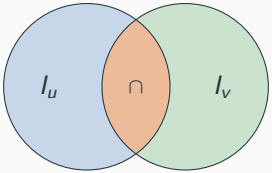
$I_{U1} = \{1, 2, 3, 4, 6\}, \quad I_{U2} = \{1, 2, 3, 4, 5, 6\} \Rightarrow \text{Jacc} = 5/6 \approx 0.83$

Jaccard similarity for implicit data

When we only know *whether* a user interacted (click, purchase, view):

$$\text{Jacc}(u, v) = \frac{|I_u \cap I_v|}{|I_u \cup I_v|}$$

$I_{U1} = \{1, 2, 3, 4, 6\}, \quad I_{U2} = \{1, 2, 3, 4, 5, 6\} \Rightarrow \text{Jacc} = 5/6 \approx 0.83$

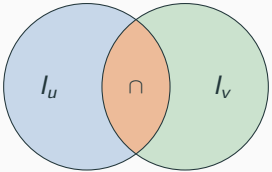


Jaccard similarity for implicit data

When we only know *whether* a user interacted (click, purchase, view):

$$\text{Jacc}(u, v) = \frac{|I_u \cap I_v|}{|I_u \cup I_v|}$$

$I_{U1} = \{1, 2, 3, 4, 6\}, \quad I_{U2} = \{1, 2, 3, 4, 5, 6\} \Rightarrow \text{Jacc} = 5/6 \approx 0.83$



Ignores rating values entirely — right for clicks, wrong for star ratings.

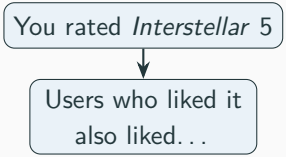
Item-based CF

Flip the question: instead of similar *users*, find similar *items*.

You rated *Interstellar* 5

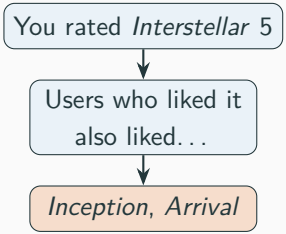
Item-based CF

Flip the question: instead of similar *users*, find similar *items*.



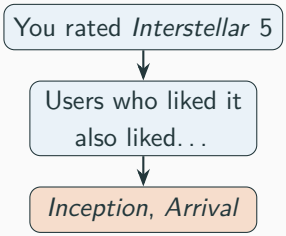
Item-based CF

Flip the question: instead of similar *users*, find similar *items*.



Item-based CF

Flip the question: instead of similar *users*, find similar *items*.

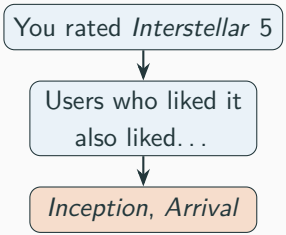


Adjusted cosine (centre by the *user* mean):

$$\mathrm{AC}(i,j) = \frac{\sum_{u \in U_{ij}} s_{ui}\, s_{uj}}{\sqrt{\sum s_{ui}^2}\sqrt{\sum s_{uj}^2}}, \quad s_{ui} = r_{ui} - \mu_u$$

Item-based CF

Flip the question: instead of similar *users*, find similar *items*.



Adjusted cosine (centre by the *user* mean):

$$\mathrm{AC}(i,j) = \frac{\sum_{u \in U_{ij}} s_{ui}\, s_{uj}}{\sqrt{\sum s_{ui}^2}\sqrt{\sum s_{uj}^2}}, \qquad s_{ui} = r_{ui} - \mu_u$$

Prediction from the user's own ratings:

$$\hat{r}_{ut} = \frac{\sum_{j \in Q_t(u)} \mathrm{AC}(j,t)\; r_{uj}}{\sum_{j \in Q_t(u)} |\mathrm{AC}(j,t)|}$$

Worked example (4): item-based for User 3

User (mean-centred)	1	2	3	4	5	6
1	1.5	0.5	1.5	−1.5	−0.5	−1.5
2	1.2	2.2	?	−0.8	−1.8	−0.8
3	?	1	1	−1	−1	?
4	−1.5	−0.5	−0.5	0.5	0.5	1.5
5	−1	?	−1	0	1	1

Worked example (4): item-based for User 3

User (mean-centred)	1	2	3	4	5	6
1	1.5	0.5	1.5	−1.5	−0.5	−1.5
2	1.2	2.2	?	−0.8	−1.8	−0.8
3	?	1	1	−1	−1	?
4	−1.5	−0.5	−0.5	0.5	0.5	1.5
5	−1	?	−1	0	1	1

Worked example (4): item-based for User 3

User (mean-centred)	1	2	3	4	5	6
1	1.5	0.5	1.5	−1.5	−0.5	−1.5
2	1.2	2.2	?	−0.8	−1.8	−0.8
3	?	1	1	−1	−1	?
4	−1.5	−0.5	−0.5	0.5	0.5	1.5
5	−1	?	−1	0	1	1
AC(1, j)	1	0.735	0.912	−0.848	−0.813	−0.990

Worked example (4): item-based for User 3

User (mean-centred)	1	2	3	4	5	6
1	1.5	0.5	1.5	−1.5	−0.5	−1.5
2	1.2	2.2	?	−0.8	−1.8	−0.8
3	?	1	1	−1	−1	?
4	−1.5	−0.5	−0.5	0.5	0.5	1.5
5	−1	?	−1	0	1	1
AC(1, j)	1	**0.735**	**0.912**	−0.848	−0.813	−0.990

$$\hat{r}_{31} = \frac{3(0.735) + 3(0.912)}{0.735 + 0.912} = \mathbf{3}$$

Worked example (4): item-based for User 3

User (mean-centred)	1	2	3	4	5	6
1	1.5	0.5	1.5	−1.5	−0.5	−1.5
2	1.2	2.2	?	−0.8	−1.8	−0.8
3	?	1	1	−1	−1	?
4	−1.5	−0.5	−0.5	0.5	0.5	1.5
5	−1	?	−1	0	1	1
AC(1, j)	1	**0.735**	**0.912**	−0.848	−0.813	−0.990
AC(6, j)	−0.990	−0.622	−0.912	0.829	0.730	1

$$\hat{r}_{31} = \frac{3(0.735) + 3(0.912)}{0.735 + 0.912} = \mathbf{3}$$

Worked example (4): item-based for User 3

User (mean-centred)	1	2	3	4	5	6
1	1.5	0.5	1.5	−1.5	−0.5	−1.5
2	1.2	2.2	?	−0.8	−1.8	−0.8
3	?	1	1	−1	−1	?
4	−1.5	−0.5	−0.5	0.5	0.5	1.5
5	−1	?	−1	0	1	1
AC(1, j)	1	**0.735**	**0.912**	−0.848	−0.813	−0.990
AC(6, j)	−0.990	−0.622	−0.912	**0.829**	**0.730**	1

$$\hat{r}_{31} = \frac{3(0.735) + 3(0.912)}{0.735 + 0.912} = \mathbf{3} \qquad \hat{r}_{36} = \frac{1(0.829) + 1(0.730)}{0.829 + 0.730} = \mathbf{1}$$

Worked example (4): item-based for User 3

User (mean-centred)	1	2	3	4	5	6
1	1.5	0.5	1.5	−1.5	−0.5	−1.5
2	1.2	2.2	?	−0.8	−1.8	−0.8
3	?	1	1	−1	−1	?
4	−1.5	−0.5	−0.5	0.5	0.5	1.5
5	−1	?	−1	0	1	1
AC(1, j)	1	**0.735**	**0.912**	−0.848	−0.813	−0.990
AC(6, j)	−0.990	−0.622	−0.912	**0.829**	**0.730**	1

$$\hat{r}_{31} = \frac{3(0.735) + 3(0.912)}{0.735 + 0.912} = \mathbf{3} \qquad \hat{r}_{36} = \frac{1(0.829) + 1(0.730)}{0.829 + 0.730} = \mathbf{1}$$

Same conclusion as user-based: **recommend item 1**, not item 6 — and the predictions stay within User 3's own rating range.

User-based vs. item-based

	User-based	**Item-based**
Similarity matrix size	users $\times$ users	items $\times$ items
Stability	Tastes drift; recompute often	Item relations stable; precompute
Explainability	"People like you..."	"Because you watched X"
Serendipity	Higher	Lower
Normalisation	Pearson / mean-centring	Adjusted cosine
Famous use	GroupLens (1994)	Amazon (2003)

Computational cost of neighbourhood methods

	User-based	**Item-based**
Similarity matrix	$m \times m$	$n \times n$
Offline cost (dense)	$O(m^2 n)$	$O(n^2 m)$
Memory	$O(m^2)$	$O(n^2)$, or $O(nk)$ keeping top-k
Online prediction	$O(k)$ per (u, i)	$O(k)$ per (u, i)
MovieLens small (m=610, n=9,724)	3.7×10^5 entries	9.5×10^7 entries
Amazon-like (m=3×10^8, n=10^7)	9×10^{16}	10^{14}, sparse, precomputable

Computational cost of neighbourhood methods

	User-based	**Item-based**
Similarity matrix	$m \times m$	$n \times n$
Offline cost (dense)	$O(m^2 n)$	$O(n^2 m)$
Memory	$O(m^2)$	$O(n^2)$, or $O(nk)$ keeping top-k
Online prediction	$O(k)$ per (u, i)	$O(k)$ per (u, i)
MovieLens small (m=610, n=9,724)	3.7×10^5 entries	9.5×10^7 entries
Amazon-like (m=3×10^8, n=10^7)	9×10^{16}	10^{14}, sparse, precomputable

- On MovieLens, user-based is *cheaper*; at web scale, item-based wins

Computational cost of neighbourhood methods

	User-based	**Item-based**
Similarity matrix	$m \times m$	$n \times n$
Offline cost (dense)	$O(m^2 n)$	$O(n^2 m)$
Memory	$O(m^2)$	$O(n^2)$, or $O(nk)$ keeping top-k
Online prediction	$O(k)$ per (u, i)	$O(k)$ per (u, i)
MovieLens small (m=610, n=9,724)	3.7×10^5 entries	9.5×10^7 entries
Amazon-like (m=3×10^8, n=10^7)	9×10^{16}	10^{14}, sparse, precomputable

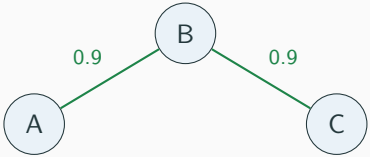
- On MovieLens, user-based is *cheaper*; at web scale, item-based wins
- Real systems store only the top-k neighbours per item, refreshed offline

Computational cost of neighbourhood methods

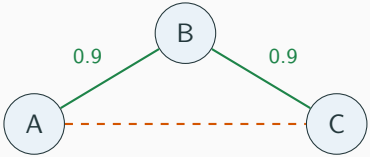
	User-based	**Item-based**
Similarity matrix	$m \times m$	$n \times n$
Offline cost (dense)	$O(m^2 n)$	$O(n^2 m)$
Memory	$O(m^2)$	$O(n^2)$, or $O(nk)$ keeping top-k
Online prediction	$O(k)$ per (u, i)	$O(k)$ per (u, i)
MovieLens small (m=610, n=9,724)	3.7×10^5 entries	9.5×10^7 entries
Amazon-like (m=3×10^8, n=10^7)	9×10^{16}	10^{14}, sparse, precomputable

- On MovieLens, user-based is *cheaper*; at web scale, item-based wins
- Real systems store only the top-k neighbours per item, refreshed offline
- Matrix factorization replaces the $O(n^2)$ table with $O((m+n)k)$ parameters

Indirect similarity



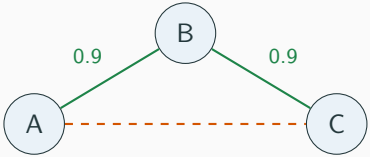
Indirect similarity



direct: ?? (few co-rated)

- Sparse data: A and C share almost no items, so their direct similarity is unreliable

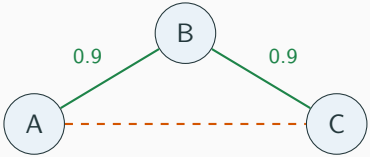
Indirect similarity



direct: ?? (few co-rated)

- Sparse data: A and C share almost no items, so their direct similarity is unreliable
- But both are very similar to B

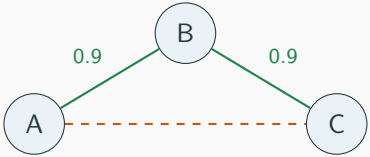
Indirect similarity



direct: ?? (few co-rated)

- Sparse data: A and C share almost no items, so their direct similarity is unreliable
- But both are very similar to B
- **Transferring similarity**: $t_{ij} = \epsilon \sum_v s_{iv} t_{vj} + s_{ij}$

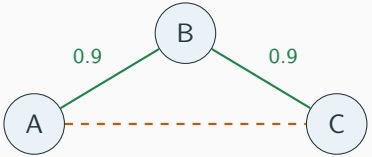
Indirect similarity



direct: ?? (few co-rated)

- Sparse data: A and C share almost no items, so their direct similarity is unreliable
- But both are very similar to B
- **Transferring similarity**: $t_{ij} = \epsilon \sum_v s_{iv} t_{vj} + s_{ij}$
- Matrix form: $T = (I - \epsilon S)^{-1} S$

Indirect similarity



direct: ?? (few co-rated)

- Sparse data: A and C share almost no items, so their direct similarity is unreliable
- But both are very similar to B
- **Transferring similarity**: $t_{ij} = \epsilon \sum_v s_{iv} t_{vj} + s_{ij}$
- Matrix form: $T = (I - \epsilon S)^{-1} S$

Helps neighbour selection and cold start. Latent factor models capture such indirect relations systematically.

Transferring similarity: a numeric example

Direct similarities S between 5 users (class notes):

$$S = \begin{bmatrix} 1 & 0.981 & 1.0 & 0.789 & 0.645 \\ 0.981 & 1 & 1.0 & 0.789 & 0.645 \\ 1.0 & 1.0 & 1 & 0.789 & 0.645 \\ 0.789 & 0.789 & 0.789 & 1 & 0.645 \\ 0.645 & 0.645 & 0.645 & 0.645 & 1 \end{bmatrix}$$

Transferring similarity: a numeric example

Direct similarities S between 5 users (class notes):

$$S = \begin{bmatrix} 1 & 0.981 & 1.0 & 0.789 & 0.645 \\ 0.981 & 1 & 1.0 & 0.789 & 0.645 \\ 1.0 & 1.0 & 1 & 0.789 & 0.645 \\ 0.789 & 0.789 & 0.789 & 1 & 0.645 \\ 0.645 & 0.645 & 0.645 & 0.645 & 1 \end{bmatrix}$$

With $\epsilon = 0.01$: $T = (I - \epsilon S)^{-1} S$

$$T = \begin{bmatrix} 1.0418 & 1.0228 & 1.0420 & 0.8261 & 0.6771 \\ 1.0228 & 1.0418 & 1.0420 & 0.8261 & 0.6771 \\ 1.0420 & 1.0420 & 1.0421 & 0.8263 & 0.6773 \\ 0.8261 & 0.8261 & 0.8263 & 1.0342 & 0.6744 \\ 0.6771 & 0.6771 & 0.6773 & 0.6744 & 1.0277 \end{bmatrix}$$

Transferring similarity: a numeric example

Direct similarities S between 5 users (class notes):

$$S = \begin{bmatrix} 1 & 0.981 & 1.0 & 0.789 & 0.645 \\ 0.981 & 1 & 1.0 & 0.789 & 0.645 \\ 1.0 & 1.0 & 1 & 0.789 & 0.645 \\ 0.789 & 0.789 & 0.789 & 1 & 0.645 \\ 0.645 & 0.645 & 0.645 & 0.645 & 1 \end{bmatrix}$$

With $\epsilon = 0.01$: $T = (I - \epsilon S)^{-1} S$

$$T = \begin{bmatrix} 1.0418 & 1.0228 & 1.0420 & 0.8261 & 0.6771 \\ 1.0228 & 1.0418 & 1.0420 & 0.8261 & 0.6771 \\ 1.0420 & 1.0420 & 1.0421 & 0.8263 & 0.6773 \\ 0.8261 & 0.8261 & 0.8263 & 1.0342 & 0.6744 \\ 0.6771 & 0.6771 & 0.6773 & 0.6744 & 1.0277 \end{bmatrix}$$

- Every entry grows a little: each pair also gets credit for paths through the other users

Transferring similarity: a numeric example

Direct similarities S between 5 users (class notes):

$$S = \begin{bmatrix} 1 & 0.981 & 1.0 & 0.789 & 0.645 \\ 0.981 & 1 & 1.0 & 0.789 & 0.645 \\ 1.0 & 1.0 & 1 & 0.789 & 0.645 \\ 0.789 & 0.789 & 0.789 & 1 & 0.645 \\ 0.645 & 0.645 & 0.645 & 0.645 & 1 \end{bmatrix}$$

With $\epsilon = 0.01$: $T = (I - \epsilon S)^{-1} S$

$$T = \begin{bmatrix} 1.0418 & 1.0228 & 1.0420 & 0.8261 & 0.6771 \\ 1.0228 & 1.0418 & 1.0420 & 0.8261 & 0.6771 \\ 1.0420 & 1.0420 & 1.0421 & 0.8263 & 0.6773 \\ 0.8261 & 0.8261 & 0.8263 & 1.0342 & 0.6744 \\ 0.6771 & 0.6771 & 0.6773 & 0.6744 & 1.0277 \end{bmatrix}$$

- Every entry grows a little: each pair also gets credit for paths through the other users
- Use T in place of S: $\hat{r}_{ui} = \dfrac{\sum_{v \in N(u)} t_{uv}\, r_{vi}}{\sum_{v \in N(u)} |t_{uv}|}$ — for prediction, neighbour selection and cold start

Transferring similarity: a numeric example

Direct similarities S between 5 users (class notes):

$$S = \begin{bmatrix} 1 & 0.981 & 1.0 & 0.789 & 0.645 \\ 0.981 & 1 & 1.0 & 0.789 & 0.645 \\ 1.0 & 1.0 & 1 & 0.789 & 0.645 \\ 0.789 & 0.789 & 0.789 & 1 & 0.645 \\ 0.645 & 0.645 & 0.645 & 0.645 & 1 \end{bmatrix}$$

With $\epsilon = 0.01$: $T = (I - \epsilon S)^{-1} S$

$$T = \begin{bmatrix} 1.0418 & 1.0228 & 1.0420 & 0.8261 & 0.6771 \\ 1.0228 & 1.0418 & 1.0420 & 0.8261 & 0.6771 \\ 1.0420 & 1.0420 & 1.0421 & 0.8263 & 0.6773 \\ 0.8261 & 0.8261 & 0.8263 & 1.0342 & 0.6744 \\ 0.6771 & 0.6771 & 0.6773 & 0.6744 & 1.0277 \end{bmatrix}$$

- Every entry grows a little: each pair also gets credit for paths through the other users
- Use T in place of S: $\hat{r}_{ui} = \dfrac{\sum_{v \in N(u)} t_{uv}\, r_{vi}}{\sum_{v \in N(u)} |t_{uv}|}$ — for prediction, neighbour selection and cold start
- Small ϵ keeps T close to S; a larger ϵ gives indirect paths more weight

Where neighbourhood methods hurt

- **Sparsity:** two users may share almost no rated movies $\Rightarrow$ similarity is noise

Where neighbourhood methods hurt

- **Sparsity:** two users may share almost no rated movies $\Rightarrow$ similarity is noise
- **Scalability:** $O(n^2)$ pairwise similarities

Where neighbourhood methods hurt

- **Sparsity:** two users may share almost no rated movies $\Rightarrow$ similarity is noise
- **Scalability:** $O(n^2)$ pairwise similarities
- **Popularity bias:** popular items are neighbours of everything

Where neighbourhood methods hurt

- **Sparsity:** two users may share almost no rated movies $\Rightarrow$ similarity is noise
- **Scalability:** $O(n^2)$ pairwise similarities
- **Popularity bias:** popular items are neighbours of everything
- **Cold start:** a new user has no neighbours at all

Where neighbourhood methods hurt

- **Sparsity:** two users may share almost no rated movies $\Rightarrow$ similarity is noise
- **Scalability:** $O(n^2)$ pairwise similarities
- **Popularity bias:** popular items are neighbours of everything
- **Cold start:** a new user has no neighbours at all
- **Shallow:** only captures direct overlap, not *why* people agree

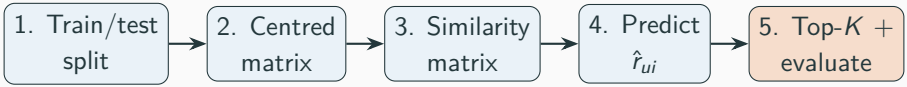
Where neighbourhood methods hurt

- **Sparsity:** two users may share almost no rated movies $\Rightarrow$ similarity is noise
- **Scalability:** $O(n^2)$ pairwise similarities
- **Popularity bias:** popular items are neighbours of everything
- **Cold start:** a new user has no neighbours at all
- **Shallow:** only captures direct overlap, not *why* people agree

Matrix factorization addresses the 1st, 2nd and 5th of these.

3. How Good Are Our Recommendations?

What we will build



- Data: 80% of each user's ratings for training, 20% held out for testing
- Model: mean-centred user-based CF with top-k positive neighbours
- Evaluation: RMSE, MAE, Precision@K, Recall@K, and NDCG@K

Step 1: Generate and Rank Recommendations

Suppose the model predicts ratings for five candidate movies that the user has not watched.

Movie	Predicted Rating
A	4.8
B	3.2
C	4.5
D	2.1
E	4.0

How do we generate the recommendation list?

1. Predict a rating for each candidate movie.
2. Sort the movies by predicted rating, highest first.
3. Recommend the first K movies.

Step 2: Which Recommendations Are Relevant?

After generating the ranking, we compare the movies with their actual ratings in the held-out test set.

Assume a movie is **relevant** if its actual rating is ≥ 4.

Movie	Predicted	Actual	Relevant?	Rank
A	4.8	5	Yes	1
C	4.5	2	No	2
E	4.0	4	Yes	3
B	3.2	4	Yes	4
D	2.1	1	No	5

- Top-3 recommendations: A, C, E
- Relevant recommended movies: A and E
- All relevant movies in the candidate set: A, B, E
- Total relevant movies: 3

Evaluating Rating Prediction: RMSE and MAE

These metrics measure how close predicted ratings are to actual ratings.

Let r_i be the actual rating and $\hat{r}_i$ be the predicted rating for movie i.

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(r_i - \hat{r}_i)^2}$$

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|r_i - \hat{r}_i|$$

- **MAE:** Average absolute prediction error.
- **RMSE:** Penalises larger prediction errors more strongly because errors are squared.
- **For both metrics, lower is better.**

Worked Example: Calculating RMSE and MAE

Use the predicted and actual ratings from our five-movie example.

Movie	r_i	$\hat{r}_i$	$\lvert r_i - \hat{r}_i \rvert$	$(r_i - \hat{r}_i)^2$
A	5	4.8	0.2	0.04
B	4	3.2	0.8	0.64
C	2	4.5	2.5	6.25
D	1	2.1	1.1	1.21
E	4	4.0	0.0	0.00
Total			**4.6**	**8.14**

$$\text{MAE} = \frac{4.6}{5} = \boxed{0.92}$$

$$\text{RMSE} = \sqrt{\frac{8.14}{5}} = \sqrt{1.628} \approx \boxed{1.276}$$

Interpretation: RMSE is higher because it penalises large errors more strongly.

Evaluating Ranking: Precision@K and Recall@K

For this example, a movie is relevant if its actual rating is ≥ 4.

$$\text{Precision@}K = \frac{|\text{Relevant} \cap \text{Top-}K|}{K}$$

$$\text{Recall@}K = \frac{|\text{Relevant} \cap \text{Top-}K|}{|\text{Relevant}|}$$

- **Precision:** Of everything we recommended, how much was relevant?
- **Recall:** Of everything relevant, how much did we recommend?

Rating prediction $\neq$ ranking
A model can have low RMSE but still produce a poor Top-10 list. RMSE evaluates rating errors; ranking metrics evaluate the relevance of the recommended items.

Worked Example: Precision@3 and Recall@3

Our recommendation list is: $\underbrace{A,\ C,\ E}_{\text{Top-3 recommendations}}$

- Relevant recommended movies: A and E
- Number of relevant recommended movies: 2
- Total relevant movies in the candidate set: 3 (A, B, E)

Precision@3 Two of the three recommended movies are relevant:
$\text{Precision@3} = \frac{2}{3} \approx \boxed{0.667}$

Recall@3 We retrieved two of the three relevant movies: $\text{Recall@3} = \frac{2}{3} \approx \boxed{0.667}$

NDCG@K: Does the Order Matter?

NDCG evaluates both the relevance of recommended items and their positions in the ranked list.

Rank	List A	List B	Ideal List
1	5-star	1-star	5-star
2	4-star	5-star	4-star
3	1-star	4-star	1-star

- All three lists contain the same movies.
- List A ranks the highly rated movies first.
- List B places a poorly rated movie first.
- The ideal list orders movies by actual relevance.

Key idea

A relevant movie near the top is more valuable than the same movie placed much lower in the list.

NDCG@K: Measuring Ranking Quality

Step 1: Discounted Cumulative Gain (DCG) DCG gives more importance to relevant movies appearing near the top of the ranked list.

$$\mathrm{DCG@}K = \sum_{p=1}^{K} \frac{2^{\mathrm{rel}_p} - 1}{\log_2(p+1)}$$

Here, rel_p is the relevance of the movie at rank p.

Step 2: Normalize the score

$$\boxed{\mathrm{NDCG@}K = \frac{\mathrm{DCG@}K}{\mathrm{IDCG@}K}}$$

NDCG rewards better ordering; a score closer to 1 indicates a ranking closer to the ideal order.

Summary: Which Metric Should We Use?

Metric	Question answered	Better value
RMSE	How far are predicted ratings from actual ratings?	Lower
MAE	What is the average absolute rating error?	Lower
Precision@K	What fraction of recommended items is relevant?	Higher
Recall@K	What fraction of relevant items was retrieved?	Higher
NDCG@K	Are the most relevant items near the top?	Higher

Key takeaway
RMSE and MAE evaluate rating prediction. Precision and Recall evaluate relevance retrieval. NDCG evaluates relevance and ranking order.

A model with low RMSE does not necessarily produce the best Top-K recommendations.

Beyond accuracy

Property	Definition	Why it matters
Coverage	Fraction of the catalogue that appears in any user's top-N	Long-tail items need exposure

Beyond accuracy

Property	Definition	Why it matters
Coverage	Fraction of the catalogue that appears in any user's top-N	Long-tail items need exposure
Diversity	$1-$ average pairwise similarity within a list	Ten near-identical films is a weak list

Beyond accuracy

Property	Definition	Why it matters
Coverage	Fraction of the catalogue that appears in any user's top-N	Long-tail items need exposure
Diversity	$1-$ average pairwise similarity within a list	Ten near-identical films is a weak list
Novelty	Average $-\log_2 p(i)$, $p(i)$ = item popularity	Telling users what they already know adds little

Beyond accuracy

Property	Definition	Why it matters
Coverage	Fraction of the catalogue that appears in any user's top-N	Long-tail items need exposure
Diversity	$1-$ average pairwise similarity within a list	Ten near-identical films is a weak list
Novelty	Average $-\log_2 p(i)$, $p(i)$ = item popularity	Telling users what they already know adds little
Serendipity	Relevant *and* unexpected given the user's history	Discovery keeps users engaged

Beyond accuracy

Property	Definition	Why it matters
Coverage	Fraction of the catalogue that appears in any user's top-N	Long-tail items need exposure
Diversity	$1-$ average pairwise similarity within a list	Ten near-identical films is a weak list
Novelty	Average $-\log_2 p(i)$, $p(i)$ = item popularity	Telling users what they already know adds little
Serendipity	Relevant *and* unexpected given the user's history	Discovery keeps users engaged
Personalisation	$1-$ average overlap between users' lists	Same list for all = popularity

Beyond accuracy

Property	Definition	Why it matters
Coverage	Fraction of the catalogue that appears in any user's top-N	Long-tail items need exposure
Diversity	$1-$ average pairwise similarity within a list	Ten near-identical films is a weak list
Novelty	Average $-\log_2 p(i)$, $p(i)$ = item popularity	Telling users what they already know adds little
Serendipity	Relevant *and* unexpected given the user's history	Discovery keeps users engaged
Personalisation	$1-$ average overlap between users' lists	Same list for all = popularity

Online, the final judge is an **A/B test**: click-through, watch time, retention.

Evaluation protocols

Random hold-out

- Hide a random 20% of ratings
- Simple; what we use today
- Leaks the future into training

Evaluation protocols

Random hold-out

- Hide a random 20% of ratings
- Simple; what we use today
- Leaks the future into training

Temporal split

- Train before time t, test after
- Mirrors deployment
- Harder; numbers are lower

Evaluation protocols

Random hold-out

- Hide a random 20% of ratings
- Simple; what we use today
- Leaks the future into training

Temporal split

- Train before time t, test after
- Mirrors deployment
- Harder; numbers are lower

Leave-one-out

- Hide each user's last interaction
- Rank it among sampled negatives
- Common for implicit data (Hit Rate@K)

Evaluation protocols

Random hold-out

- Hide a random 20% of ratings
- Simple; what we use today
- Leaks the future into training

Temporal split

- Train before time t, test after
- Mirrors deployment
- Harder; numbers are lower

Leave-one-out

- Hide each user's last interaction
- Rank it among sampled negatives
- Common for implicit data (Hit Rate@K)

- k-fold cross-validation averages over folds and gives a spread, not just one number

Evaluation protocols

Random hold-out

- Hide a random 20% of ratings
- Simple; what we use today
- Leaks the future into training

Temporal split

- Train before time t, test after
- Mirrors deployment
- Harder; numbers are lower

Leave-one-out

- Hide each user's last interaction
- Rank it among sampled negatives
- Common for implicit data (Hit Rate@K)

- k-fold cross-validation averages over folds and gives a spread, not just one number
- Tune hyper-parameters on a validation split, report once on the test split

4. Matrix Factorization: Discovering Hidden Preferences

A different idea

Our matrix is $610 \times 9{,}724$ and 98% empty.

A different idea

Our matrix is $610 \times 9{,}724$ and 98% empty.

Can we describe every user and every movie with just a handful of hidden numbers?

A different idea

Our matrix is $610 \times 9{,}724$ and 98% empty.

Can we describe every user and every movie
with just a handful of hidden numbers?

- Suppose taste is driven by a few factors: action, romance, "serious vs. light", era...

A different idea

Our matrix is $610 \times 9{,}724$ and 98% empty.

Can we describe every user and every movie
with just a handful of hidden numbers?

- Suppose taste is driven by a few factors: action, romance, "serious vs. light", era...
- Each **movie** has an amount of each factor

A different idea

Our matrix is $610 \times 9{,}724$ and 98% empty.

Can we describe every user and every movie
with just a handful of hidden numbers?

- Suppose taste is driven by a few factors: action, romance, "serious vs. light", era...
- Each **movie** has an amount of each factor
- Each **user** has an appetite for each factor

A different idea

Our matrix is $610 \times 9{,}724$ and 98% empty.

Can we describe every user and every movie
with just a handful of hidden numbers?

- Suppose taste is driven by a few factors: action, romance, "serious vs. light", era...
- Each **movie** has an amount of each factor
- Each **user** has an appetite for each factor
- Rating $\approx$ how well appetite matches content

Latent factor models

- Assume the observed data are driven by a few **unobserved (latent) factors**

Latent factor models

- Assume the observed data are driven by a few **unobserved (latent) factors**
- Find a **low-dimensional space** that preserves the important structure

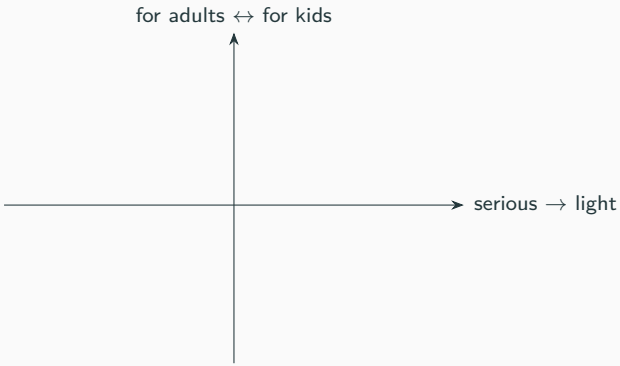
Latent factor models

- Assume the observed data are driven by a few **unobserved (latent) factors**
- Find a **low-dimensional space** that preserves the important structure
- Learn the **position** of every user and every movie in that space from observed ratings

Latent factor models

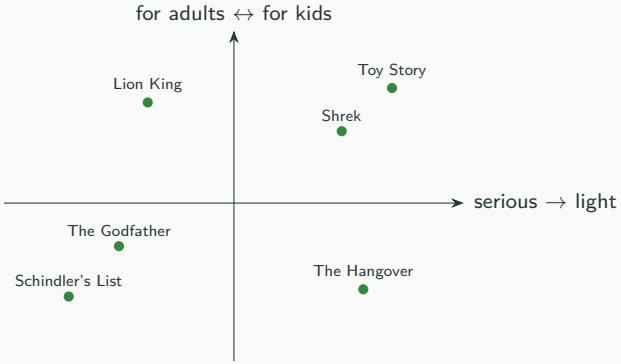
- Assume the observed data are driven by a few **unobserved (latent) factors**
- Find a **low-dimensional space** that preserves the important structure
- Learn the **position** of every user and every movie in that space from observed ratings
- Predict unseen ratings by how well a user's preferences **align** with a movie's characteristics

Latent factors: geometric intuition



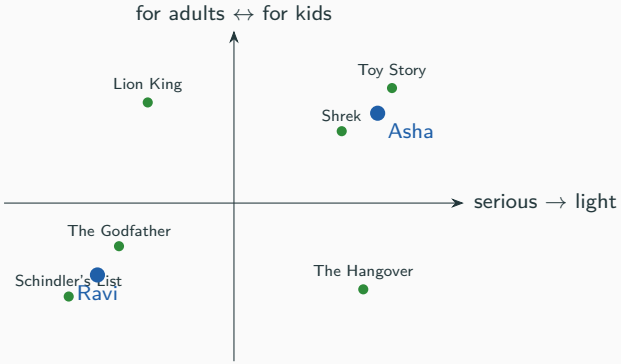
Users and movies live in the **same** space. Close together ⇒ high predicted rating.

Latent factors: geometric intuition



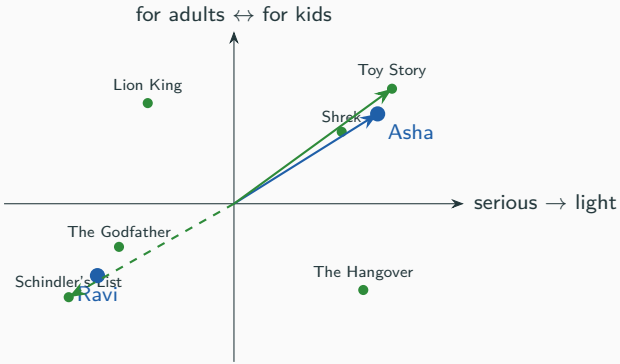
Users and movies live in the **same** space. Close together ⇒ high predicted rating.

Latent factors: geometric intuition



Users and movies live in the **same** space. Close together ⇒ high predicted rating.

Latent factors: geometric intuition



Low-rank intuition: a rank-2 ratings matrix

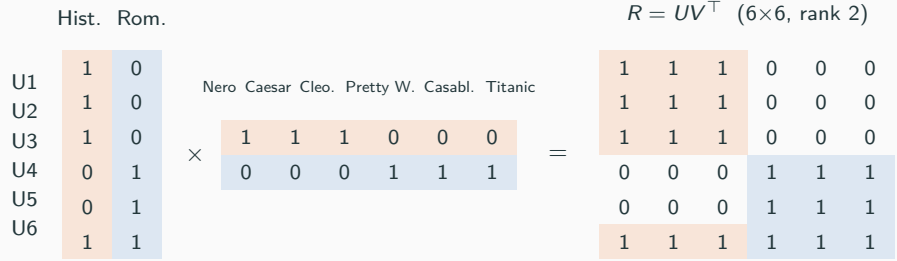
	Hist.	Rom.
U1	1	0
U2	1	0
U3	1	0
U4	0	1
U5	0	1
U6	1	1

$\times$

	Nero	Caesar	Cleo.	Pretty W.	Casabl.	Titanic
	1	1	1	0	0	0
	0	0	0	1	1	1

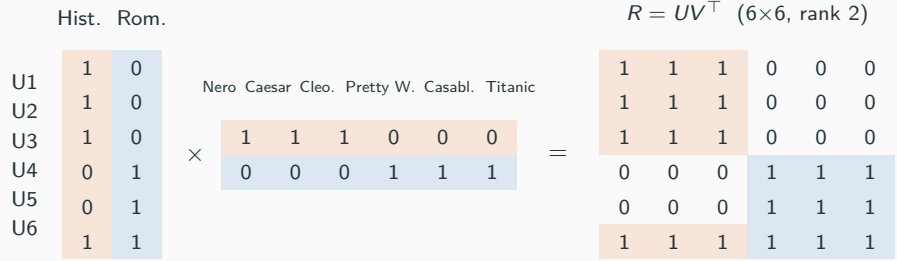
Idea from Aggarwal, *Recommender Systems: The Textbook*, 2016, Ch. 3.

Low-rank intuition: a rank-2 ratings matrix



Idea from Aggarwal, *Recommender Systems: The Textbook*, 2016, Ch. 3.

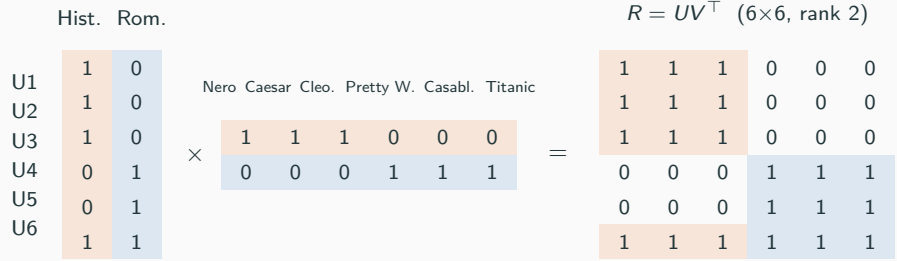
Low-rank intuition: a rank-2 ratings matrix



- 36 numbers in R are fully described by $6 \times 2 + 2 \times 6 = 24$ numbers — and much fewer for big matrices

Idea from Aggarwal, *Recommender Systems: The Textbook*, 2016, Ch. 3.

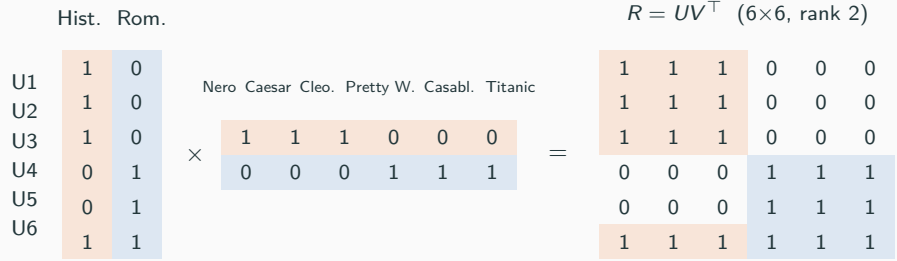
Low-rank intuition: a rank-2 ratings matrix



- 36 numbers in R are fully described by $6 \times 2 + 2 \times 6 = 24$ numbers — and much fewer for big matrices
- If some entries of R were missing, the block structure would still tell us what they must be

Idea from Aggarwal, *Recommender Systems: The Textbook*, 2016, Ch. 3.

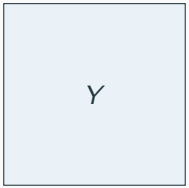
Low-rank intuition: a rank-2 ratings matrix



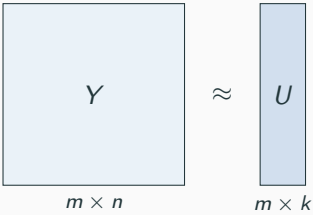
- 36 numbers in R are fully described by $6 \times 2 + 2 \times 6 = 24$ numbers — and much fewer for big matrices
- If some entries of R were missing, the block structure would still tell us what they must be
- Real ratings are *approximately* low-rank: a few tastes explain most of the variation

Idea from Aggarwal, *Recommender Systems: The Textbook*, 2016, Ch. 3.

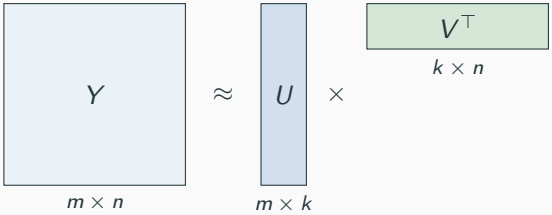
The model



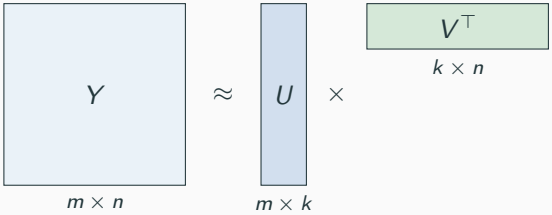
The model



The model



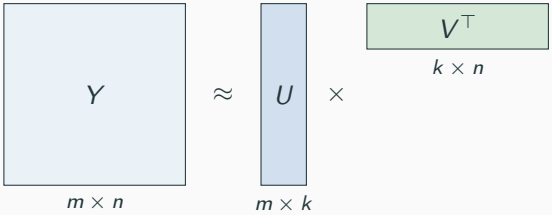
The model



$$\hat{y}_{ij} = \mathbf{u}_i^\top \mathbf{v}_j = \sum_{s=1}^{k} u_{is}\, v_{js}$$

u_{is} = how much user i likes concept s; v_{js} = how much of concept s item j has

The model

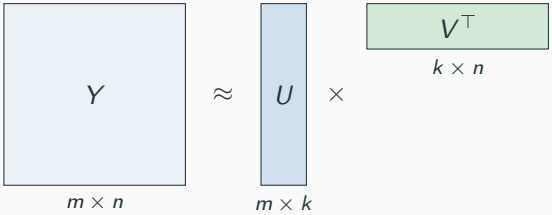


$$\hat{y}_{ij} = \mathbf{u}_i^\top \mathbf{v}_j = \sum_{s=1}^{k} u_{is}\, v_{js}$$

u_{is} = how much user i likes concept s; v_{js} = how much of concept s item j has

- $k \ll m, n$ (typically 20–200)

The model



$$\hat{y}_{ij} = \mathbf{u}_i^\top \mathbf{v}_j = \sum_{s=1}^{k} u_{is}\, v_{js}$$

u_{is} = how much user i likes concept s; v_{js} = how much of concept s item j has

- $k \ll m, n$ (typically 20–200)
- MovieLens: $(610 + 9{,}724) \times 50 \approx 0.5$M parameters instead of 5.9M cells

A toy example: 5 users, 4 movies

Y	M1	M2	M3	M4
U1	5	3	?	1
U2	4	?	?	1
U3	1	1	?	5
U4	1	?	?	4
U5	?	1	5	4

A toy example: 5 users, 4 movies

Y	M1	M2	M3	M4
U1	5	3	?	1
U2	4	?	?	1
U3	1	1	?	5
U4	1	?	?	4
U5	?	1	5	4

U	Action	Romance
U1	2.13	−0.49
U2	1.73	−0.28
U3	1.06	2.01
U4	0.92	1.57
U5	1.06	1.61

A toy example: 5 users, 4 movies

Y	M1	M2	M3	M4
U1	5	3	?	1
U2	4	?	?	1
U3	1	1	?	5
U4	1	?	?	4
U5	?	1	5	4

U	Action	Romance
U1	2.13	−0.49
U2	1.73	−0.28
U3	1.06	2.01
U4	0.92	1.57
U5	1.06	1.61

$V^\top$	M1	M2	M3	M4
Action	2.18	1.34	1.44	0.90
Romance	−0.64	−0.23	2.07	1.96

A toy example: 5 users, 4 movies

Y	M1	M2	M3	M4
U1	5	3	?	1
U2	4	?	?	1
U3	1	1	?	5
U4	1	?	?	4
U5	?	1	5	4

U	Action	Romance
U1	2.13	−0.49
U2	1.73	−0.28
U3	1.06	2.01
U4	0.92	1.57
U5	1.06	1.61

$V^\top$	M1	M2	M3	M4
Action	2.18	1.34	1.44	0.90
Romance	−0.64	−0.23	2.07	1.96

$\hat{Y} = UV^\top$	M1	M2	M3	M4
U1	4.98	2.98	2.07	0.98
U2	3.97	2.39	1.90	1.01
U3	1.02	0.95	5.70	4.92
U4	1.00	0.87	4.59	3.93
U5	1.28	1.05	4.88	4.14

A toy example: 5 users, 4 movies

Y	M1	M2	M3	M4
U1	5	3	?	1
U2	4	?	?	1
U3	1	1	?	5
U4	1	?	?	4
U5	?	1	5	4

U	Action	Romance
U1	2.13	−0.49
U2	1.73	−0.28
U3	1.06	2.01
U4	0.92	1.57
U5	1.06	1.61

$V^\top$	M1	M2	M3	M4
Action	2.18	1.34	1.44	0.90
Romance	−0.64	−0.23	2.07	1.96

$\hat{Y} = UV^\top$	M1	M2	M3	M4
U1	4.98	2.98	2.07	0.98
U2	3.97	2.39	1.90	1.01
U3	1.02	0.95	5.70	4.92
U4	1.00	0.87	4.59	3.93
U5	1.28	1.05	4.88	4.14

U1, U2 are action fans ⇒ low prediction for romantic M3.

U3, U4 like romance ⇒ M3 predicted high.

The learning problem

Find U, V that reproduce the **observed** entries:

$$\min_{U,V} \; J(U,V) = \sum_{(i,j)\in\Omega} \text{Loss}\big(y_{ij},\; \mathbf{u}_i^\top \mathbf{v}_j\big)$$

The learning problem

Find U, V that reproduce the **observed** entries:

$$\min_{U,V} J(U,V) = \sum_{(i,j)\in\Omega} \text{Loss}(y_{ij},\ \mathbf{u}_i^\top \mathbf{v}_j)$$

Add regularisation to avoid overfitting:

$$J(U,V) = \sum_{(i,j)\in\Omega} \left(y_{ij} - \mathbf{u}_i^\top \mathbf{v}_j\right)^2 + \lambda(\|U\|_F^2 + \|V\|_F^2)$$

The learning problem

Find U, V that reproduce the **observed** entries:

$$\min_{U,V} J(U,V) = \sum_{(i,j)\in\Omega} \text{Loss}(y_{ij},\ \mathbf{u}_i^\top \mathbf{v}_j)$$

Add regularisation to avoid overfitting:

$$J(U,V) = \sum_{(i,j)\in\Omega} (y_{ij} - \mathbf{u}_i^\top \mathbf{v}_j)^2 + \lambda(\|U\|_F^2 + \|V\|_F^2)$$

Solve by gradient descent:

$$U_{t+1} = U_t - c\,\frac{\partial J}{\partial U}, \qquad V_{t+1} = V_t - c\,\frac{\partial J}{\partial V}$$

The learning problem

Find U, V that reproduce the **observed** entries:

$$\min_{U,V} J(U,V) = \sum_{(i,j)\in\Omega} \text{Loss}(y_{ij},\ \mathbf{u}_i^\top \mathbf{v}_j)$$

Add regularisation to avoid overfitting:

$$J(U,V) = \sum_{(i,j)\in\Omega} \left(y_{ij} - \mathbf{u}_i^\top \mathbf{v}_j\right)^2 + \lambda(\|U\|_F^2 + \|V\|_F^2)$$

Solve by gradient descent:

$$U_{t+1} = U_t - c\,\frac{\partial J}{\partial U}, \qquad V_{t+1} = V_t - c\,\frac{\partial J}{\partial V}$$

Only the $|\Omega|$ known cells appear in J — the "?"s never enter training.

Deriving the gradient

For one observed rating (u, i), let $e_{ui} = r_{ui} - \mathbf{p}_u^\top \mathbf{q}_i$ and

$$\ell_{ui} = e_{ui}^2 + \lambda(\|\mathbf{p}_u\|^2 + \|\mathbf{q}_i\|^2)$$

Deriving the gradient

For one observed rating (u, i), let $e_{ui} = r_{ui} - \mathbf{p}_u^\top \mathbf{q}_i$ and

$$\ell_{ui} = e_{ui}^2 + \lambda(\|\mathbf{p}_u\|^2 + \|\mathbf{q}_i\|^2)$$

Using $\partial(\mathbf{p}_u^\top \mathbf{q}_i)/\partial \mathbf{p}_u = \mathbf{q}_i$:

$$\frac{\partial \ell_{ui}}{\partial \mathbf{p}_u} = 2e_{ui} \cdot (-\mathbf{q}_i) + 2\lambda \mathbf{p}_u = -2(e_{ui}\,\mathbf{q}_i - \lambda\,\mathbf{p}_u)$$

Deriving the gradient

For one observed rating (u, i), let $e_{ui} = r_{ui} - \mathbf{p}_u^\top \mathbf{q}_i$ and

$$\ell_{ui} = e_{ui}^2 + \lambda(\|\mathbf{p}_u\|^2 + \|\mathbf{q}_i\|^2)$$

Using $\partial(\mathbf{p}_u^\top \mathbf{q}_i)/\partial \mathbf{p}_u = \mathbf{q}_i$:

$$\frac{\partial \ell_{ui}}{\partial \mathbf{p}_u} = 2e_{ui} \cdot (-\mathbf{q}_i) + 2\lambda \mathbf{p}_u = -2(e_{ui}\, \mathbf{q}_i - \lambda\, \mathbf{p}_u)$$

By symmetry: $\dfrac{\partial \ell_{ui}}{\partial \mathbf{q}_i} = -2(e_{ui}\, \mathbf{p}_u - \lambda\, \mathbf{q}_i)$

Deriving the gradient

For one observed rating (u, i), let $e_{ui} = r_{ui} - \mathbf{p}_u^\top \mathbf{q}_i$ and

$$\ell_{ui} = e_{ui}^2 + \lambda(\|\mathbf{p}_u\|^2 + \|\mathbf{q}_i\|^2)$$

Using $\partial(\mathbf{p}_u^\top \mathbf{q}_i)/\partial \mathbf{p}_u = \mathbf{q}_i$:

$$\frac{\partial \ell_{ui}}{\partial \mathbf{p}_u} = 2e_{ui} \cdot (-\mathbf{q}_i) + 2\lambda \mathbf{p}_u = -2(e_{ui}\,\mathbf{q}_i - \lambda\,\mathbf{p}_u)$$

By symmetry: $\dfrac{\partial \ell_{ui}}{\partial \mathbf{q}_i} = -2(e_{ui}\,\mathbf{p}_u - \lambda\,\mathbf{q}_i)$

Gradient step with rate η (the factor 2 absorbed into η):

$$\mathbf{p}_u \leftarrow \mathbf{p}_u + \eta(e_{ui}\,\mathbf{q}_i - \lambda\,\mathbf{p}_u), \qquad \mathbf{q}_i \leftarrow \mathbf{q}_i + \eta(e_{ui}\,\mathbf{p}_u - \lambda\,\mathbf{q}_i)$$

Deriving the gradient

For one observed rating (u, i), let $e_{ui} = r_{ui} - \mathbf{p}_u^\top \mathbf{q}_i$ and

$$\ell_{ui} = e_{ui}^2 + \lambda(\|\mathbf{p}_u\|^2 + \|\mathbf{q}_i\|^2)$$

Using $\partial(\mathbf{p}_u^\top \mathbf{q}_i)/\partial \mathbf{p}_u = \mathbf{q}_i$:

$$\frac{\partial \ell_{ui}}{\partial \mathbf{p}_u} = 2e_{ui} \cdot (-\mathbf{q}_i) + 2\lambda \mathbf{p}_u = -2(e_{ui}\,\mathbf{q}_i - \lambda\,\mathbf{p}_u)$$

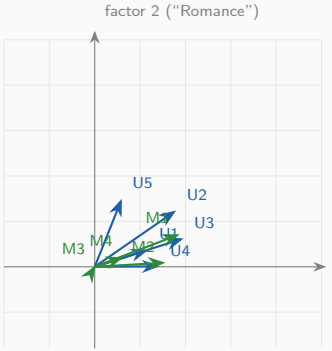
By symmetry: $\dfrac{\partial \ell_{ui}}{\partial \mathbf{q}_i} = -2(e_{ui}\,\mathbf{p}_u - \lambda\,\mathbf{q}_i)$

Gradient step with rate η (the factor 2 absorbed into η):

$$\mathbf{p}_u \leftarrow \mathbf{p}_u + \eta(e_{ui}\,\mathbf{q}_i - \lambda\,\mathbf{p}_u), \qquad \mathbf{q}_i \leftarrow \mathbf{q}_i + \eta(e_{ui}\,\mathbf{p}_u - \lambda\,\mathbf{q}_i)$$

Both updates use the *old* values of $\mathbf{p}_u$ and $\mathbf{q}_i$.

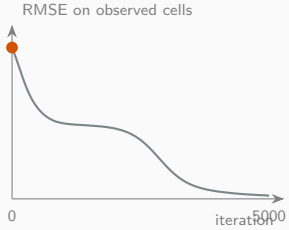
Watch gradient descent learn the toy example



$\hat{Y}$	M1	M2	M3	M4
U1	0.62	0.47	0.01	0.21
U2	1.06	0.73	0.02	0.35
U3	1.03	0.78	0.02	0.34
U4	0.67	0.56	0.01	0.23
U5	0.55	0.27	0.02	0.17

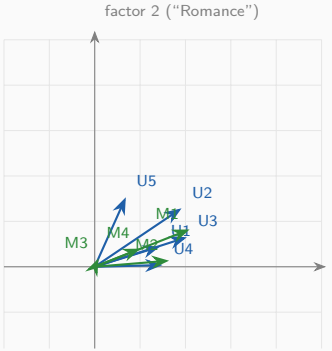
Iteration 0

RMSE = 2.927



Blue arrows: user vectors $\mathbf{u}_i$. Green arrows: movie vectors $\mathbf{v}_j$. Shaded cells in $\hat{Y}$ are the unknown ratings being predicted. SGD, k=2, step 0.0002, λ=0.02

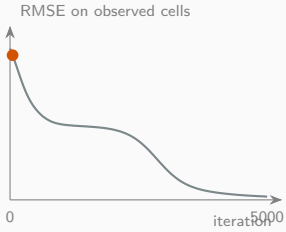
Watch gradient descent learn the toy example



$\hat{Y}$	M1	M2	M3	M4
U1	0.84	0.61	0.05	0.41
U2	1.26	0.83	0.10	0.61
U3	1.20	0.86	0.07	0.58
U4	0.79	0.61	0.03	0.38
U5	0.67	0.33	0.09	0.33

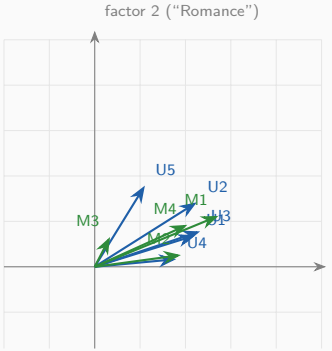
Iteration 50

RMSE = 2.799



Blue arrows: user vectors $\mathbf{u}_i$. Green arrows: movie vectors $\mathbf{v}_j$. Shaded cells in $\hat{Y}$ are the unknown ratings being predicted. SGD, k=2, step 0.0002, λ=0.02

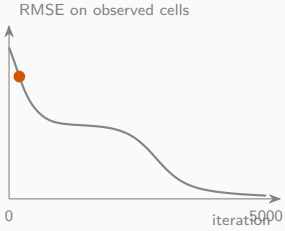
Watch gradient descent learn the toy example



$\hat{Y}$	M1	M2	M3	M4
U1	1.70	1.10	0.30	1.29
U2	1.93	1.16	0.42	1.48
U3	1.80	1.15	0.32	1.37
U4	1.27	0.86	0.18	0.96
U5	1.25	0.64	0.39	0.98

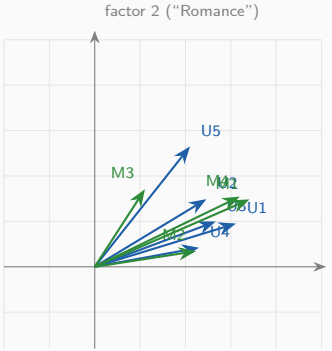
Iteration 200

RMSE = 2.367



Blue arrows: user vectors $\mathbf{u}_i$. Green arrows: movie vectors $\mathbf{v}_j$. Shaded cells in $\hat{Y}$ are the unknown ratings being predicted. SGD, k=2, step 0.0002, λ=0.02

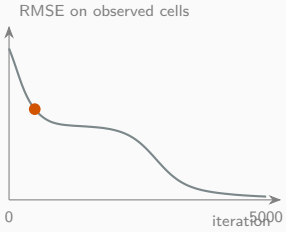
Watch gradient descent learn the toy example



$\hat{Y}$	M1	M2	M3	M4
U1	3.01	1.82	1.27	2.86
U2	2.65	1.50	1.32	2.55
U3	2.65	1.58	1.17	2.52
U4	2.12	1.32	0.82	2.00
U5	2.78	1.40	1.72	2.71

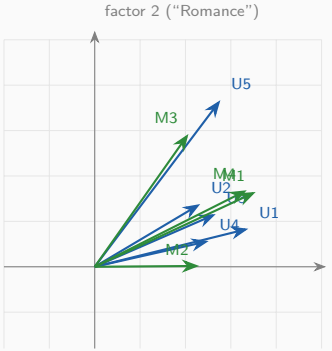
Iteration 500

RMSE = 1.755



Blue arrows: user vectors $\mathbf{u}_i$. Green arrows: movie vectors $\mathbf{v}_j$. Shaded cells in $\hat{Y}$ are the unknown ratings being predicted. SGD, k=2, step 0.0002, λ=0.02

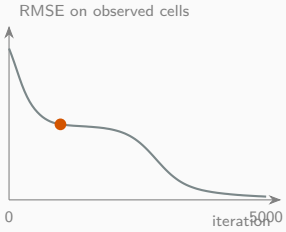
Watch gradient descent learn the toy example



$\hat{Y}$	M1	M2	M3	M4
U1	3.35	1.95	2.36	3.19
U2	2.63	1.35	2.21	2.53
U3	2.85	1.54	2.23	2.73
U4	2.46	1.45	1.71	2.34
U5	3.97	1.61	4.11	3.87

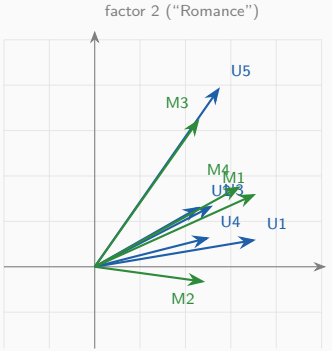
Iteration 1000

RMSE = 1.461



Blue arrows: user vectors $\mathbf{u}_i$. Green arrows: movie vectors $\mathbf{v}_j$. Shaded cells in $\hat{Y}$ are the unknown ratings being predicted. SGD, k=2, step 0.0002, λ=0.02

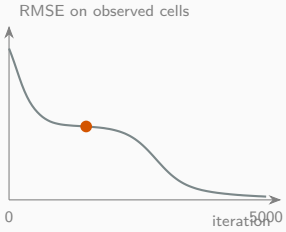
Watch gradient descent learn the toy example



$\hat{Y}$	M1	M2	M3	M4
U1	3.40	2.11	2.53	3.13
U2	2.60	1.31	2.42	2.46
U3	2.86	1.47	2.60	2.70
U4	2.51	1.49	1.99	2.33
U5	4.04	1.35	4.81	3.98

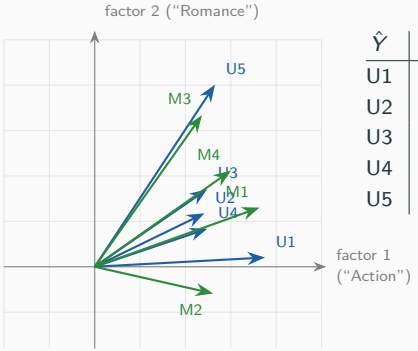
Iteration 1500

RMSE = 1.419



Blue arrows: user vectors $\mathbf{u}_i$. Green arrows: movie vectors $\mathbf{v}_j$. Shaded cells in $\hat{Y}$ are the unknown ratings being predicted. SGD, k=2, step 0.0002, λ=0.02

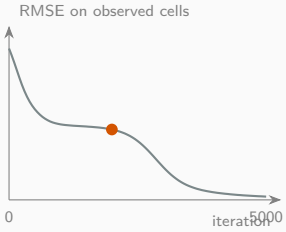
Watch gradient descent learn the toy example



$\hat{Y}$	M1	M2	M3	M4
U1	3.48	2.42	2.39	2.93
U2	2.58	1.41	2.41	2.44
U3	2.81	1.37	2.89	2.77
U4	2.53	1.50	2.17	2.31
U5	3.72	1.14	4.92	4.12

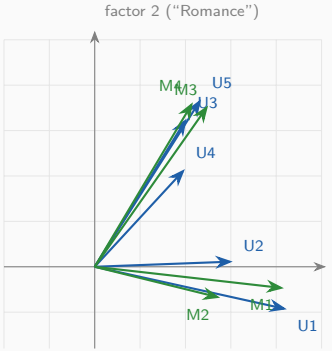
Iteration 2000

RMSE = 1.361



Blue arrows: user vectors $\mathbf{u}_i$. Green arrows: movie vectors $\mathbf{v}_j$. Shaded cells in $\hat{Y}$ are the unknown ratings being predicted. SGD, k=2, step 0.0002, λ=0.02

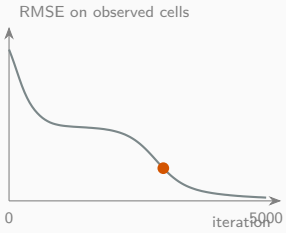
Watch gradient descent learn the toy example



$\hat{Y}$	M1	M2	M3	M4
U1	4.52	3.09	1.81	1.44
U2	3.16	2.09	2.00	1.75
U3	1.73	0.84	4.17	4.05
U4	1.82	1.01	3.15	3.03
U5	2.01	0.99	4.74	4.61

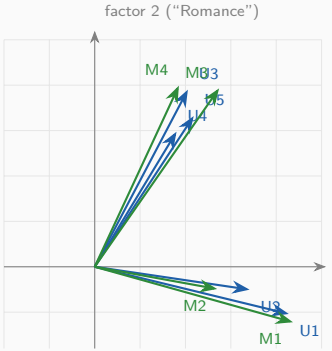
Iteration 3000

RMSE = 0.633



Blue arrows: user vectors $\mathbf{u}_i$. Green arrows: movie vectors $\mathbf{v}_j$. Shaded cells in $\hat{Y}$ are the unknown ratings being predicted. SGD, k=2, step 0.0002, λ=0.02

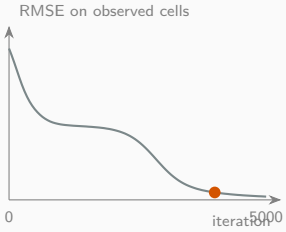
Watch gradient descent learn the toy example



$\hat{Y}$	M1	M2	M3	M4
U1	4.99	3.01	1.91	0.95
U2	3.88	2.37	1.84	1.08
U3	1.05	0.91	5.24	4.84
U4	1.06	0.85	4.16	3.81
U5	1.37	1.07	4.76	4.32

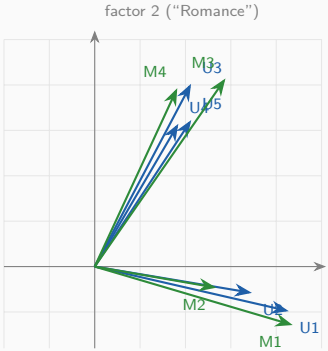
Iteration 4000

RMSE = 0.144



Blue arrows: user vectors $\mathbf{u}_i$. Green arrows: movie vectors $\mathbf{v}_j$. Shaded cells in $\hat{Y}$ are the unknown ratings being predicted. SGD, k=2, step 0.0002, λ=0.02

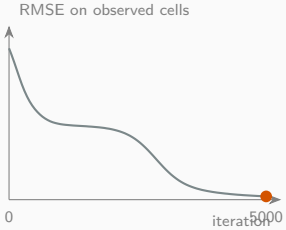
Watch gradient descent learn the toy example



$\hat{Y}$	M1	M2	M3	M4
U1	4.98	2.98	2.07	0.98
U2	3.97	2.39	1.90	1.01
U3	1.02	0.95	5.70	4.92
U4	1.00	0.87	4.59	3.93
U5	1.28	1.05	4.88	4.14

Iteration 5000

RMSE = 0.064



Blue arrows: user vectors $\mathbf{u}_i$. Green arrows: movie vectors $\mathbf{v}_j$. Shaded cells in $\hat{Y}$ are the unknown ratings being predicted. SGD, k=2, step 0.0002, λ=0.02

Latent vectors initially (random start)

U	f_1	f_2
U1	0.59	0.19
U2	0.90	0.62
U3	0.98	0.31
U4	0.71	0.01
U5	0.30	0.75

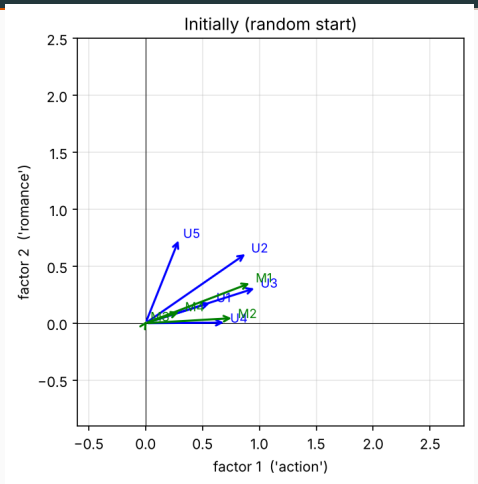
$V^\top$	M1	M2	M3	M4
f_1	0.94	0.78	0.01	0.32
f_2	0.36	0.05	0.02	0.10

$\hat{Y} = UV^\top$	M1	M2	M3	M4
U1	0.62	0.47	0.01	0.21
U2	1.06	0.73	0.02	0.35
U3	1.03	0.78	0.02	0.34
U4	0.67	0.56	0.01	0.23
U5	0.55	0.27	0.02	0.17

Error on the 12 known ratings: **2.927**

Random start: short arrows, all in the upper-right quarter; every prediction is near 0–1.

Shaded cells: unknown ratings.



Blue: users (rows of U) Green: movies (rows of V)

Latent vectors after 1,500 iterations

U	f_1	f_2
U1	1.78	0.30
U2	1.16	0.66
U3	1.30	0.67
U4	1.27	0.32
U5	1.38	1.98

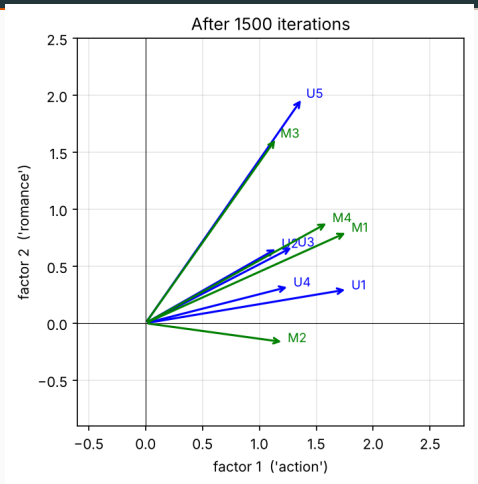
$V^\top$	M1	M2	M3	M4
f_1	1.78	1.22	1.15	1.61
f_2	0.80	−0.17	1.63	0.89

$\hat{Y} = UV^\top$	M1	M2	M3	M4
U1	3.40	2.11	2.53	3.13
U2	2.60	1.31	2.42	2.46
U3	2.86	1.47	2.60	2.70
U4	2.51	1.49	1.99	2.33
U5	4.04	1.35	4.81	3.98

Error on the 12 known ratings: **1.419**

Arrows grow and begin to separate; U5 and M3 move towards the romance axis.

Shaded cells: unknown ratings.



Blue: users (rows of U) Green: movies (rows of V)

Latent vectors after 3,000 iterations

U	f_1	f_2
U1	2.12	−0.47
U2	1.52	0.06
U3	1.01	1.63
U4	0.99	1.08
U5	1.17	1.85

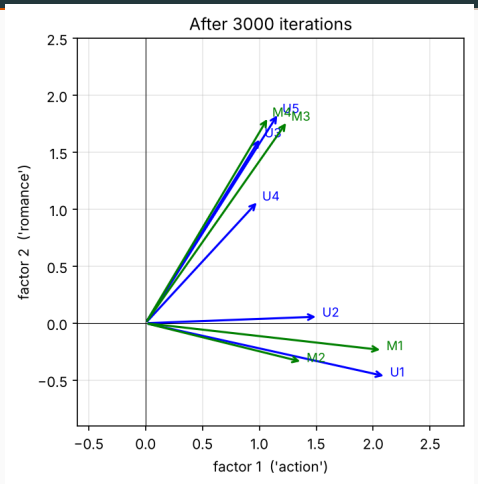
$V^\top$	M1	M2	M3	M4
f_1	2.09	1.38	1.25	1.08
f_2	−0.24	−0.34	1.78	1.81

$\hat{Y} = UV^\top$	M1	M2	M3	M4
U1	4.52	3.09	1.81	1.44
U2	3.16	2.09	2.00	1.75
U3	1.73	0.84	4.17	4.05
U4	1.82	1.01	3.15	3.03
U5	2.01	0.99	4.74	4.61

Error on the 12 known ratings: **0.633**

Two groups appear: U1, U2 with M1, M2 and U3, U4, U5 with M3, M4.

Shaded cells: unknown ratings.



Blue: users (rows of U) Green: movies (rows of V)

Latent vectors after 5,000 iterations

U	f_1	f_2
U1	2.14	−0.49
U2	1.73	−0.29
U3	1.06	2.01
U4	0.92	1.57
U5	1.06	1.61

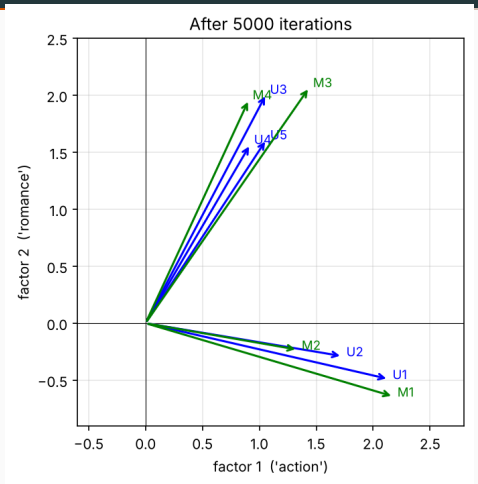
$V^\top$	M1	M2	M3	M4
f_1	2.18	1.34	1.44	0.91
f_2	−0.64	−0.23	2.07	1.96

$\hat{Y} = UV^\top$	M1	M2	M3	M4
U1	4.98	2.98	2.07	0.98
U2	3.97	2.39	1.90	1.01
U3	1.02	0.95	5.70	4.92
U4	1.00	0.87	4.59	3.93
U5	1.28	1.05	4.88	4.14

Error on the 12 known ratings: **0.064**

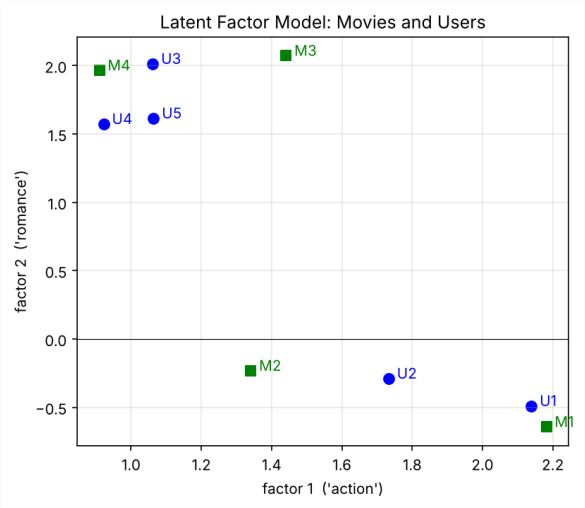
Converged: known ratings fitted (error 0.06); unknown cells are the predictions.

Shaded cells: unknown ratings.



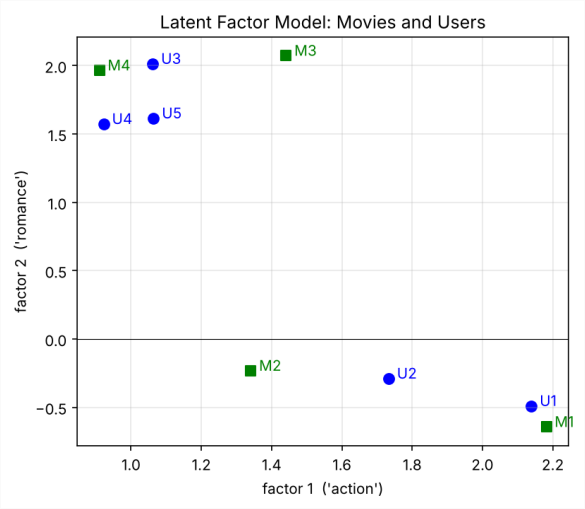
Blue: users (rows of U) Green: movies (rows of V)

Latent factor model: movies and users



Final factors after 5,000 iterations:

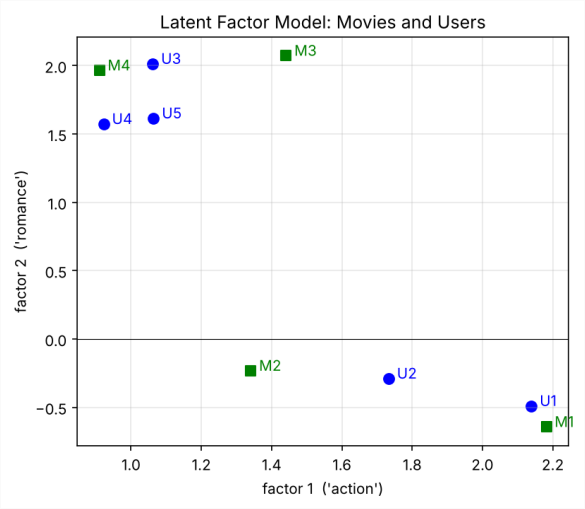
Latent factor model: movies and users



Final factors after 5,000 iterations:

- U1, U2 sit beside M1, M2: high *action*, negative *romance*

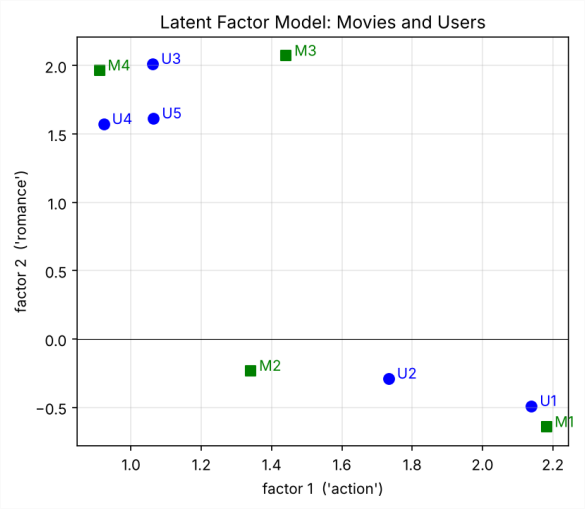
Latent factor model: movies and users



Final factors after 5,000 iterations:

- U1, U2 sit beside M1, M2: high *action*, negative *romance*
- U3, U4, U5 sit beside M3, M4: high *romance*

Latent factor model: movies and users

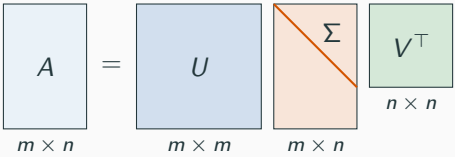


Final factors after 5,000 iterations:

- U1, U2 sit beside M1, M2: high *action*, negative *romance*
- U3, U4, U5 sit beside M3, M4: high *romance*
- Unknown U3–M3 $\Rightarrow$ predicted **5.70**; unknown U1–M3 $\Rightarrow$ **2.07**

Singular Value Decomposition (SVD)

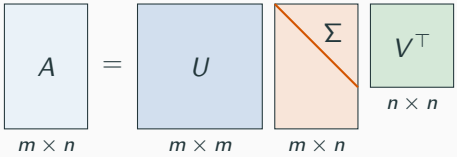
- Any $m \times n$ matrix can be written as $A = U\Sigma V^\top$



Used for dimensionality reduction, noise removal, image compression — and recommender systems.

Singular Value Decomposition (SVD)

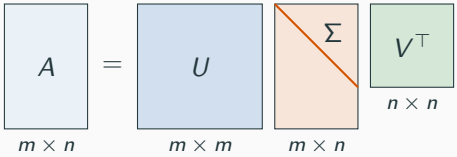
- Any $m \times n$ matrix can be written as $A = U\Sigma V^\top$
- U $(m \times m)$: left singular vectors $\rightarrow$ **users**
- Σ $(m \times n)$: singular values on the diagonal, largest first $\rightarrow$ **importance** of each factor
- V $(n \times n)$: right singular vectors $\rightarrow$ **items**



Used for dimensionality reduction, noise removal, image compression — and recommender systems.

Singular Value Decomposition (SVD)

- Any $m \times n$ matrix can be written as $A = U\Sigma V^\top$
- U ($m \times m$): left singular vectors $\rightarrow$ **users**
- Σ ($m \times n$): singular values on the diagonal, largest first $\rightarrow$ **importance** of each factor
- V ($n \times n$): right singular vectors $\rightarrow$ **items**
- **How to compute:** columns of V = eigenvectors of $A^\top A$; columns of U = eigenvectors of $AA^\top$; singular values $\sigma = \sqrt{\lambda}$



Used for dimensionality reduction, noise removal, image compression — and recommender systems.

SVD by hand (1): compute $A^\top A$ and $AA^\top$

$$A = \begin{bmatrix} 3 & 0 \\ 4 & 5 \end{bmatrix}$$

SVD by hand (1): compute $A^\top A$ and $AA^\top$

$$A = \begin{bmatrix} 3 & 0 \\ 4 & 5 \end{bmatrix}$$

$$A^\top A = \begin{bmatrix} 3 & 4 \\ 0 & 5 \end{bmatrix} \begin{bmatrix} 3 & 0 \\ 4 & 5 \end{bmatrix} = \begin{bmatrix} 9+16 & 0+20 \\ 0+20 & 0+25 \end{bmatrix} = \begin{bmatrix} 25 & 20 \\ 20 & 25 \end{bmatrix}$$

SVD by hand (1): compute $A^\top A$ and $AA^\top$

$$A = \begin{bmatrix} 3 & 0 \\ 4 & 5 \end{bmatrix}$$

$$A^\top A = \begin{bmatrix} 3 & 4 \\ 0 & 5 \end{bmatrix} \begin{bmatrix} 3 & 0 \\ 4 & 5 \end{bmatrix} = \begin{bmatrix} 9+16 & 0+20 \\ 0+20 & 0+25 \end{bmatrix} = \begin{bmatrix} 25 & 20 \\ 20 & 25 \end{bmatrix}$$

$$AA^\top = \begin{bmatrix} 3 & 0 \\ 4 & 5 \end{bmatrix} \begin{bmatrix} 3 & 4 \\ 0 & 5 \end{bmatrix} = \begin{bmatrix} 9+0 & 12+0 \\ 12+0 & 16+25 \end{bmatrix} = \begin{bmatrix} 9 & 12 \\ 12 & 41 \end{bmatrix}$$

SVD by hand (1): compute $A^\top A$ and $AA^\top$

$$A = \begin{bmatrix} 3 & 0 \\ 4 & 5 \end{bmatrix}$$

$$A^\top A = \begin{bmatrix} 3 & 4 \\ 0 & 5 \end{bmatrix} \begin{bmatrix} 3 & 0 \\ 4 & 5 \end{bmatrix} = \begin{bmatrix} 9+16 & 0+20 \\ 0+20 & 0+25 \end{bmatrix} = \begin{bmatrix} 25 & 20 \\ 20 & 25 \end{bmatrix}$$

$$AA^\top = \begin{bmatrix} 3 & 0 \\ 4 & 5 \end{bmatrix} \begin{bmatrix} 3 & 4 \\ 0 & 5 \end{bmatrix} = \begin{bmatrix} 9+0 & 12+0 \\ 12+0 & 16+25 \end{bmatrix} = \begin{bmatrix} 9 & 12 \\ 12 & 41 \end{bmatrix}$$

Both are symmetric, so their eigenvectors are perpendicular — that is why U and V come out orthogonal.

SVD by hand (2): eigenvalues $\rightarrow$ singular values

Solve $\det(A^\top A - \lambda I) = 0$:

$$\begin{vmatrix} 25-\lambda & 20 \\ 20 & 25-\lambda \end{vmatrix} = (25-\lambda)^2 - 400 = 0$$

SVD by hand (2): eigenvalues $\rightarrow$ singular values

Solve $\det(A^\top A - \lambda I) = 0$:

$$\begin{vmatrix} 25-\lambda & 20 \\ 20 & 25-\lambda \end{vmatrix} = (25-\lambda)^2 - 400 = 0$$

$$25 - \lambda = \pm 20 \;\Rightarrow\; \lambda_1 = 45, \quad \lambda_2 = 5$$

SVD by hand (2): eigenvalues $\rightarrow$ singular values

Solve $\det(A^\top A - \lambda I) = 0$:

$$\begin{vmatrix} 25-\lambda & 20 \\ 20 & 25-\lambda \end{vmatrix} = (25-\lambda)^2 - 400 = 0$$

$$25 - \lambda = \pm 20 \;\Rightarrow\; \lambda_1 = 45, \quad \lambda_2 = 5$$

$AA^\top$ has the **same** eigenvalues (check: trace $9 + 41 = 50 = 45 + 5$, determinant $369 - 144 = 225 = 45 \times 5$).

SVD by hand (2): eigenvalues $\to$ singular values

Solve $\det(A^\top A - \lambda I) = 0$:

$$\begin{vmatrix} 25-\lambda & 20 \\ 20 & 25-\lambda \end{vmatrix} = (25-\lambda)^2 - 400 = 0$$

$$25 - \lambda = \pm 20 \;\Rightarrow\; \lambda_1 = 45, \quad \lambda_2 = 5$$

$AA^\top$ has the **same** eigenvalues (check: trace $9 + 41 = 50 = 45 + 5$, determinant $369 - 144 = 225 = 45 \times 5$).

$$\sigma_1 = \sqrt{45} = 3\sqrt{5} \approx 6.71, \qquad \sigma_2 = \sqrt{5} \approx 2.24$$

σ_1 is three times σ_2: the first factor carries most of the structure.

SVD by hand (3): eigenvectors $\rightarrow V$ and U

V: eigenvectors of $A^\top A$

$$\lambda_1 = 45: \quad \begin{bmatrix} -20 & 20 \\ 20 & -20 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = 0 \Rightarrow x = y$$

$$\mathbf{v}_1 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

SVD by hand (3): eigenvectors $\rightarrow V$ and U

V: eigenvectors of $A^\top A$

$$\lambda_1 = 45: \quad \begin{bmatrix} -20 & 20 \\ 20 & -20 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = 0 \Rightarrow x = y$$

$$\mathbf{v}_1 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

$$\lambda_2 = 5: \quad \begin{bmatrix} 20 & 20 \\ 20 & 20 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = 0 \Rightarrow x = -y$$

$$\mathbf{v}_2 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

SVD by hand (3): eigenvectors $\rightarrow V$ and U

V: eigenvectors of $A^\top A$

$$\lambda_1 = 45: \quad \begin{bmatrix} -20 & 20 \\ 20 & -20 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = 0 \Rightarrow x = y$$

$$\mathbf{v}_1 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

$$\lambda_2 = 5: \quad \begin{bmatrix} 20 & 20 \\ 20 & 20 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = 0 \Rightarrow x = -y$$

$$\mathbf{v}_2 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

U: eigenvectors of $AA^\top$

$$\lambda_1 = 45: \quad \begin{bmatrix} -36 & 12 \\ 12 & -4 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = 0 \Rightarrow y = 3x$$

$$\mathbf{u}_1 = \frac{1}{\sqrt{10}} \begin{bmatrix} 1 \\ 3 \end{bmatrix}$$

SVD by hand (3): eigenvectors $\rightarrow V$ and U

V: eigenvectors of $A^\top A$

$$\lambda_1 = 45: \quad \begin{bmatrix} -20 & 20 \\ 20 & -20 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = 0 \Rightarrow x = y$$

$$\mathbf{v}_1 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

$$\lambda_2 = 5: \quad \begin{bmatrix} 20 & 20 \\ 20 & 20 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = 0 \Rightarrow x = -y$$

$$\mathbf{v}_2 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

U: eigenvectors of $AA^\top$

$$\lambda_1 = 45: \quad \begin{bmatrix} -36 & 12 \\ 12 & -4 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = 0 \Rightarrow y = 3x$$

$$\mathbf{u}_1 = \frac{1}{\sqrt{10}} \begin{bmatrix} 1 \\ 3 \end{bmatrix}$$

$$\lambda_2 = 5: \quad \begin{bmatrix} 4 & 12 \\ 12 & 36 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = 0 \Rightarrow x = -3y$$

$$\mathbf{u}_2 = \frac{1}{\sqrt{10}} \begin{bmatrix} 3 \\ -1 \end{bmatrix}$$

SVD by hand (3): eigenvectors $\to V$ and U

V: eigenvectors of $A^\top A$

$$\lambda_1 = 45: \quad \begin{bmatrix} -20 & 20 \\ 20 & -20 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = 0 \Rightarrow x = y$$

$$\mathbf{v}_1 = \tfrac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

$$\lambda_2 = 5: \quad \begin{bmatrix} 20 & 20 \\ 20 & 20 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = 0 \Rightarrow x = -y$$

$$\mathbf{v}_2 = \tfrac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

U: eigenvectors of $AA^\top$

$$\lambda_1 = 45: \quad \begin{bmatrix} -36 & 12 \\ 12 & -4 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = 0 \Rightarrow y = 3x$$

$$\mathbf{u}_1 = \tfrac{1}{\sqrt{10}} \begin{bmatrix} 1 \\ 3 \end{bmatrix}$$

$$\lambda_2 = 5: \quad \begin{bmatrix} 4 & 12 \\ 12 & 36 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = 0 \Rightarrow x = -3y$$

$$\mathbf{u}_2 = \tfrac{1}{\sqrt{10}} \begin{bmatrix} 3 \\ -1 \end{bmatrix}$$

Normalise each vector to length 1. Signs must match: check $A\mathbf{v}_s = \sigma_s \mathbf{u}_s$ (e.g. $A\mathbf{v}_2 = \frac{1}{\sqrt{2}}(3, -1) = \sqrt{5}\,\mathbf{u}_2$).

SVD by hand (4): put it together and check

$$A = \underbrace{\frac{1}{\sqrt{10}}\begin{bmatrix} 1 & 3 \\ 3 & -1 \end{bmatrix}}_{U} \underbrace{\begin{bmatrix} 3\sqrt{5} & 0 \\ 0 & \sqrt{5} \end{bmatrix}}_{\Sigma} \underbrace{\frac{1}{\sqrt{2}}\begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}}_{V^\top}$$

SVD by hand (4): put it together and check

$$A = \underbrace{\frac{1}{\sqrt{10}}\begin{bmatrix} 1 & 3 \\ 3 & -1 \end{bmatrix}}_{U} \underbrace{\begin{bmatrix} 3\sqrt{5} & 0 \\ 0 & \sqrt{5} \end{bmatrix}}_{\Sigma} \underbrace{\frac{1}{\sqrt{2}}\begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}}_{V^\top}$$

Check. First $U\Sigma$:

$$U\Sigma = \frac{1}{\sqrt{10}}\begin{bmatrix} 3\sqrt{5} & 3\sqrt{5} \\ 9\sqrt{5} & -\sqrt{5} \end{bmatrix}$$

SVD by hand (4): put it together and check

$$A = \underbrace{\frac{1}{\sqrt{10}}\begin{bmatrix}1 & 3\\ 3 & -1\end{bmatrix}}_{U}\ \underbrace{\begin{bmatrix}3\sqrt{5} & 0\\ 0 & \sqrt{5}\end{bmatrix}}_{\Sigma}\ \underbrace{\frac{1}{\sqrt{2}}\begin{bmatrix}1 & 1\\ 1 & -1\end{bmatrix}}_{V^\top}$$

Check. First $U\Sigma$:

$$U\Sigma = \frac{1}{\sqrt{10}}\begin{bmatrix}3\sqrt{5} & 3\sqrt{5}\\ 9\sqrt{5} & -\sqrt{5}\end{bmatrix}$$

Then multiply by $V^\top$ (note $\frac{\sqrt{5}}{\sqrt{10}\sqrt{2}} = \frac{1}{2}$):

$$U\Sigma V^\top = \frac{1}{2}\begin{bmatrix}3+3 & 3-3\\ 9-1 & 9+1\end{bmatrix} = \frac{1}{2}\begin{bmatrix}6 & 0\\ 8 & 10\end{bmatrix} = \begin{bmatrix}3 & 0\\ 4 & 5\end{bmatrix} = A \checkmark$$

SVD by hand (4): put it together and check

$$A = \underbrace{\tfrac{1}{\sqrt{10}}\begin{bmatrix}1 & 3\\ 3 & -1\end{bmatrix}}_{U}\ \underbrace{\begin{bmatrix}3\sqrt{5} & 0\\ 0 & \sqrt{5}\end{bmatrix}}_{\Sigma}\ \underbrace{\tfrac{1}{\sqrt{2}}\begin{bmatrix}1 & 1\\ 1 & -1\end{bmatrix}}_{V^\top}$$

Check. First $U\Sigma$:

$$U\Sigma = \tfrac{1}{\sqrt{10}}\begin{bmatrix}3\sqrt{5} & 3\sqrt{5}\\ 9\sqrt{5} & -\sqrt{5}\end{bmatrix}$$

Then multiply by $V^\top$ (note $\frac{\sqrt{5}}{\sqrt{10}\sqrt{2}} = \frac{1}{2}$):

$$U\Sigma V^\top = \tfrac{1}{2}\begin{bmatrix}3+3 & 3-3\\ 9-1 & 9+1\end{bmatrix} = \tfrac{1}{2}\begin{bmatrix}6 & 0\\ 8 & 10\end{bmatrix} = \begin{bmatrix}3 & 0\\ 4 & 5\end{bmatrix} = A\ \checkmark$$

The **full** SVD reproduces every cell exactly — including the 0.

Keep only the strongest factor: rank-1 approximation

Keep σ_1 and drop σ_2 (truncated SVD with $k = 1$):

$$A_1 = \sigma_1 \, \mathbf{u}_1 \mathbf{v}_1^\top = 3\sqrt{5} \cdot \tfrac{1}{\sqrt{10}} \begin{bmatrix} 1 \\ 3 \end{bmatrix} \cdot \tfrac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \end{bmatrix} = \tfrac{3}{2} \begin{bmatrix} 1 & 1 \\ 3 & 3 \end{bmatrix} = \begin{bmatrix} 1.5 & 1.5 \\ 4.5 & 4.5 \end{bmatrix}$$

Keep only the strongest factor: rank-1 approximation

Keep σ_1 and drop σ_2 (truncated SVD with $k = 1$):

$$A_1 = \sigma_1 \mathbf{u}_1 \mathbf{v}_1^\top = 3\sqrt{5} \cdot \tfrac{1}{\sqrt{10}} \begin{bmatrix} 1 \\ 3 \end{bmatrix} \cdot \tfrac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \end{bmatrix} = \tfrac{3}{2} \begin{bmatrix} 1 & 1 \\ 3 & 3 \end{bmatrix} = \begin{bmatrix} 1.5 & 1.5 \\ 4.5 & 4.5 \end{bmatrix}$$

- Compare with $A = \begin{bmatrix} 3 & 0 \\ 4 & 5 \end{bmatrix}$: the 0 becomes 1.5. A low-rank approximation fills in every cell

Keep only the strongest factor: rank-1 approximation

Keep σ_1 and drop σ_2 (truncated SVD with $k = 1$):

$$A_1 = \sigma_1 \mathbf{u}_1 \mathbf{v}_1^\top = 3\sqrt{5} \cdot \tfrac{1}{\sqrt{10}} \begin{bmatrix} 1 \\ 3 \end{bmatrix} \cdot \tfrac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \end{bmatrix} = \tfrac{3}{2} \begin{bmatrix} 1 & 1 \\ 3 & 3 \end{bmatrix} = \begin{bmatrix} 1.5 & 1.5 \\ 4.5 & 4.5 \end{bmatrix}$$

- Compare with $A = \begin{bmatrix} 3 & 0 \\ 4 & 5 \end{bmatrix}$: the 0 becomes 1.5. A low-rank approximation fills in every cell
- The error is exactly the dropped singular value: $\|A - A_1\|_F = \sigma_2 = \sqrt{5} \approx 2.24$

Keep only the strongest factor: rank-1 approximation

Keep σ_1 and drop σ_2 (truncated SVD with $k = 1$):

$$A_1 = \sigma_1 \mathbf{u}_1 \mathbf{v}_1^\top = 3\sqrt{5} \cdot \tfrac{1}{\sqrt{10}} \begin{bmatrix} 1 \\ 3 \end{bmatrix} \cdot \tfrac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \end{bmatrix} = \tfrac{3}{2} \begin{bmatrix} 1 & 1 \\ 3 & 3 \end{bmatrix} = \begin{bmatrix} 1.5 & 1.5 \\ 4.5 & 4.5 \end{bmatrix}$$

- Compare with $A = \begin{bmatrix} 3 & 0 \\ 4 & 5 \end{bmatrix}$: the 0 becomes 1.5. A low-rank approximation fills in every cell
- The error is exactly the dropped singular value: $\|A - A_1\|_F = \sigma_2 = \sqrt{5} \approx 2.24$
- Keeping the top k singular values gives the **best** rank-k approximation (Eckart–Young) — compression that keeps the main pattern

So why not use this SVD for recommendations?

- Classical SVD needs a **complete** table — our rating table is 98% empty

So why not use this SVD for recommendations?

- Classical SVD needs a **complete** table — our rating table is 98% empty
- Fill the gaps with 0? In ratings, 0 means "*not rated*", not "rated zero"

So why not use this SVD for recommendations?

- Classical SVD needs a **complete** table — our rating table is 98% empty
- Fill the gaps with 0? In ratings, 0 means "*not rated*", not "rated zero"
- Full SVD would reproduce those fake zeros exactly; low-rank SVD would bend the real ratings to fit them

So why not use this SVD for recommendations?

- Classical SVD needs a **complete** table — our rating table is 98% empty
- Fill the gaps with 0? In ratings, 0 means "*not rated*", not "rated zero"
- Full SVD would reproduce those fake zeros exactly; low-rank SVD would bend the real ratings to fit them
- And it is expensive for huge matrices

So why not use this SVD for recommendations?

- Classical SVD needs a **complete** table — our rating table is 98% empty
- Fill the gaps with 0? In ratings, 0 means "*not rated*", not "rated zero"
- Full SVD would reproduce those fake zeros exactly; low-rank SVD would bend the real ratings to fit them
- And it is expensive for huge matrices

The fix (Simon Funk, Netflix Prize 2006)
Learn the user and movie vectors from the **known ratings only**. Never fill the gaps.

Recommender “SVD”: what it predicts

predicted rating = average + user bias + movie bias + taste match

$$\hat{r}_{ui} = \mu + b_u + b_i + \mathbf{p}_u^\top \mathbf{q}_i$$

Part	Meaning	Example

Recommender “SVD”: what it predicts

predicted rating = average + user bias + movie bias + taste match

$$\hat{r}_{ui} = \mu + b_u + b_i + \mathbf{p}_u^\top \mathbf{q}_i$$

Part	Meaning	Example
μ average	all ratings in the data	3.5

Recommender “SVD”: what it predicts

predicted rating = average + user bias + movie bias + taste match

$$\hat{r}_{ui} = \mu + b_u + b_i + \mathbf{p}_u^\top \mathbf{q}_i$$

Part	Meaning	Example
μ average	all ratings in the data	3.5
b_u user bias	this user is a generous rater	+0.5

Recommender “SVD”: what it predicts

predicted rating = average + user bias + movie bias + taste match

$$\hat{r}_{ui} = \mu + b_u + b_i + \mathbf{p}_u^\top \mathbf{q}_i$$

Part	Meaning	Example
μ average	all ratings in the data	3.5
b_u user bias	this user is a generous rater	+0.5
b_i movie bias	this movie is liked by most people	+0.3

Recommender “SVD”: what it predicts

predicted rating = average + user bias + movie bias + taste match

$$\hat{r}_{ui} = \mu + b_u + b_i + \mathbf{p}_u^\top \mathbf{q}_i$$

Part	Meaning	Example
μ average	all ratings in the data	3.5
b_u user bias	this user is a generous rater	+0.5
b_i movie bias	this movie is liked by most people	+0.3
$\mathbf{p}_u^\top \mathbf{q}_i$ taste match	how well the user’s taste fits the movie	+0.4

Recommender “SVD”: what it predicts

predicted rating = average + user bias + movie bias + taste match

$$\hat{r}_{ui} = \mu + b_u + b_i + \mathbf{p}_u^\top \mathbf{q}_i$$

Part	Meaning	Example
μ average	all ratings in the data	3.5
b_u user bias	this user is a generous rater	+0.5
b_i movie bias	this movie is liked by most people	+0.3
$\mathbf{p}_u^\top \mathbf{q}_i$ taste match	how well the user’s taste fits the movie	+0.4
Prediction		**4.7**

Recommender "SVD": what it predicts

predicted rating = average + user bias + movie bias + taste match

$$\hat{r}_{ui} = \mu + b_u + b_i + \mathbf{p}_u^\top \mathbf{q}_i$$

Part	Meaning	Example
μ average	all ratings in the data	3.5
b_u user bias	this user is a generous rater	+0.5
b_i movie bias	this movie is liked by most people	+0.3
$\mathbf{p}_u^\top \mathbf{q}_i$ taste match	how well the user's taste fits the movie	+0.4
Prediction		**4.7**

Training goal: make these predictions close to the **known** ratings, while keeping all numbers small (regularisation λ) so the model does not memorise.

How it learns: stochastic gradient descent

For every known rating, again and again:

How it learns: stochastic gradient descent

For every known rating, again and again:

1. **Predict** the rating with the current numbers

How it learns: stochastic gradient descent

For every known rating, again and again:

1. **Predict** the rating with the current numbers
2. **Measure the error:** $e = \text{true} - \text{predicted}$

Example
True rating 4, predicted 3.2
error $= +0.8 \Rightarrow$ we were too low

How it learns: stochastic gradient descent

For every known rating, again and again:

1. **Predict** the rating with the current numbers
2. **Measure the error:** $e = \text{true} - \text{predicted}$
3. **Nudge** the numbers a little to shrink the error
   - user bias and movie bias move towards the error
   - user vector and movie vector move *towards* each other if we predicted too low, *apart* if too high

Example
True rating 4, predicted 3.2
error $= +0.8 \Rightarrow$ we were too low
$\Rightarrow$ raise b_u and b_i a little,
pull $\mathbf{p}_u$ and $\mathbf{q}_i$ closer

How it learns: stochastic gradient descent

For every known rating, again and again:

1. **Predict** the rating with the current numbers
2. **Measure the error:** $e = \text{true} - \text{predicted}$
3. **Nudge** the numbers a little to shrink the error
   - user bias and movie bias move towards the error
   - user vector and movie vector move *towards* each other if we predicted too low, *apart* if too high

One pass over all ratings = one **epoch**; repeat about 20 times.

Example
True rating 4, predicted 3.2
error $= +0.8 \Rightarrow$ we were too low
$\Rightarrow$ raise b_u and b_i a little,
pull $\mathbf{p}_u$ and $\mathbf{q}_i$ closer

How it learns: stochastic gradient descent

For every known rating, again and again:

1. **Predict** the rating with the current numbers
2. **Measure the error:** $e = \text{true} - \text{predicted}$
3. **Nudge** the numbers a little to shrink the error
   - user bias and movie bias move towards the error
   - user vector and movie vector move *towards* each other if we predicted too low, *apart* if too high

One pass over all ratings = one **epoch**; repeat about 20 times.

Example
True rating 4, predicted 3.2
error $= +0.8 \Rightarrow$ we were too low

$\Rightarrow$ raise b_u and b_i a little,
pull $\mathbf{p}_u$ and $\mathbf{q}_i$ closer

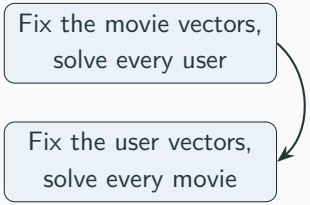
Two settings: **learning rate** (how big a nudge) and **regularisation** (keep numbers small). `surprise.SVD` does exactly this.

Another way to learn: alternating least squares (ALS)

Fix the movie vectors, solve every user

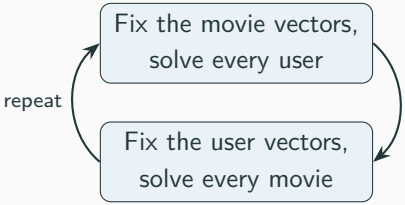
- With the movies fixed, each user's best vector is an ordinary **least-squares** problem — solved exactly, in one step

Another way to learn: alternating least squares (ALS)



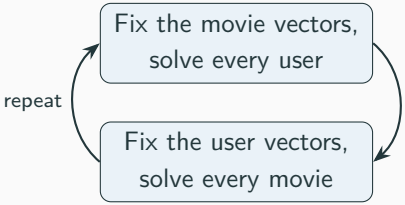
- With the movies fixed, each user's best vector is an ordinary **least-squares** problem — solved exactly, in one step

Another way to learn: alternating least squares (ALS)



- With the movies fixed, each user's best vector is an ordinary **least-squares** problem — solved exactly, in one step
- Alternate until the error stops falling

Another way to learn: alternating least squares (ALS)



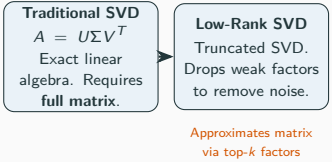
- With the movies fixed, each user's best vector is an ordinary **least-squares** problem — solved exactly, in one step
- Alternate until the error stops falling

	SGD	**ALS**
Updates	one rating at a time	all users (or all movies) at once
Runs in parallel	hard	easy
Used in	Surprise, small data	Spark, very large data

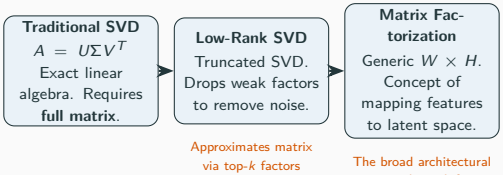
The SVD Landscape: From Pure Math to Recommendation Engines

Traditional SVD
$A = U\Sigma V^T$
Exact linear algebra. Requires **full matrix**.

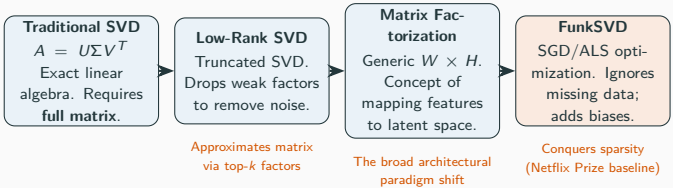
The SVD Landscape: From Pure Math to Recommendation Engines



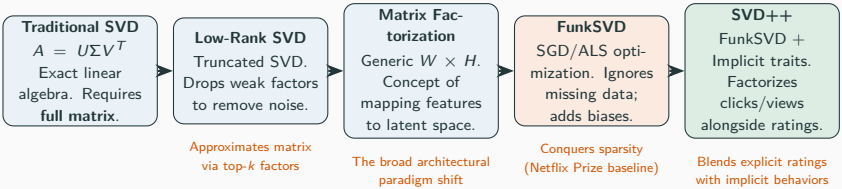
The SVD Landscape: From Pure Math to Recommendation Engines



The SVD Landscape: From Pure Math to Recommendation Engines

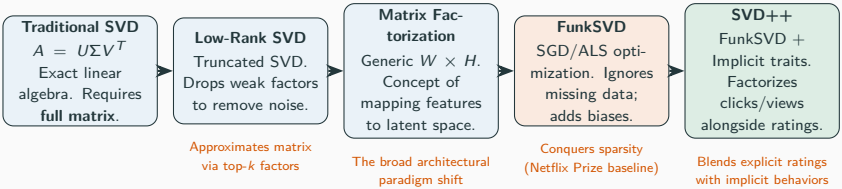


The SVD Landscape: From Pure Math to Recommendation Engines



$$\hat{r}_{ui} = \mu + b_u + b_i + \mathbf{q}_i^\top \left(\mathbf{p}_u + |I_u|^{-\frac{1}{2}} \sum_{j \in I_u} \mathbf{y}_j \right)$$

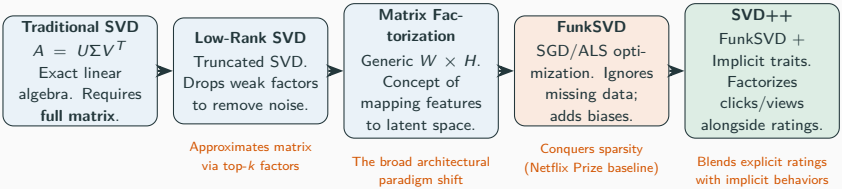
The SVD Landscape: From Pure Math to Recommendation Engines



$$\hat{r}_{ui} = \mu + b_u + b_i + \mathbf{q}_i^\top \left(\mathbf{p}_u + |I_u|^{-\frac{1}{2}} \sum_{j \in I_u} \mathbf{y}_j \right)$$

- **Linear Algebra vs. ML:** Classical SVD / Low-Rank SVD demand calculation over a full matrix (imputation breaks sparsity). MF algorithms learn *around* missing data.
- **FunkSVD Paradigm:** Shipped the mathematical concept into real-world applications by training parameters strictly on observed ratings via SGD, adjusting for user/item baseline biases.

The SVD Landscape: From Pure Math to Recommendation Engines



$$\hat{r}_{ui} = \mu + b_u + b_i + \mathbf{q}_i^\top \left(\mathbf{p}_u + |I_u|^{-\frac{1}{2}} \sum_{j \in I_u} \mathbf{y}_j \right)$$

- **Linear Algebra vs. ML:** Classical SVD / Low-Rank SVD demand calculation over a full matrix (imputation breaks sparsity). MF algorithms learn *around* missing data.
- **FunkSVD Paradigm:** Shipped the mathematical concept into real-world applications by training parameters strictly on observed ratings via SGD, adjusting for user/item baseline biases.
- **SVD++ Evolution:** Adjusts the user profile by adding a sum of implicit factor vectors ($\mathbf{y}_j$) for all items in the user's history (I_u), capturing highly valuable hidden context even when explicit ratings are zero.

Types of matrix factorization

- **Regularised MF (RMF)** — squared loss + Frobenius penalty (what we just did)

$$J = \sum_{(i,j)\in\Omega} (y_{ij} - \mathbf{u}_i \mathbf{v}_j^\top)^2 + \lambda(\|U\|_F^2 + \|V\|_F^2)$$

Types of matrix factorization

- **Regularised MF (RMF)** — squared loss + Frobenius penalty (what we just did)

$$J = \sum_{(i,j)\in\Omega} (y_{ij} - \mathbf{u}_i \mathbf{v}_j^\top)^2 + \lambda(\|U\|_F^2 + \|V\|_F^2)$$

- **Non-negative MF (NMF)** — constrain $U, V \geq 0$ instead of regularising
  - factors become additive "parts" $\Rightarrow$ easier to interpret

Types of matrix factorization

- **Regularised MF (RMF)** — squared loss + Frobenius penalty (what we just did)

$$J = \sum_{(i,j)\in\Omega} (y_{ij} - \mathbf{u}_i \mathbf{v}_j^\top)^2 + \lambda(\|U\|_F^2 + \|V\|_F^2)$$

- **Non-negative MF (NMF)** — constrain $U, V \geq 0$ instead of regularising
  - factors become additive "parts" $\Rightarrow$ easier to interpret
- **Maximum Margin MF (MMMF)** — treats ratings as *ordinal*, uses a hinge loss and per-user thresholds

Types of matrix factorization

- **Regularised MF (RMF)** — squared loss + Frobenius penalty (what we just did)

$$J = \sum_{(i,j)\in\Omega} (y_{ij} - \mathbf{u}_i \mathbf{v}_j^\top)^2 + \lambda(\|U\|_F^2 + \|V\|_F^2)$$

- **Non-negative MF (NMF)** — constrain $U, V \geq 0$ instead of regularising
  - factors become additive "parts" $\Rightarrow$ easier to interpret
- **Maximum Margin MF (MMMF)** — treats ratings as *ordinal*, uses a hinge loss and per-user thresholds
- **SVD++, timeSVD++** — add implicit feedback and time (Netflix Prize)

Non-negative MF: multiplicative updates

Find $U \geq 0$ ($m \times k$) and $V \geq 0$ ($n \times k$) that minimise

$$J(U, V) = \tfrac{1}{2} \|R - UV^\top\|_F^2 \quad \text{subject to } U, V \geq 0$$

Non-negative MF: multiplicative updates

Find $U \geq 0$ ($m \times k$) and $V \geq 0$ ($n \times k$) that minimise

$$J(U, V) = \tfrac{1}{2} \|R - UV^\top\|_F^2 \quad \text{subject to } U, V \geq 0$$

- Non-convex in U and V together, but convex in V when U is fixed $\Rightarrow$ alternate, as in ALS

Non-negative MF: multiplicative updates

Find $U \geq 0$ ($m \times k$) and $V \geq 0$ ($n \times k$) that minimise

$$J(U, V) = \tfrac{1}{2} \|R - UV^\top\|_F^2 \quad \text{subject to } U, V \geq 0$$

- Non-convex in U and V together, but convex in V when U is fixed $\Rightarrow$ alternate, as in ALS
- Lagrange multipliers and the Kuhn–Tucker conditions ($\beta_{ij} V_{ij} = 0$) give **multiplicative** updates (Lee & Seung, 2001):

$$V_{ij} \leftarrow V_{ij} \frac{(R^\top U)_{ij}}{(VU^\top U)_{ij}}, \qquad U_{ij} \leftarrow U_{ij} \frac{(RV)_{ij}}{(UV^\top V)_{ij}}$$

Non-negative MF: multiplicative updates

Find $U \geq 0$ ($m \times k$) and $V \geq 0$ ($n \times k$) that minimise

$$J(U, V) = \tfrac{1}{2} \|R - UV^\top\|_F^2 \quad \text{subject to } U, V \geq 0$$

- Non-convex in U and V together, but convex in V when U is fixed $\Rightarrow$ alternate, as in ALS
- Lagrange multipliers and the Kuhn–Tucker conditions ($\beta_{ij} V_{ij} = 0$) give **multiplicative** updates (Lee & Seung, 2001):

$$V_{ij} \leftarrow V_{ij} \frac{(R^\top U)_{ij}}{(VU^\top U)_{ij}}, \qquad U_{ij} \leftarrow U_{ij} \frac{(RV)_{ij}}{(UV^\top V)_{ij}}$$

- Start from positive values $\Rightarrow$ every update stays non-negative; no step size to tune

Non-negative MF: multiplicative updates

Find $U \geq 0$ ($m \times k$) and $V \geq 0$ ($n \times k$) that minimise

$$J(U, V) = \tfrac{1}{2} \|R - UV^\top\|_F^2 \quad \text{subject to } U, V \geq 0$$

- Non-convex in U and V together, but convex in V when U is fixed $\Rightarrow$ alternate, as in ALS
- Lagrange multipliers and the Kuhn–Tucker conditions ($\beta_{ij} V_{ij} = 0$) give **multiplicative** updates (Lee & Seung, 2001):

$$V_{ij} \leftarrow V_{ij} \frac{(R^\top U)_{ij}}{(VU^\top U)_{ij}}, \qquad U_{ij} \leftarrow U_{ij} \frac{(RV)_{ij}}{(UV^\top V)_{ij}}$$

- Start from positive values $\Rightarrow$ every update stays non-negative; no step size to tune
- For ratings, use only observed cells: replace R by $W \circ R$ and $UV^\top$ by $W \circ (UV^\top)$, with mask W

SVD++ — add *which* items a user rated (implicit signal):

$$\hat{r}_{ui} = \mu + b_u + b_i + \mathbf{q}_i^\top \left(\mathbf{p}_u + |N(u)|^{-\frac{1}{2}} \sum_{j \in N(u)} \mathbf{y}_j \right)$$

SVD++ — add *which* items a user rated (implicit signal):

$$\hat{r}_{ui} = \mu + b_u + b_i + \mathbf{q}_i^\top \left(\mathbf{p}_u + |N(u)|^{-\frac{1}{2}} \sum_{j \in N(u)} \mathbf{y}_j \right)$$

$N(u)$: items u interacted with (rated or not).

SVD++ — add *which* items a user rated (implicit signal):

$$\hat{r}_{ui} = \mu + b_u + b_i + \mathbf{q}_i^\top \left(\mathbf{p}_u + |N(u)|^{-\frac{1}{2}} \sum_{j \in N(u)} \mathbf{y}_j \right)$$

$N(u)$: items u interacted with (rated or not).

timeSVD++ — let parameters drift over time:

$$\hat{r}_{ui}(t) = \mu + b_u(t) + b_i(t) + \mathbf{q}_i^\top \left(\mathbf{p}_u(t) + \dots \right)$$

e.g. $b_i(t) = b_i + b_{i,\mathrm{Bin}(t)}$: a movie's appeal changes; a user's rating scale drifts.

SVD++ — add *which* items a user rated (implicit signal):

$$\hat{r}_{ui} = \mu + b_u + b_i + \mathbf{q}_i^\top \Big(\mathbf{p}_u + |N(u)|^{-\frac{1}{2}} \sum_{j \in N(u)} \mathbf{y}_j \Big)$$

$N(u)$: items u interacted with (rated or not).

timeSVD++ — let parameters drift over time:

$$\hat{r}_{ui}(t) = \mu + b_u(t) + b_i(t) + \mathbf{q}_i^\top \left(\mathbf{p}_u(t) + \dots \right)$$

e.g. $b_i(t) = b_i + b_{i,\mathrm{Bin}(t)}$: a movie's appeal changes; a user's rating scale drifts.

Implicit-feedback MF (Hu, Koren & Volinsky 2008)
Preference $p_{ui} = \mathbb{1}[r_{ui} > 0]$, confidence $c_{ui} = 1 + \alpha r_{ui}$: $\min \sum_{u,i} c_{ui}(p_{ui} - \mathbf{x}_u^\top \mathbf{y}_i)^2 + \lambda(\cdots)$ over **all** cells, solved by ALS.

Maximum Margin MF with two rating levels

Ratings $y_{ij} \in \{+1, -1\}$:

$$J(U, V) = \sum_{(i,j)\in\Omega} h\left(y_{ij}\, \mathbf{u}_i \mathbf{v}_j^\top\right) + \frac{\lambda}{2}\left(\|U\|_F^2 + \|V\|_F^2\right)$$

$$h(z) = \max(0,\, 1 - z)$$

Maximum Margin MF with two rating levels

Ratings $y_{ij} \in \{+1, -1\}$:

$$J(U, V) = \sum_{(i,j)\in\Omega} h\big(y_{ij}\, \mathbf{u}_i \mathbf{v}_j^\top\big) + \tfrac{\lambda}{2}\big(\|U\|_F^2 + \|V\|_F^2\big)$$

$$h(z) = \max(0,\, 1 - z)$$

Actual	Predicted	Hinge loss
$+1$	0.97	0.03
$+1$	1.10	0.00
$+1$	-0.25	1.25
-1	-0.85	0.15
-1	-1.05	0.00
-1	0.60	1.60

Maximum Margin MF with two rating levels

Ratings $y_{ij} \in \{+1, -1\}$:

$$J(U, V) = \sum_{(i,j)\in\Omega} h\big(y_{ij}\, \mathbf{u}_i \mathbf{v}_j^\top\big) + \tfrac{\lambda}{2}\big(\|U\|_F^2 + \|V\|_F^2\big)$$

$$h(z) = \max(0,\, 1 - z)$$

Actual	Predicted	Hinge loss
$+1$	0.97	0.03
$+1$	1.10	0.00
$+1$	-0.25	1.25
-1	-0.85	0.15
-1	-1.05	0.00
-1	0.60	1.60

Right side with margin ≥ 1: no loss. Too close, or wrong side: the loss grows linearly.

Maximum Margin MF with two rating levels

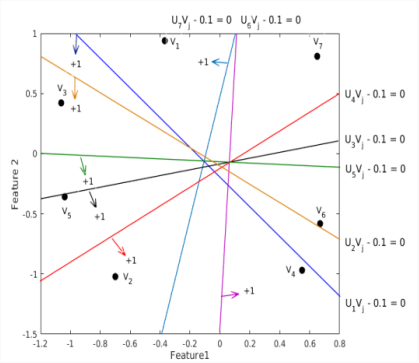
Ratings $y_{ij} \in \{+1, -1\}$:

$$J(U, V) = \sum_{(i,j)\in\Omega} h\left(y_{ij}\, \mathbf{u}_i \mathbf{v}_j^\top\right) + \frac{\lambda}{2}\left(\|U\|_F^2 + \|V\|_F^2\right)$$

$$h(z) = \max(0,\, 1 - z)$$

Actual	Predicted	Hinge loss
$+1$	0.97	0.03
$+1$	1.10	0.00
$+1$	-0.25	1.25
-1	-0.85	0.15
-1	-1.05	0.00
-1	0.60	1.60

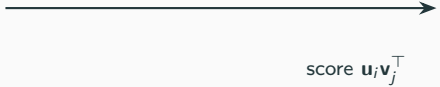
Right side with margin ≥ 1: no loss. Too close, or wrong side: the loss grows linearly.



Items $\mathbf{v}_j$ are points; each user $\mathbf{u}_i$ is a hyperplane separating that user's $+1$ items from the -1 items (Srebro, Rennie & Jaakkola, NIPS 2004).

Maximum Margin MF: ratings are ordinal

- A rating of 4 is "between 3 and 5" — not a real number to regress



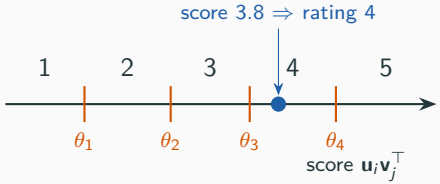
Maximum Margin MF: ratings are ordinal

- A rating of 4 is "between 3 and 5" — not a real number to regress
- Learn per-user thresholds $\theta_{i1} < \cdots < \theta_{i,R-1}$



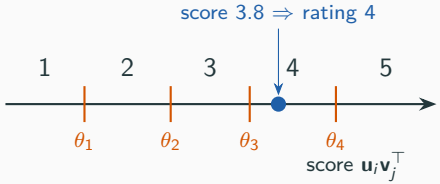
Maximum Margin MF: ratings are ordinal

- A rating of 4 is "between 3 and 5" — not a real number to regress
- Learn per-user thresholds $\theta_{i1} < \cdots < \theta_{i,R-1}$
- Real score $\mathbf{u}_i \mathbf{v}_j^\top$ is mapped to a discrete rating by which interval it falls into



Maximum Margin MF: ratings are ordinal

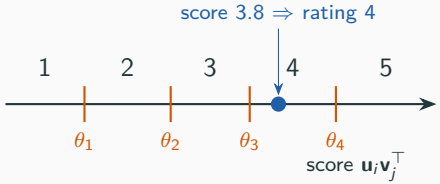
- A rating of 4 is "between 3 and 5" — not a real number to regress
- Learn per-user thresholds $\theta_{i1} < \cdots < \theta_{i,R-1}$
- Real score $\mathbf{u}_i \mathbf{v}_j^\top$ is mapped to a discrete rating by which interval it falls into
- Smoothed hinge loss penalises a score on the *wrong side* of each threshold



Maximum Margin MF: ratings are ordinal

- A rating of 4 is "between 3 and 5" — not a real number to regress
- Learn per-user thresholds $\theta_{i1} < \cdots < \theta_{i,R-1}$
- Real score $\mathbf{u}_i \mathbf{v}_j^\top$ is mapped to a discrete rating by which interval it falls into
- Smoothed hinge loss penalises a score on the *wrong side* of each threshold

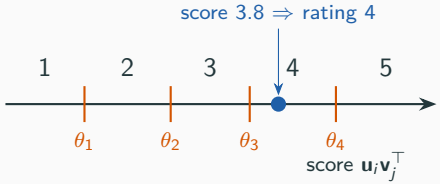
$$J = \sum_{(i,j)\in\Omega} \sum_{a=1}^{R-1} h\Big(T_{ij}^a(\theta_{ia} - \mathbf{u}_i \mathbf{v}_j^\top)\Big) + \tfrac{\lambda}{2}(\|U\|_F^2 + \|V\|_F^2)$$



Maximum Margin MF: ratings are ordinal

- A rating of 4 is "between 3 and 5" — not a real number to regress
- Learn per-user thresholds $\theta_{i1} < \cdots < \theta_{i,R-1}$
- Real score $\mathbf{u}_i \mathbf{v}_j^\top$ is mapped to a discrete rating by which interval it falls into
- Smoothed hinge loss penalises a score on the *wrong side* of each threshold

$$J = \sum_{(i,j)\in\Omega} \sum_{a=1}^{R-1} h\Big(T_{ij}^a(\theta_{ia} - \mathbf{u}_i \mathbf{v}_j^\top)\Big) + \tfrac{\lambda}{2}(\|U\|_F^2 + \|V\|_F^2)$$



Rennie & Srebro, ICML 2005; Devi et al., *Collaborative filtering by PSO-based MMMF*, IEEE SMC 2014.

Neighbourhood vs. latent factor models

	Neighbourhood (k-NN)	**Latent factor (MF)**
Works on	Raw matrix / similarity matrix	Learned low-dimensional space

Neighbourhood vs. latent factor models

	Neighbourhood (k-NN)	**Latent factor (MF)**
Works on	Raw matrix / similarity matrix	Learned low-dimensional space
Captures	Local, pairwise overlap	Global structure across all ratings

Neighbourhood vs. latent factor models

	Neighbourhood (k-NN)	**Latent factor (MF)**
Works on	Raw matrix / similarity matrix	Learned low-dimensional space
Captures	Local, pairwise overlap	Global structure across all ratings
Sparsity	Struggles	Robust (pooled evidence)

Neighbourhood vs. latent factor models

	Neighbourhood (k-NN)	**Latent factor (MF)**
Works on	Raw matrix / similarity matrix	Learned low-dimensional space
Captures	Local, pairwise overlap	Global structure across all ratings
Sparsity	Struggles	Robust (pooled evidence)
Explainability	"Because you watched X"	Factors are unlabeled

Neighbourhood vs. latent factor models

	Neighbourhood (k-NN)	**Latent factor (MF)**
Works on	Raw matrix / similarity matrix	Learned low-dimensional space
Captures	Local, pairwise overlap	Global structure across all ratings
Sparsity	Struggles	Robust (pooled evidence)
Explainability	"Because you watched X"	Factors are unlabeled
New ratings	Instant (just recompute neighbours)	Needs retraining / fold-in

Neighbourhood vs. latent factor models

	Neighbourhood (k-NN)	**Latent factor (MF)**
Works on	Raw matrix / similarity matrix	Learned low-dimensional space
Captures	Local, pairwise overlap	Global structure across all ratings
Sparsity	Struggles	Robust (pooled evidence)
Explainability	"Because you watched X"	Factors are unlabeled
New ratings	Instant (just recompute neighbours)	Needs retraining / fold-in
Memory	$O(n^2)$ similarities	$O((m+n)k)$ parameters

Neighbourhood vs. latent factor models

	Neighbourhood (k-NN)	**Latent factor (MF)**
Works on	Raw matrix / similarity matrix	Learned low-dimensional space
Captures	Local, pairwise overlap	Global structure across all ratings
Sparsity	Struggles	Robust (pooled evidence)
Explainability	"Because you watched X"	Factors are unlabeled
New ratings	Instant (just recompute neighbours)	Needs retraining / fold-in
Memory	$O(n^2)$ similarities	$O((m+n)k)$ parameters

The best Netflix Prize models **combined both** (Koren, Bell & Volinsky, 2009).

Why does MF work when 98% is missing?

- CF compares users only through **directly shared** movies

Why does MF work when 98% is missing?

- CF compares users only through **directly shared** movies
- MF lets information flow **transitively**:

  you $\leftrightarrow$ movie A $\leftrightarrow$ other users $\leftrightarrow$ movie B

Why does MF work when 98% is missing?

- CF compares users only through **directly shared** movies
- MF lets information flow **transitively**:

  you $\leftrightarrow$ movie A $\leftrightarrow$ other users $\leftrightarrow$ movie B
- Each movie's vector is learned from *all* its raters — pooled evidence

Why does MF work when 98% is missing?

- CF compares users only through **directly shared** movies
- MF lets information flow **transitively**:

  you $\leftrightarrow$ movie A $\leftrightarrow$ other users $\leftrightarrow$ movie B
- Each movie's vector is learned from *all* its raters — pooled evidence
- Low rank k is a strong assumption that acts as compression and denoising

Probabilistic view: regularisation is a prior

Probabilistic MF

- $r_{ui} \sim \mathcal{N}(\mathbf{p}_u^\top \mathbf{q}_i,\ \sigma^2)$
- $\mathbf{p}_u \sim \mathcal{N}(\mathbf{0}, \sigma_p^2 I)$, $\mathbf{q}_i \sim \mathcal{N}(\mathbf{0}, \sigma_q^2 I)$

Salakhutdinov & Mnih, *Probabilistic Matrix Factorization*, NIPS 2007.

5. What would a real system do?

Open problems in real systems

Situation	What fails	Typical remedy
New user (no ratings)	No neighbours, no $\mathbf{p}_u$	Popularity, onboarding, demographics, context

Open problems in real systems

Situation	What fails	Typical remedy
New user (no ratings)	No neighbours, no $\mathbf{p}_u$	Popularity, onboarding, demographics, context
New item (no ratings)	Never rated by neighbours, no $\mathbf{q}_i$	Content features, hybrid models

Open problems in real systems

Situation	What fails	Typical remedy
New user (no ratings)	No neighbours, no $\mathbf{p}_u$	Popularity, onboarding, demographics, context
New item (no ratings)	Never rated by neighbours, no $\mathbf{q}_i$	Content features, hybrid models
Extreme sparsity	Unreliable similarities, overfitting	Regularisation, side information, cross-domain transfer

Open problems in real systems

Situation	What fails	Typical remedy
New user (no ratings)	No neighbours, no $\mathbf{p}_u$	Popularity, onboarding, demographics, context
New item (no ratings)	Never rated by neighbours, no $\mathbf{q}_i$	Content features, hybrid models
Extreme sparsity	Unreliable similarities, overfitting	Regularisation, side information, cross-domain transfer
Implicit feedback only	No negatives, no rating scale	Weighted ALS, BPR, ranking losses

Open problems in real systems

Situation	What fails	Typical remedy
New user (no ratings)	No neighbours, no $\mathbf{p}_u$	Popularity, onboarding, demographics, context
New item (no ratings)	Never rated by neighbours, no $\mathbf{q}_i$	Content features, hybrid models
Extreme sparsity	Unreliable similarities, overfitting	Regularisation, side information, cross-domain transfer
Implicit feedback only	No negatives, no rating scale	Weighted ALS, BPR, ranking losses
Web scale	$O(m^2)$ similarities, latency	Two-stage retrieval + ranking, ANN search

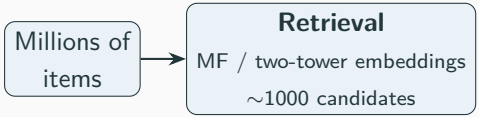
Open problems in real systems

Situation	What fails	Typical remedy
New user (no ratings)	No neighbours, no $\mathbf{p}_u$	Popularity, onboarding, demographics, context
New item (no ratings)	Never rated by neighbours, no $\mathbf{q}_i$	Content features, hybrid models
Extreme sparsity	Unreliable similarities, overfitting	Regularisation, side information, cross-domain transfer
Implicit feedback only	No negatives, no rating scale	Weighted ALS, BPR, ranking losses
Web scale	$O(m^2)$ similarities, latency	Two-stage retrieval + ranking, ANN search
Feedback loops	Popular items get more popular	Exploration, diversity re-ranking, debiasing

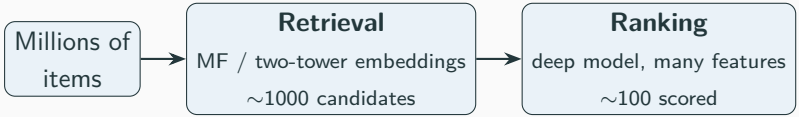
How industry does it (a glimpse)

Millions of items

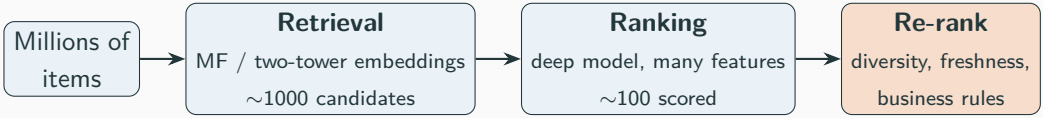
How industry does it (a glimpse)



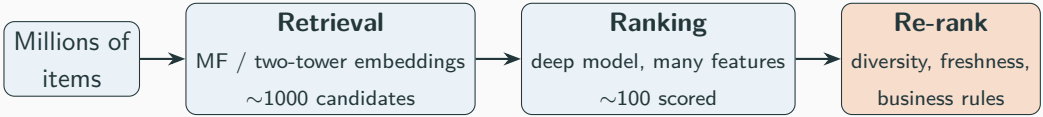
How industry does it (a glimpse)



How industry does it (a glimpse)

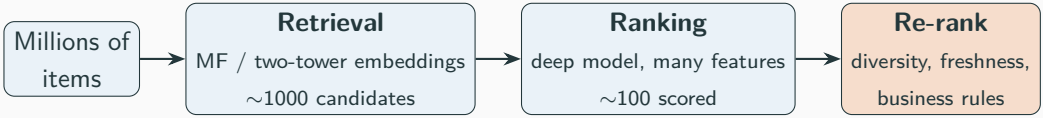


How industry does it (a glimpse)



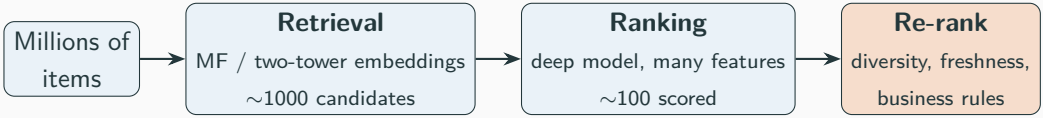
- Today's MF is the ancestor of the **embedding retrieval** stage

How industry does it (a glimpse)



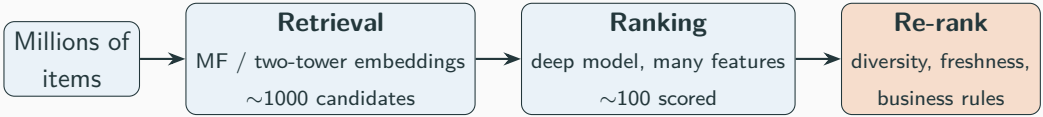
- Today's MF is the ancestor of the **embedding retrieval** stage
- Beyond: sequential models (what you watched *last*), graph neural networks, LLM-based recommenders

How industry does it (a glimpse)



- Today's MF is the ancestor of the **embedding retrieval** stage
- Beyond: sequential models (what you watched *last*), graph neural networks, LLM-based recommenders
- Evaluation moves from RMSE to **A/B tests** on engagement and retention

How industry does it (a glimpse)



- Today's MF is the ancestor of the **embedding retrieval** stage
- Beyond: sequential models (what you watched *last*), graph neural networks, LLM-based recommenders
- Evaluation moves from RMSE to **A/B tests** on engagement and retention
- Concerns: filter bubbles, fairness to niche creators, popularity feedback loops

Why cross-domain recommendation?

Limits of a single domain

- **Data sparsity**: users rate few of the items they watch, buy or read

Why cross-domain recommendation?

Limits of a single domain

- **Data sparsity**: users rate few of the items they watch, buy or read
- **Cold start**: a new user has no ratings — fall back on popular items, or ask for preferences

Why cross-domain recommendation?

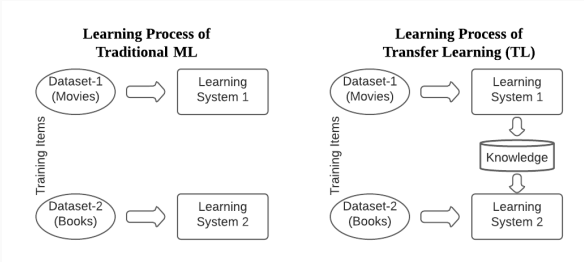
Limits of a single domain

- **Data sparsity**: users rate few of the items they watch, buy or read
- **Cold start**: a new user has no ratings — fall back on popular items, or ask for preferences
- **Cross-domain RS**: transfer what we know in one domain to another, e.g. book ratings $\rightarrow$ movie suggestions

Why cross-domain recommendation?

Limits of a single domain

- **Data sparsity**: users rate few of the items they watch, buy or read
- **Cold start**: a new user has no ratings — fall back on popular items, or ask for preferences
- **Cross-domain RS**: transfer what we know in one domain to another, e.g. book ratings → movie suggestions



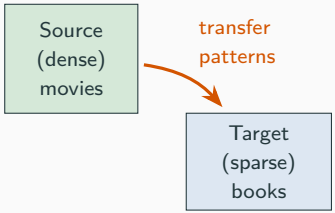
Traditional ML learns each domain on its own; transfer learning reuses knowledge from the source (movies) in the target (books).

From my research: when the target domain is too sparse

- As sparsity grows, even MF loses accuracy

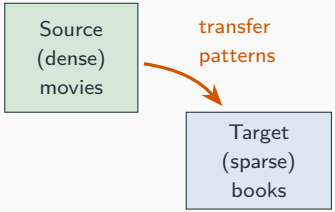
From my research: when the target domain is too sparse

- As sparsity grows, even MF loses accuracy
- **Cross-domain recommendation**: borrow knowledge from a *dense* domain (movies) to help a *sparse* one (books)



From my research: when the target domain is too sparse

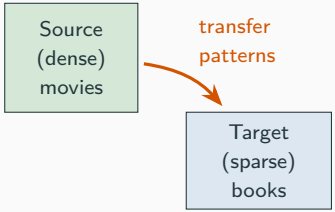
- As sparsity grows, even MF loses accuracy
- **Cross-domain recommendation**: borrow knowledge from a *dense* domain (movies) to help a *sparse* one (books)
- **Transfer learning** questions: *what*, *how* and *when* to transfer?



Pan & Yang, *A Survey on Transfer Learning*, IEEE TKDE 2010; Cremonesi et al., ICDMW 2011.

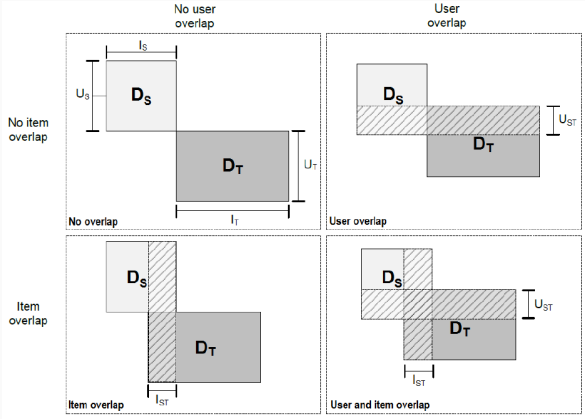
From my research: when the target domain is too sparse

- As sparsity grows, even MF loses accuracy
- **Cross-domain recommendation**: borrow knowledge from a *dense* domain (movies) to help a *sparse* one (books)
- **Transfer learning** questions: *what*, *how* and *when* to transfer?
- Hardest case: **no shared users and no shared items**



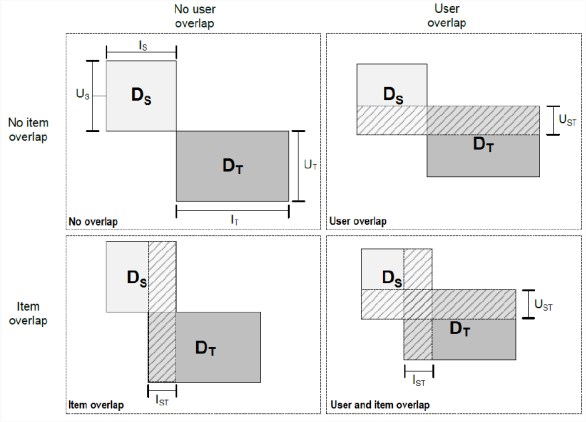
Pan & Yang, *A Survey on Transfer Learning*, IEEE TKDE 2010; Cremonesi et al., ICDMW 2011.

Cross-domain scenarios: what do the domains share?



Cremonesi, Tripodi & Turrin, *Cross-domain recommender systems*, ICDMW 2011.

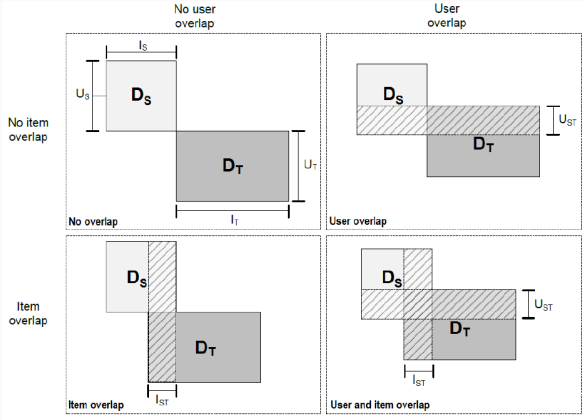
Cross-domain scenarios: what do the domains share?



Cremonesi, Tripodi & Turrin, *Cross-domain recommender systems*, ICDMW 2011.

- **User overlap**: the same people in both domains — link through the users

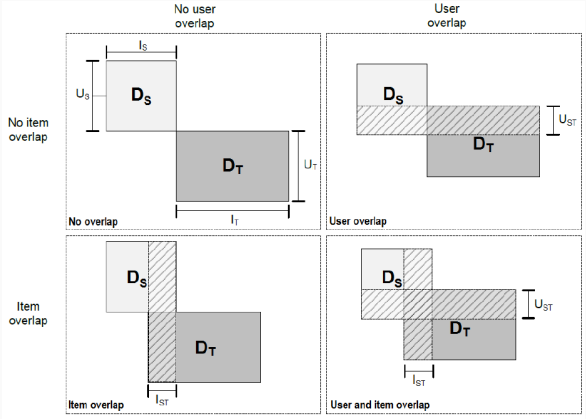
Cross-domain scenarios: what do the domains share?



Cremonesi, Tripodi & Turrin, *Cross-domain recommender systems*, ICDMW 2011.

- **User overlap**: the same people in both domains — link through the users
- **Item overlap**: the same items in both — link through the items

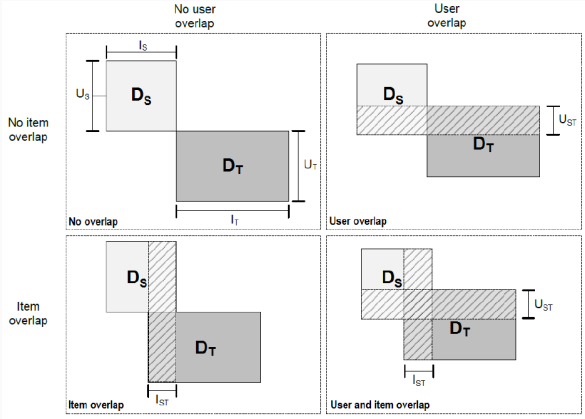
Cross-domain scenarios: what do the domains share?



Cremonesi, Tripodi & Turrin, *Cross-domain recommender systems*, ICDMW 2011.

- **User overlap**: the same people in both domains — link through the users
- **Item overlap**: the same items in both — link through the items
- **User and item overlap**: the easiest case

Cross-domain scenarios: what do the domains share?



Cremonesi, Tripodi & Turrin, *Cross-domain recommender systems*, ICDMW 2011.

- **User overlap**: the same people in both domains — link through the users
- **Item overlap**: the same items in both — link through the items
- **User and item overlap**: the easiest case
- **No overlap**: nothing shared — only *rating patterns* can be transferred $\Rightarrow$ codebook transfer

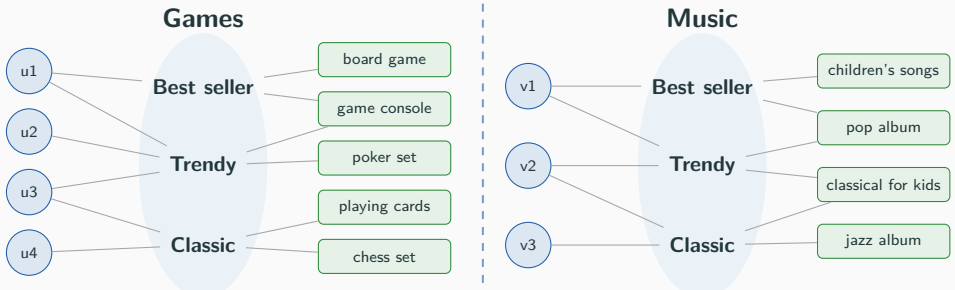
Example: the same rating pattern in two domains



- Users and items fall into a few **groups**; the rating pattern is between groups

Idea from Li, Yang & Xue, *Can Movies and Books Collaborate?*, IJCAI 2009.

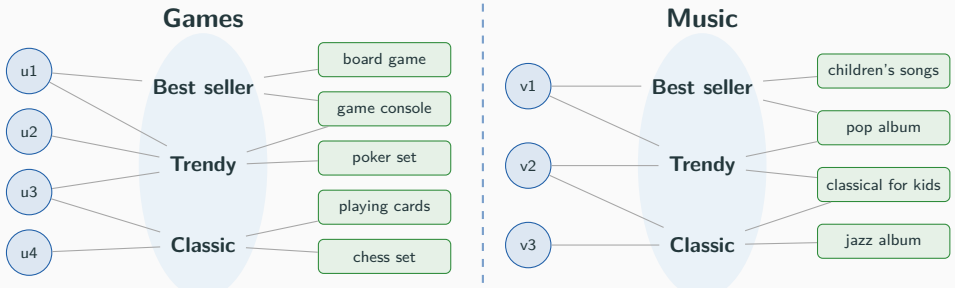
Example: the same rating pattern in two domains



- Users and items fall into a few **groups**; the rating pattern is between groups
- A different domain, **different people, different items** — but the same kinds of groups

Idea from Li, Yang & Xue, *Can Movies and Books Collaborate?*, IJCAI 2009.

Example: the same rating pattern in two domains

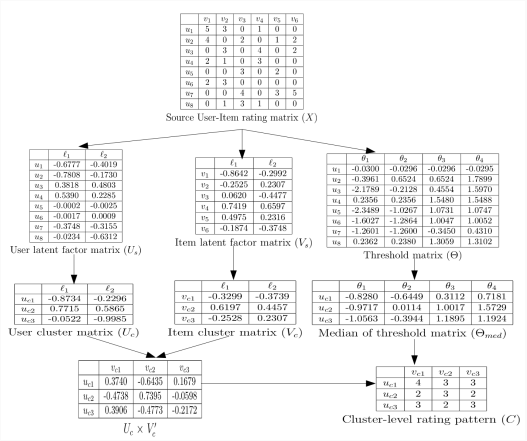


- Users and items fall into a few **groups**; the rating pattern is between groups
- A different domain, **different people, different items** — but the same kinds of groups
- So we can learn the group-to-group rating pattern where data is dense, and **reuse** it where data is sparse: this pattern is the codebook

Idea from Li, Yang & Xue, *Can Movies and Books Collaborate?*, IJCAI 2009.

Codebook transfer: the idea

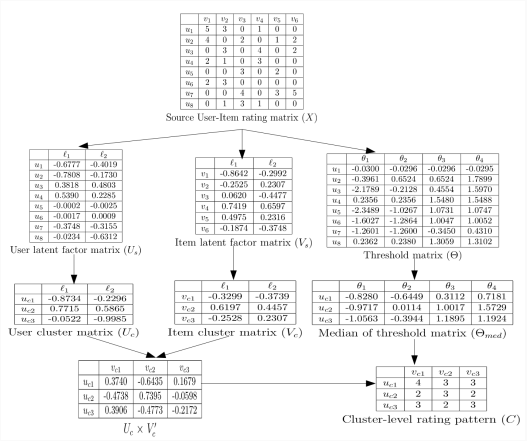
1. Factorise the dense source with **MMMF** $\Rightarrow U_s, V_s, \Theta$



Veeramachaneni, Pujari, Padmanabhan & Kumar, *Applied Soft Computing* 85, 2019.

Codebook transfer: the idea

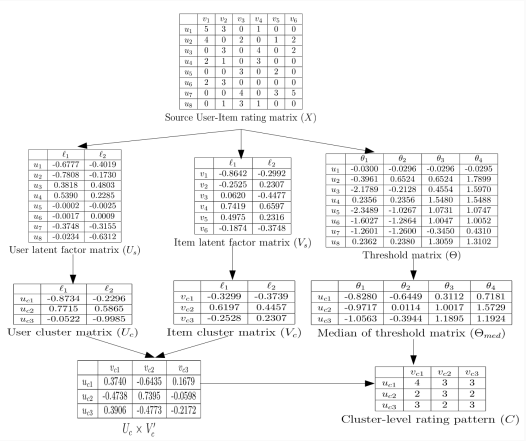
1. Factorise the dense source with **MMMF** $\Rightarrow U_s, V_s, \Theta$
2. **Cluster** user and item factors



Veeramachaneni, Pujari, Padmanabhan & Kumar, *Applied Soft Computing* 85, 2019.

Codebook transfer: the idea

1. Factorise the dense source with **MMMF** $\Rightarrow U_s, V_s, \Theta$
2. **Cluster** user and item factors
3. Cluster-level rating pattern = the **codebook** C $(k_1 \times k_2)$

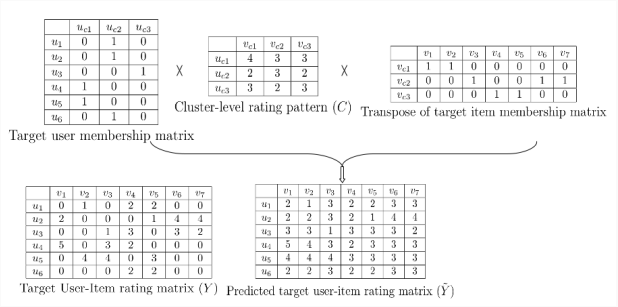


Veeramachaneni, Pujari, Padmanabhan & Kumar, *Applied Soft Computing* 85, 2019.

Codebook transfer: the idea

1. Factorise the dense source with **MMMF** $\Rightarrow U_s, V_s, \Theta$
2. **Cluster** user and item factors
3. Cluster-level rating pattern = the **codebook** C $(k_1 \times k_2)$
4. Expand C to the target: find memberships F_1, F_2

$$\min \|[Y - F_1 C F_2^\top] \circ W\|_F^2$$



Veeramachaneni, Pujari, Padmanabhan & Kumar, *Applied Soft Computing* 85, 2019.

Does it help? (MovieLens $\rightarrow$ Book-Crossing)

Method	RMSE	MAE
MMMF (target only)	1.0017	0.6910

Does it help? (MovieLens $\rightarrow$ Book-Crossing)

Method	RMSE	MAE
MMMF (target only)	1.0017	0.6910
TRACER	1.0354	0.8421
CBT (codebook)	0.8682	0.6856

Does it help? (MovieLens $\rightarrow$ Book-Crossing)

Method	RMSE	MAE
MMMF (target only)	1.0017	0.6910
TRACER	1.0354	0.8421
CBT (codebook)	0.8682	0.6856
DSTL (ours, soft)	**0.8604**	**0.6182**

Does it help? (MovieLens $\rightarrow$ Book-Crossing)

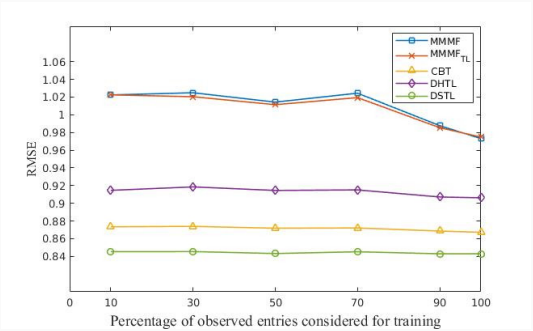
Method	RMSE	MAE
MMMF (target only)	1.0017	0.6910
TRACER	1.0354	0.8421
CBT (codebook)	0.8682	0.6856
DSTL (ours, soft)	**0.8604**	**0.6182**

Transferring *rating patterns* from a dense domain beats learning the sparse domain alone — even with **no common users or items**.

Does it help? (MovieLens $\rightarrow$ Book-Crossing)

Method	RMSE	MAE
MMMF (target only)	1.0017	0.6910
TRACER	1.0354	0.8421
CBT (codebook)	0.8682	0.6856
DSTL (ours, soft)	**0.8604**	**0.6182**

Transferring *rating patterns* from a dense domain beats learning the sparse domain alone — even with **no common users or items**.



RMSE as the fraction of observed target ratings varies

Application: Recommending Earth Observation Resources

Example: A researcher wants to study floods.

The researcher may need:

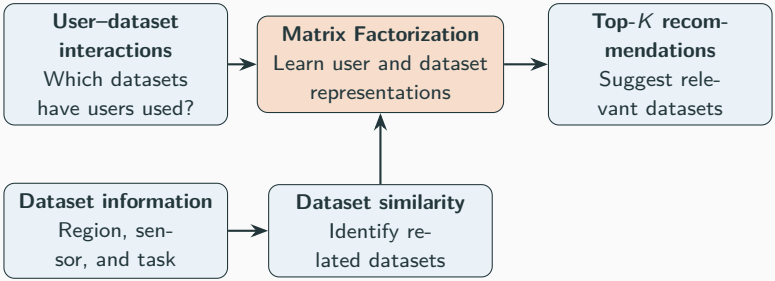
- Satellite images to identify flooded areas.
- Rainfall data to understand precipitation.
- River-level data to study rising water levels.

Finding related resources using keywords alone can be difficult.

Our idea: Recommend useful datasets, just as Netflix recommends movies.

Movie Recommendation	EO Recommendation
Users	Researchers
Movies	Datasets
Movie interactions	Dataset usage
Similar movies	Related datasets

EO-GeoMF: How Does It Work?



- Related datasets are encouraged to have similar learned representations.
- The method combines user interactions with dataset similarity.
- Recommendation quality is evaluated using Recall@10 and NDCG@10.

Accepted at InGARSS 2026

Beyond Accuracy: Research Directions in Recommender Systems

- **Fairness:** Are recommendations fair to all users and item providers?
- **Attacks and Robustness:** Can fake ratings manipulate recommendations?
- **Diversity and Novelty:** Can we recommend different and new items?
- **Explainability:** Can we explain why an item is recommended?
- **Privacy:** Can we protect users' personal information?
- **Popularity Bias:** Can we recommend useful, less popular items?
- **Cold Start:** How can we recommend items to new users?
- **Sequential Recommendation:** Can recommendations adapt to changing interests?
- **LLM-based Recommendation:** Can large language models improve recommendations?
- **Multi-objective Recommendation:** Can we balance accuracy, fairness, and diversity?

Goal: Identify a limitation, propose an improvement, and evaluate it against existing methods.

What you take home

1. Recommendation = filling a **sparse** matrix, then **ranking**

What you take home

1. Recommendation = filling a **sparse** matrix, then **ranking**
2. Always build **baselines** first — they are embarrassingly strong

What you take home

1. Recommendation = filling a **sparse** matrix, then **ranking**
2. Always build **baselines** first — they are embarrassingly strong
3. **CF:** normalise ratings (mean-centre / z-score), choose k, beware sparsity

What you take home

1. Recommendation = filling a **sparse** matrix, then **ranking**
2. Always build **baselines** first — they are embarrassingly strong
3. **CF:** normalise ratings (mean-centre / z-score), choose k, beware sparsity
4. **MF:** users and items as vectors in a shared latent space; regularise

What you take home

1. Recommendation = filling a **sparse** matrix, then **ranking**
2. Always build **baselines** first — they are embarrassingly strong
3. **CF:** normalise ratings (mean-centre / z-score), choose k, beware sparsity
4. **MF:** users and items as vectors in a shared latent space; regularise
5. Lower RMSE $\neq$ better recommendations — measure what users *see*

What you take home

1. Recommendation = filling a **sparse** matrix, then **ranking**
2. Always build **baselines** first — they are embarrassingly strong
3. **CF:** normalise ratings (mean-centre / z-score), choose k, beware sparsity
4. **MF:** users and items as vectors in a shared latent space; regularise
5. Lower RMSE $\neq$ better recommendations — measure what users *see*

Further reading

Koren, Bell & Volinsky, *Matrix Factorization Techniques for Recommender Systems*, IEEE Computer (2009)

Aggarwal, *Recommender Systems: The Textbook*, Springer (2016) — Ch. 2 (neighbourhood), Ch. 3 (latent factors)

Ricci, Rokach & Shapira (eds.), *Recommender Systems Handbook*

Thank you

Questions?